МИНИСТЕРСТВО ОБЩЕГО И ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ РФ

РЫБИНСКАЯ ГОСУДАРСТВЕННАЯ АВИАЦИОННАЯ ТЕХНОЛОГИЧЕСКАЯ АКАДЕМИЯ

В.М. КОМАРОВ

# МИКРОПРОЦЕССОРНЫЕ СИСТЕМЫ

## Программирование на ассемблере

Учебное пособие

РЫБИНСК

2004

УДК 681.322 (07)

Комаров В.М. Микропроцессорные системы: Учебное пособие. – 2 изд., перераб. и доп. – Рыбинск: РГАТА, 2004. – 163 с.

В настоящем пособии рассматривается базовый ассемблер MASM версии 5.10 фирмы Microsoft, а также режим MASM турбоассемблера TASM версии 3.2 фирмы Borland.

Пособие подготовлено на кафедре "Вычислительные системы" Рыбинской государственной авиационной технологической академии и рекомендовано Министерством общего и профессионального образования Российской Федерации в качестве учебного пособия для студентов направления 551100 "Проектирование и технология электронных средств" и специальности 220500 "Конструирование и технология электронно-вычислительных средств".

РЕЦЕНЗЕНТЫ:

Кафедра вычислительных систем Санкт-Петербургской государственной академии аэрокосмического приборостроения;

Зав.кафедрой микроэлектроники и технологии радиоаппаратуры Санкт-Петербургского государственного электротехнического университета д.т.н., профессор И.Г.Мироненко.

# СОДЕРЖАНИЕ

УКАЗАТЕЛЬ СОКРАЩЕНИЙ . . . . . 6
ВВЕДЕНИЕ . . . . . 7
1. ОРГАНИЗАЦИЯ МИКРОЭВМ . . . . . 7
1.1. Фундаментальные принципы организации вычислительных машин 7
1.2. Программное управление вычислительным процессом . . . . . 8
1.3. Структура команд вычислительных машин . . . . . 9
1.4. Структура и принцип действия микроЭВМ . . . . . 11
1.5. Базовая система команд микроЭВМ . . . . . 13
Функциональные группы команд (13). Способы адресации операндов (14).
2. ПРОГРАММИРОВАНИЕ НА ЯЗЫКЕ АССЕМБЛЕРА . . . . . 17
2.1. Архитектура микропроцессоров К1810ВМ86/К1810ВМ88 . . . . . 17
2.2. Архитектура памяти . . . . . 21
2.2.1. Логическая организация памяти . . . . . 21
Вычисление физического адреса памяти (22). Сегментная структура памяти (24).
2.3. Архитектура интерфейса . . . . . 25
2.4. Основные элементы ассемблера. Формат программной строки . . . . 26
2.5. Директивы управления сегментами . . . . . 27
2.6. Типовая структура текста программы . . . . . 30
2.7. Символические имена . . . . . 32
2.8. Константы. Их типы и директивы описания . . . . . 32
2.8.1. Числовые константы . . . . . 33
2.8.2. Символьные константы . . . . . 33
2.8.3. Поименованные константы . . . . . 34
2.8.4. Константы-выражения . . . . . 34
2.9. Переменные. Их типы и директивы описания . . . . . 35
2.9.1. Стандартные типы данных и их размещение в памяти . . . . . 35
2.9.2. Директивы описания переменных и распределение в памяти . . . 36
2.10. Атрибуты переменных и меток. Директивы, возвращающие атрибутные значения . . . . . 40
2.10.1. Атрибуты переменных . . . . . 40
2.10.2. Атрибуты меток . . . . . 41
2.11. Замена атрибутов переменных и меток . . . . . 42
2.11.1. Замена смещения . . . . . 43
2.11.2. Замена сегмента . . . . . 43
2.11.3. Замена типа . . . . . 44
2.12. Способы адресации операндов . . . . . 46
2.12.1. Адресация данных . . . . . 47
Регистровая адресация (47). Непосредственная адресация (47). Прямая адресация (47). Косвенная регистровая адресация (48).

Базовая адресация (48). Индексная адресация (49). Базово-индексная адресация (49). Относительная базово-индексная адресация (50).
2.12.2. Адресация переходов . . . . . 50
Внутрисегментный прямой переход (51). Внутрисегментный косвенный переход (51). Межсегментный прямой переход (51). Межсегментный косвенный переход (52).
2.13. Система команд . . . . . 52
2.13.1. Команды пересылки данных . . . . . 52
Команды пересылки общего назначения (53). Команды загрузки адресных значений (55). Команды обращения к стеку (57). Команды пересылки флагов (58). Команды ввода/вывода (59).
2.13.2. Арифметические команды . . . . . 59
Типы арифметических данных (60). Команды инкрементирования и декрементирования (60). Команды сложения и вычитания (61). Команды расширения знака (65). Команды умножения и деления (66). Команды десятичной коррекции (69). Сложение BCD-формате (69). Вычитание в BCD-формате (69). Сложение в ASCII-формате (70). Вычитание в ASCII-формате (72). Умножение в ASCII-формате (73). Деление в ASCII-формате (74).
2.13.3. Логические команды . . . . . 75
Команды логических операций (75). Команды сравнения (78). Команды сдвигов (79). Команды логических сдвигов (79). Команды арифметических сдвигов (81). Команды циклических сдвигов (82).
2.13.4. Команды передачи управления . . . . . 83
Команды безусловных переходов (83). Команды условных переходов (84). Команды перехода по состоянию арифметических флагов (85). Команды перехода по соотношению между числами (85). Команды перехода по состоянию регистра CX (87). Команды управления циклами (88). Команды для организации подпрограмм(90). Команды прерываний (92).
2.13.5. Команды обработки строк . . . . . 93
Строковые примитивы (94). Префиксы повторения (99).
2.13.6. Команды управления микропроцессором . . . . . 102
Команды управления флагами (102). Команды синхронизации (102).
2.13.7. Принципы кодирования команд . . . . . 103
2.13.8. Время выполнения команд . . . . . 107
2.14. Организация подпрограмм . . . . . 109
2.14.1. Директивы описания подпрограмм . . . . . 109

2.14.2. Передача параметров в подпрограммы . . . . . 110
Передача параметров через регистры (111). Передача параметров через общие ячейки памяти (112). Передача параметров через зоны параметров (113). Передача параметров через стек (115).
2.15. Организация системы прерываний . . . . . 118
2.15.1. Внешние прерывания . . . . . 119
2.15.2. Внутренние прерывания . . . . . 119
2.15.3. Обслуживание прерываний . . . . . 120
2.16. Нестандартные типы данных . . . . . 123
2.16.1. Структуры . . . . . 123
2.16.2. Записи . . . . . 127
2.17. Макрокоманды . . . . . 132
2.17.1. Макроопределения и макровызовы . . . . . 132
2.17.2. Сравнение макрокоманд и подпрограмм . . . . . 135
2.17.3. Макрокоманды повторения . . . . . 136
2.17.4. Библиотеки макроопределений . . . . . 137
3. ТЕХНОЛОГИЯ ПОДГОТОВКИ ПРОГРАММ . . . . . 139
3.1. Создание исходного файла программ . . . . . 139
3.2. Трансляция программы . . . . . 139
3.3. Компоновка программы . . . . . 142
3.4. Отладка программы . . . . . 143
4. МОДУЛЬНОЕ ПРОГРАММИРОВАНИЕ . . . . . 144
4.1. Методы и проблемы модульного программирования . . . . . 144
4.1.1. Реализация модульности за счет библиотек макроопределений . 144
4.1.2. Реализация модульности за счет связывания объектных модулей . . . . . 144
Объединение логических сегментов (146). Описание межмодульных обращений (153).
4.2. Библиотеки объектных модулей . . . . . 155
4.2.1. Создание библиотек объектных модулей и операций над ними . 156
Команды библиотекаря (156).
4.2.2. Использование библиотек объектных модулей . . . . . 158
ЗАКЛЮЧЕНИЕ . . . . . 160
СПИСОК РЕКОМЕНДУЕМЫХ ИСТОЧНИКОВ . . . . . 161
ПРИЛОЖЕНИЕ. Система команд микропроцессоров К1810ВМ86/К1810ВМ88 . . . . . 162

# УКАЗАТЕЛЬ СОКРАЩЕНИЙ

АЛУ – арифметико-логическое устройство
АС – аппаратные средства
АЦП – аналого-цифровой преобразователь
БИС – большая интегральная схема
БПЗС – байт префикса замены сегмента
ГСА – графическая схема алгоритма
ИМС – интегральная микросхема
ЗУ – запоминающее устройство
КОП – код операции
КОПА – код операции и адресации
КП – команды пересылки
КУВОП – код управления выполнением операции
МП – микропроцессор
МПК – микропроцессорный комплект
МПС – микропроцессорная система
ОБ – операционный блок
ОЗУ – оперативное запоминающее устройство
ОК – операционные команды
ПДП – прямой доступ в память
ПЗУ – постоянное запоминающее устройство
ПК – персональный компьютер
ППЗУ – программируемое постоянное запоминающее устройство
ППИ – программируемый параллельный интерфейс
ПС – программные средства
РОН – регистр общего назначения
РУ – регистр управления
СБИС – сверхбольшая интегральная схема
СК – система команд
УА – управляющий автомат
УБ – управляющий блок
УВВ – устройства ввода/вывода
УК – управляющие команды
УС – управляющие сигналы
ЦАП – цифро-аналоговый преобразователь
ША – шина адреса
ШБ – шинный буфер
ШД – шина данных
ШУ – шина управления
ЭЛТ – электронно-лучевая трубка
ЭВМ – электронная вычислительная машина

# ВВЕДЕНИЕ

Программирование представляет собой процесс записи алгоритма решения поставленной задачи с помощью операторов используемого языка. В отличие от языков высокого уровня ассемблер является языком низкого уровня программирования и имеет ярко выраженный машинно-ориентированный характер. Это позволяет программисту использовать все ресурсы вычислительной системы и обеспечивает максимально высокую эффективность создаваемых программ с точки зрения их быстродействия и объема памяти.

Однако для успешного программирования на ассемблере необходимо четко представлять аппаратные средства используемой вычислительной системы, что требует рассмотрения вопросов ее организации. В настоящее время подавляющее большинство вычислительных систем строится на базе различных микропроцессоров, и поэтому их основу составляют различные микроЭВМ. Большинство современных микроЭВМ организованы по единым принципам, предложенным американским ученым фон Нейманом.

# 1. ОРГАНИЗАЦИЯ МИКРОЭВМ

## 1.1. ФУНДАМЕНТАЛЬНЫЕ ПРИНЦИПЫ ОРГАНИЗАЦИИ ВЫЧИСЛИТЕЛЬНЫХ МАШИН

Принципы организации вычислительных машин были сформулированы фон Нейманом в 1945 году. Он выделил пять фундаментальных принципов.

**1. Принцип использования двоичной системы счисления для представления информации.** Двоичная система обеспечивает помехоустойчивое хранение и передачу информации, экономичное кодирование и обработку данных.

**2. Принцип программного управления**. Обработка информации осуществляется по шагам, каждый из которых выполняется отдельной командой. Команды выполняются последовательно во времени. Совокупность шагов по обработке данных называется вычислительным процессом, а совокупность команд – программой.

**3. Принцип условного перехода в программе.** Этот принцип обеспечивает возможность перехода к выполнению тех или иных действий по обработке информации в зависимости от признаков полученного промежуточного результата. Это позволяет реализовать ветвление вычислений, циклические и итерационные вычисления, что уменьшает длину программы за счет многократного использования одних и тех же команд.

**4. Принцип хранимой программы.** Команды хранятся в той же памяти, что и данные, и ничем не отличаются от них по внешнему виду.

**5. Принцип иерархичности памяти.** ЭВМ должна иметь различные типы запоминающих устройств: внутреннее быстродействующее запоминающее устройство ограниченной емкости и внешние запоминающие устройства меньшего быстродействия, но громадного объема.

В микроЭВМ, работающих в составе МПС, как правило, представлены первые четыре принципа фон Неймана. Соблюдение пятого принципа для них нехарактерно, т. к. в связи со специализацией функций сложность задач сравнительно невелика, и требующийся незначительный объем памяти полностью удовлетворяется внутренним быстродействующим запоминающим устройством.

## 1.2. Программное управление вычислительным процессом

В соответствии с принципами фон Неймана функционирование микроЭВМ сводится к последовательному выполнению команд программы, расположенной в памяти. Этот процесс может быть изображен алгоритмом интерпретации команд, приведенным на рис. 1.1,а.

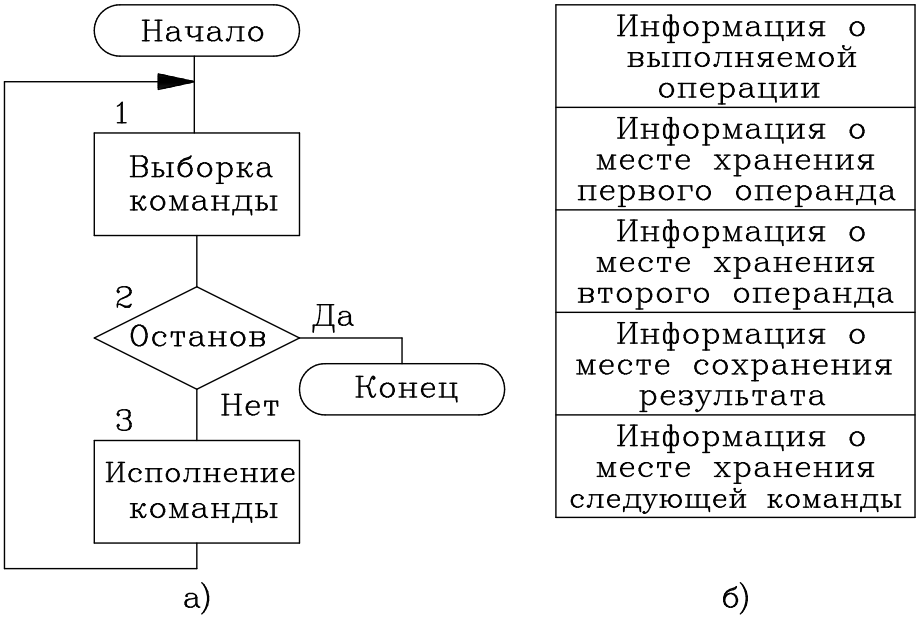


Рис. 1.1.Программное управление вычислительным процессом:
а) цикл интерпретации команд; б) обобщенная структура команды

Характерным для этого алгоритма является циклическое повторение фаз выборки и исполнения команды до появления команды останова. При этом на фазе выборки команды обязательно обращение к памяти, а на фазе ее исполнения действия зависят от типа команды.

С математической точки зрения для функциональной полноты оптимальной системы команд она должна содержать команды, выполняющие двухместные операции. Иначе функции многих переменных будут вообще нереализуемы. В этом случае обобщенная структура команды должна иметь вид, приве-

денный на рис. 1.1,б. Однако, такая структура очень сложна для реализации. С целью упрощения команды принимается ряд соглашений, которые выполняются по умолчанию. В этом случае некоторые поля команды могут отсутствовать.

## 1.3. Структура команд вычислительных машин

Вся информация, содержащаяся в команде, должна быть закодирована двоичными кодами. Информация о выполняемой операции представляется кодом операции (КОП). Вся остальная информация носит адресный характер и, в простейшем случае, представляется соответствующими адресами. Количество адресов в команде называется адресностью команды. Следовательно, обобщенная команда является четырехадресной (см. рис. 1.1,б). Чем меньше адресность команды, тем проще ее реализация. Для уменьшения адресности приняты четыре соглашения.

**1. Соглашение о естественном порядке следования команд.**

В соответствии с этим соглашением коды команд, выполняемых друг за другом, находятся в последовательных ячейках памяти. Поэтому адрес следующей команды может быть получен путем инкрементирования текущего адреса. Очевидно, что это соглашение требует, чтобы в составе вычислительной машины был специальный узел, способный хранить и инкрементировать адрес команды. Такой узел был назван программным счетчиком PC (Program Counter). При этом соглашении команда становится трехадресной.

Однако естественное следование команд не обеспечивает решения любой задачи. Для этого оно должно нарушаться в некоторых точках, называемых точками ветвления программы. Для реализации ветвлений необходимо ввести специальные управляющие команды, изменяющие естественное следование команд. Управляющие команды могут быть безусловными или условными. В качестве условий ветвления должны использоваться признаки результата. Таким образом, при переходе к трехадресной системе команд, возникают два типа команд: операционные (ОК) и управляющие (УК), структура которых приведена на рис. 1.2,а. В зависимости от значения признака результата, указанного в поле условия управляющей команды, следующая команда будет выбираться по адресу 1 или адресу 2.

**2. Соглашение о совмещении места хранения одного из операндов и результата.**

При этом соглашении выполнение команды приведет к уничтожению одного из операндов, т. к. на его место запишется результат. Поэтому, если этот операнд потребуется в дальнейшем, то перед выполнением такой команды его необходимо скопировать. Для этой цели необходимы специальные команды пересылки (КП), которые позволяют пересылать содержимое из ячейки-источника информации в ячейку-приемник без разрушения источника. Следовательно, двухадресная система будет включать в себя три разновидности команд, показанных на рис. 1.2,б. При этом в управляющих командах остается

один адрес, который используется при истинности условия. При ложном условии естественное следование команд не нарушается.

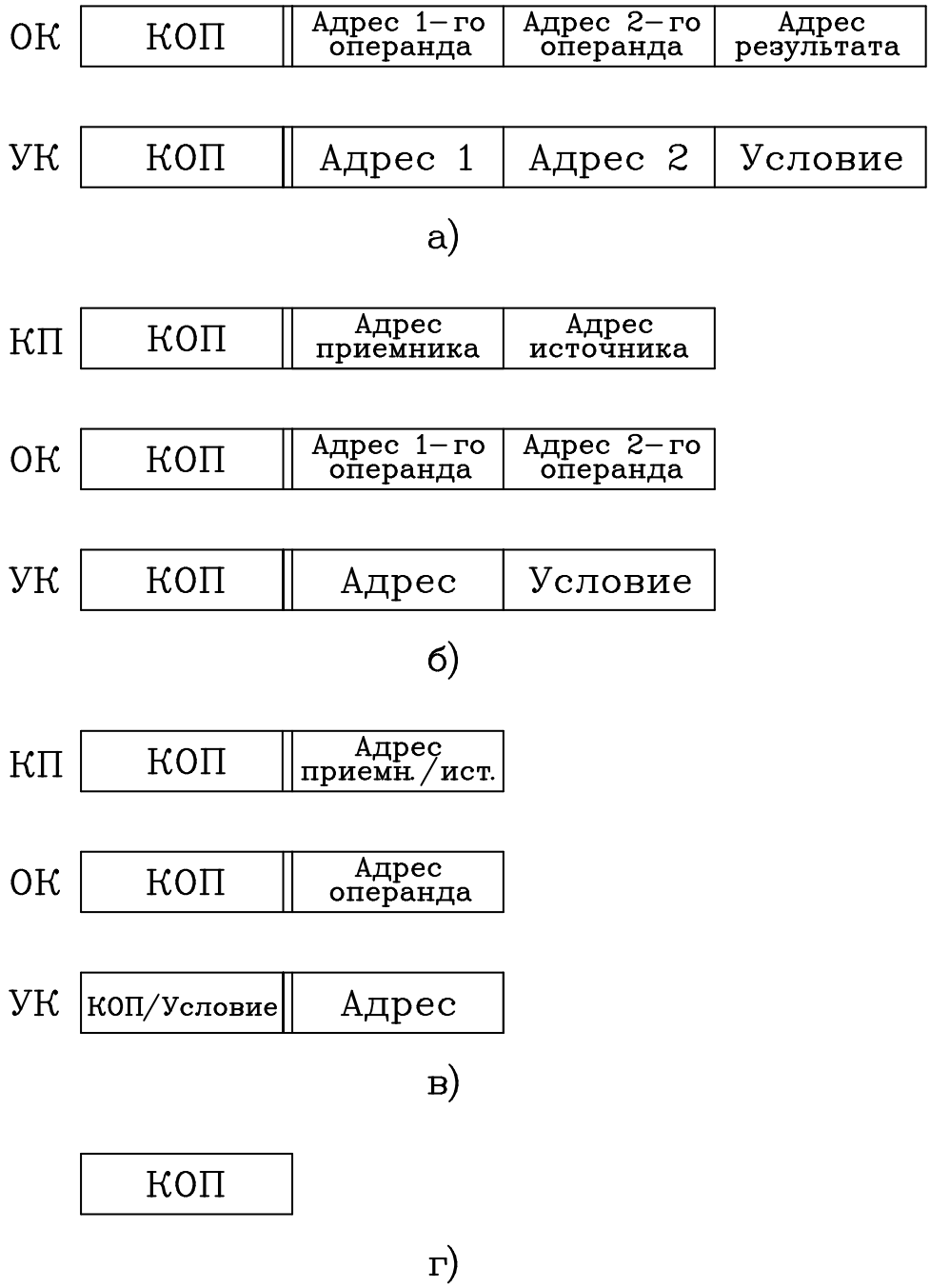


Рис. 1.2. Структура команд:
а) трехадресные команды; б) двухадресные команды;
в) одноадресные команды; г) безадресные команды

**3. Соглашение о постоянном месте хранения одного из операндов и результата.**

Это соглашение позволяет перейти к одноадресной системе команд (рис. 1.2,в). При этом последовательное выполнение операции сложения приве-

дет к накоплению суммы в месте постоянного хранения одного из операндов и результата. В связи с этим узел, в котором до операции находится операнд, а после операции – результат, называется аккумулятором (accumulator – накопитель).

**4.Соглашение о стековой организации памяти.**

Это соглашение позволяет отказаться от последнего поля адреса и превратить команду в безадресную (рис. 1.2,г). Однако для его реализации необходима специальная организация памяти.

Действительно, формирование адреса операнда по умолчанию возможно лишь в том случае, если к ячейкам некоторой области памяти разрешить только последовательный доступ. Такая область памяти называется стеком. Стек работает по принципу LIFO (Last Input – First Output – последний вошел – первый вышел). Адрес очередного элемента стека может формироваться путем декрементирования или инкрементирования текущего адреса. Очевидно, что для хранения этого адреса необходим специальный узел, называемый указателем стека SP(Stack Pointer).

Ячейка памяти, адресуемая указателем стека, называется верхушкой стека. Перед записью информации в стек указатель стека автоматически декрементируется, выбирая очередную свободную ячейку. В результате этого стек расширяется в сторону меньших адресов. При чтении информации из стека указатель стека автоматически инкрементируется после выполнения операции чтения, обращаясь к очередной ячейке, из которой еще не было чтения. Таким образом, указатель стека всегда указывает на последнюю занятую ячейку стека.

Стек может быть организован в общей памяти, причем его текущее положение определяется только начальной загрузкой указателя стека. Ячейка, соответствующая начальному значению SP, называется дном стека.

В систему команд современных микроЭВМ входят команды различной адресности: двух-, одно- и безадресные команды.

## 1.4. Структура и принцип действия микроЭВМ

Типовая структура микроЭВМ включает в себя три устройства: процессор, память и интерфейс (рис. 1.3).

Память микроЭВМ предназначена для хранения программы, описывающей процесс обработки информации, а также для хранения различных промежуточных данных. Каждый элемент информации, закодированный двоичным кодом, находится в отдельной ячейке. Обращение к конкретной ячейке памяти осуществляется по двоичному коду номера ячейки, являющемуся ее адресом. При этом возможны два типа обращения: чтение или запись информации. Тип обращения к выбранной ячейке памяти указывается с помощью управляющих сигналов чтения памяти MEMR (MEMory Read) или записи в память MEMW (MEMory Write).

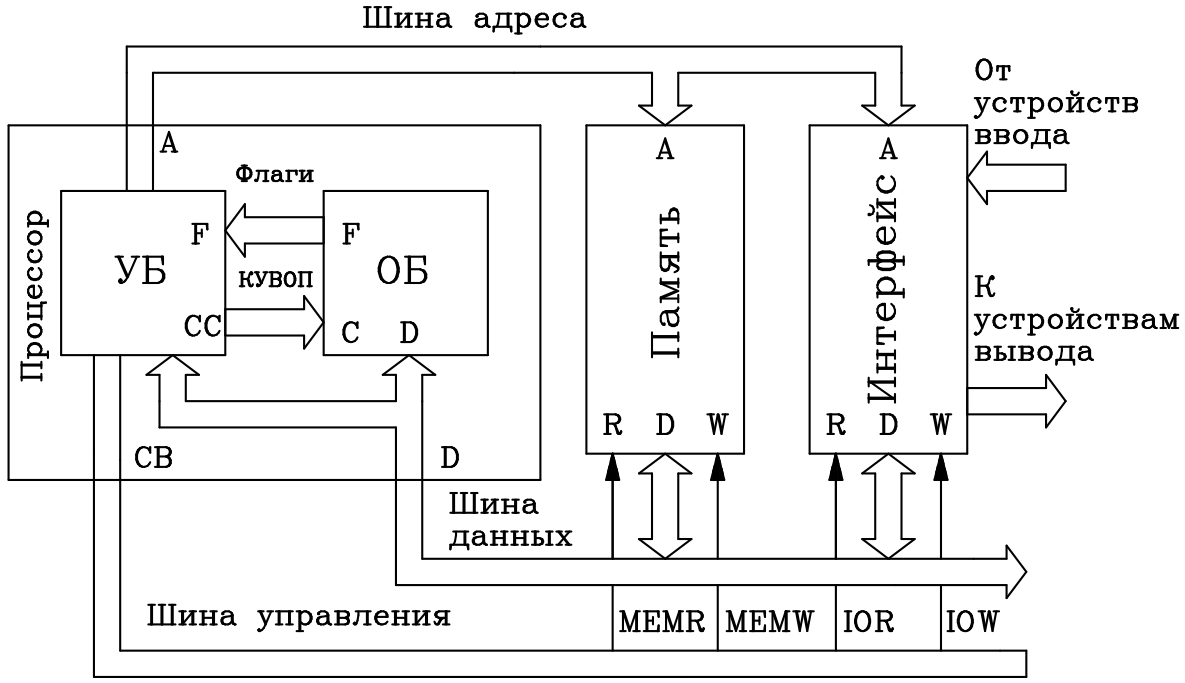


Рис. 1.3. Типовая структура микроЭВМ

Процессор микроЭВМ обеспечивает последовательное выполнение элементарных шагов по обработке данных, т. е. реализацию вычислительного процесса. Для осуществления этой функции он состоит из двух блоков: управляющего и операционного. Операционный блок (ОБ) выполняет собственно элементарные шаги – операции над данными, а управляющий блок (УБ) обеспечивает выполнение этих операций в заданной последовательности.

Для выполнения некоторого действия процессор считывает очередную команду из памяти. Для этого его УБ выдает на шину адреса (ША) содержимое программного счетчика, содержащего адрес этой команды, и генерирует управляющий сигнал чтения MEMR. В результате команда извлекается из памяти и через шину данных (ШД) поступает в процессор. УБ определяет тип команды и начинает ее выполнение.

Если это операционная команда, то в ОБ направляется последовательность кодов управления выполнением операции (КУВОП), заданной КОП команды. В результате ОБ выполняет предписанную операцию. Если для ее выполнения необходим еще и операнд из памяти, то он считывается из нее аналогично команде. При этом адрес операнда, в простейшем случае содержащийся в команде (см. рис. 1.2), выдается на ША в сопровождении сигнала MEMR. В результате операнд извлекается из памяти и поступает в ОБ процессора, который формирует искомый результат.

Для записи в память результата операции, полученного в ОБ, УБ выдает на ША адрес ячейки назначения, на ШД – записываемый результат и генерирует управляющий сигнал записи MEMW.

Для реализации принципа условного перехода ОБ одновременно с результатом операции формирует его признаки, называемые флагами, и направ-

ляет их в УБ. После выполнения операционной команды УБ выдает на ША адрес очередной команды и т. д.

При выполнении управляющей команды УБ сразу определяет адрес следующей команды, выдает его на ША и т. д.

Интерфейс микроЭВМ предназначен для сопряжения устройств ввода/вывода (УВВ) с ее процессором или памятью. Обращение к УВВ осуществляется аналогично обращению к памяти. Адрес, выдаваемый процессором на ША, определяет конкретное УВВ, к которому осуществляется обращение. При этом также возможны два типа обращения: чтение или запись информации в УВВ. Тип обращения к выбранному УВВ указывается с помощью управляющих сигналов ввода IOR (Input/Output Read) или вывода IOW (Input/Output Write).

Поскольку обращение к ячейкам памяти и УВВ осуществляется с помощью различных управляющих сигналов, то они могут иметь одинаковые адреса.

## 1.5. Базовая система команд микроЭВМ

Система команд (СК) любой микроЭВМ является функционально полной, т. е. позволяет решать самые различные задачи. Однако ее специфика накладывает отпечаток на параметры программ. Чем более конкретная микроЭВМ ориентирована на решение некоторого класса задач, тем более оптимальными будут соответствующие программы.

Все множество команд современных микроЭВМ может быть классифицировано по различным признакам. Основными из них являются функциональное назначение команды и способы адресации операндов, допускаемые командой.

### Функциональные группы команд

Главным устройством любой микроЭВМ является процессор, в основе которого находится МП. Фактически, говоря о СК микроЭВМ, говорят о СК МП. По функциональному назначению все команды МП делятся на следующие группы:

1) команды пересылки данных;
2) арифметические команды;
3) логические команды;
4) команды передачи управления;
5) команды управления микропроцессором.

Для обозначения операций, реализуемых командами, принято использовать соответствующие англоязычные эквиваленты.

Команды пересылки обеспечивают пересылку данных между различными элементами микроЭВМ. Как правило, они реализуют следующие операции: переслать – MOVE, загрузить (в процессор) – LOAD, запомнить (в памяти) – STORE, обменять – EXCHAHGE, включить в стек – PUSH, извлечь из стека – POP, ввести (из УВВ) – INPUT, вывести (в УВВ) – OUTPUT.

Арифметические команды обеспечивают выполнение арифметических операций над данными. Основными из них являются следующие операции: увеличить на 1 – INCREMENT, уменьшить на 1 – DECREMENT, сложить – ADD, вычесть – SUBTRACT, умножить – MULTIPLICATION, разделить – DIVIDE, скорректировать десятичный результат – DECIMAL ADJUST.

Логические команды обеспечивают выполнение логических операций над данными: логическое И – AND, логическое ИЛИ – OR, исключающее ИЛИ – EXCLUSIVE OR, инвертировать – NOT, сравнить – COMPARE, сдвинуть – ROTATE – и т.п.

Команды передачи управления обеспечивают ветвление вычислительного процесса в программе. Как правило, они выполняют следующие операции: перейти – JUMP, вызвать (подпрограмму) – CALL, вернуться (из подпрограммы) – RETURN, зациклить – LOOP.

Команды управления обеспечивают управление работой МП. При этом они реализуют следующие операции: остановить (МП) – HALT, установить (флаг) – SET, очистить (флаг) – CLEAN, пропустить операцию – NO OPERATION.

Для обозначения различных команд используются аббревиатуры англоязычных названий реализуемых ими операций: например, MOV, XCHG, IN, OUT, INC, DEC, SUB, MUL, XOR, JMP, RET, HLT, NOP и т.п.

## Способы адресации операндов

Для выполнения некоторых действий в команде должны быть определены операнды, по отношению к которым выполняются эти действия. Местоположение операндов может быть задано различными способами. Эти способы называются способами адресации операндов.

Для правильной интерпретации адресной информации, содержащейся в команде, КОП содержит и некоторые адресные сведения и поэтому может называться кодом операции и адресации (КОПА).

В современных микроЭВМ используются разнообразные способы адресации, упрощающие организацию и обработку сложных структур данных. Основными из них являются:

– прямая адресация;

– непосредственная адресация;

– косвенная адресация;

– прямая регистровая адресация;

– косвенная регистровая адресация.

Прямая адресация является простейшим способом адресации памяти. В этом случае адресное поле команды содержит адрес ячейки памяти, в которой находится операнд (рис. 1.4,а). Этот способ удобен для адресации простых переменных.

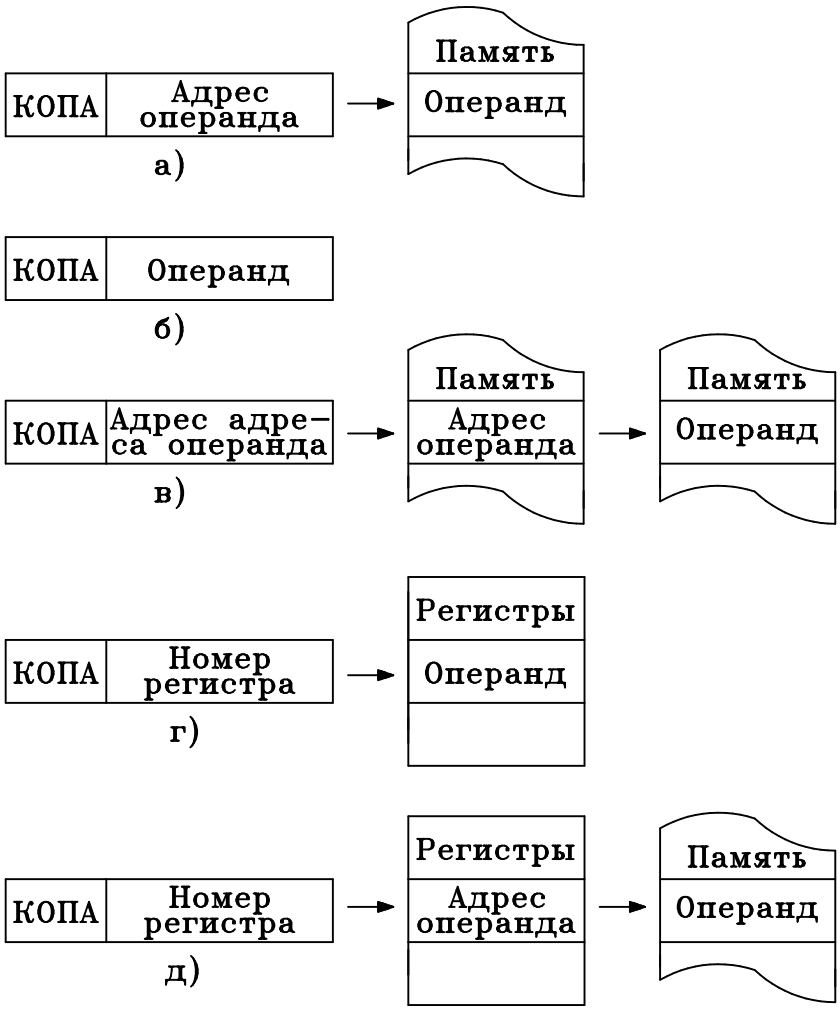


Рис. 1.4. Адресация операндов в командах:
а) прямая; б) непосредственная; в) косвенная;
г) прямая регистровая; д) косвенная регистровая

При непосредственной адресации адресное поле команды содержит не адрес, а непосредственно сам операнд (рис. 1.4,б). Этот способ удобен для задания констант, используемых в программе.

При косвенной адресации адресное поле команды содержит адрес адреса операнда (рис. 1.4,в). Этот способ используется в циклических программах, в которых при каждом проходе тела цикла обрабатывается очередной элемент данных. Для доступа к очередному элементу необходимо в конце каждого прохода модифицировать адрес операнда. Поскольку местоположение этого адреса не изменяется, то он может модифицироваться одной и той же командой. Это обеспечивает обработку различных элементов данных в разных проходах тела цикла.

Недостатком хранения промежуточных данных в памяти является значительное время обращения к ней. Для повышения производительности микро-ЭВМ МП содержит сверхбыстродействующую память, выполненную непосредственно на его кристалле. Эта память представляет собой набор регистров.

Регистры МП обычно используются для выполнения самых различных функций и называются регистрами общего назначения (РОН). Для обращения к РОН в адресном поле команды указывается номер соответствующего регистра. Адресация с использованием РОН называется регистровой адресацией.

При прямой регистровой адресации адресное поле команды содержит номер регистра, в котором находится операнд (рис. 1.4,г).

При косвенной регистровой адресации адресное поле команды содержит номер регистра, в котором находится адрес операнда (рис. 1.4,д). Поскольку количество регистров невелико, то код их номера имеет малую разрядность, что сокращает длину команд.

Очевидно, что в командах с двумя операндами каждый операнд может быть адресован любым способом, и подобные команды могут одновременно использовать два различных способа адресации.

# 2. ПРОГРАММИРОВАНИЕ НА ЯЗЫКЕ АССЕМБЛЕРА

В настоящем пособии рассматривается базовый ассемблер MASM версии 5.10 фирмы Microsoft [1], а также режим MASM турбоассемблера TASM версии 3.2 фирмы Borland [2].

Ассемблер является основным языком программирования для МПС, т. к. обеспечивает максимальную эффективность создаваемых программ с точки зрения их быстродействия и объема.

Ассемблер – это машинно-ориентированный язык, позволяющий использовать все ресурсы системы. Однако для успешного программирования на нем необходимо четко представлять аппаратные средства используемой микроЭВМ, т. е. ту материальную среду, в которой протекает процесс решения задачи. Поскольку задача решается программными средствами, то в эту среду входят только программно доступные элементы аппаратных средств, т. е. элементы, доступные с помощью системы команд.

Совокупность программно доступных элементов системы называется ее архитектурой. Архитектура микроЭВМ включает в себя архитектуры микропроцессора, памяти и интерфейса. При этом архитектура процессора полностью совпадает с архитектурой МП, который был использован для его построения.

## 2.1. Архитектура микропроцессоров К1810ВМ86/К1810ВМ88

Архитектура МП ВМ86/ВМ88 приведена на рис. 2.1. Она включает в свой состав 14 программно доступных регистров. Все они делятся на две группы:

1) регистры общего назначения (РОН);

2) сегментные регистры.

В свою очередь РОН делятся на регистры данных и регистры – указатели адреса. Отличительной особенностью регистров данных является возможность их использования как в виде полных 16-битных регистров AX, BX, CX, DX, так и в виде частичных 8-битных регистров AH, AL, BH, BL, CH, CL, DH, DL. Это обеспечивает возможность эффективной обработки как однобайтных, так и двухбайтных данных. Регистры данных предназначены для хранения данных, и они единообразно участвуют во всех арифметических и логических операциях.

Регистры-указатели адреса SP, BP, SI, DI могут использоваться только как полные 16-битные регистры. Они предназначены в основном для хранения адресной информации. Однако эти регистры могут участвовать и в арифметических и логических операциях, что позволяет отнести их к РОН.

Определение регистров данных и регистров-указателей адреса как регистров общего назначения в МП ВМ86/ВМ88 весьма условно. Многие команды рассматривают различные регистры этой группы как специализированные регистры, что обуславливает их функциональную неравнозначность. Основная специализация регистров этой группы указана на рис. 2.1, а полные сведения о ней – в табл. 2.1.

Рис. 2.1. Архитектура МП ВМ86/ВМ88

Специализация внутренних регистров МП требует, чтобы программист или компилятор языка высокого уровня внимательно распределял и тщательно следил за использованием конкретных регистров. Однако это дает возможность их неявного использования соответствующими командами, что позволяет получить более короткий машинный код, чем в случае идеальных РОН.

Сегментные регистры CS, DS, ES, SS служат для поддержки далее рассмотренного механизма сегментации памяти, и каждый из них обеспечивает хранение базового (начального) адреса segment соответствующего сегмента.

Указатель команд IP обеспечивает хранение смещения offset очередной команды, подлежащей считыванию из текущего сегмента программного кода.

Таблица 2.1

Специализация РОН

| Регистр | Специализированные функции |
|---|---|
| AX | Умножение, деление, ввод-вывод слов |
| AH | Умножение и деление байт |
| AL | Умножение, деление, ввод-вывод байт, десятичная арифметика, табличное преобразование байта |
| BX | Базовый регистр адреса данных, табличное преобразование |
| CX | Счетчик циклов в командах управления циклами и обработки цепочек |
| CL | Счетчик циклов в командах сдвигов |
| DX | Умножение и деление слов, косвенный ввод-вывод |
| SP | Указатель стека |
| BP | Базовый регистр адреса стека |
| SI | Индексный регистр, операции с цепочками (строками) |
| DI | Индексный регистр, операции с цепочками (строками) |

Регистр флагов служит для хранения признаков состояния МП. Все флаги делятся на две группы:

1) арифметические флаги;

2) управляющие флаги.

Арифметические флаги характеризуют признаки результата операции, полученного в АЛУ. К ним относятся:

CF – флаг переноса, фиксирующий значение переноса (заема), возникающего при сложении (вычитании) байтов или слов, а также значение выдвигаемого бита при сдвиге операндов;

PF – флаг четности (паритета), фиксирующий наличие четного количества единиц в младшем байте результата;

AF – флаг вспомогательного переноса, фиксирующий перенос (заем) из младшей тетрады (бита D3) регистра AL при сложении (вычитании) двоично-десятичных чисел, обрабатываемых только побайтно;

ZF – флаг нуля, фиксирующий получение нулевого результата;

SF – флаг знака, дублирующий значение старшего бита результата, который при использовании дополнительного кода соответствует знаку числа;

OF – флаг переполнения, фиксирующий переполнение разрядной сетки МП при сложении и вычитании знаковых чисел. Переполнение приводит к потере старшего бита результата. Например, для 4-разрядной машины имеем:

| 1) $(+3)+(+2)=+5$ | 2) $(+5)+(+4)=+9$ | 3) $(-3)+(-2)=-5$ | 4) $(-5)+(-4)=-9$ |
|---|---|---|---|
| $+\,0.011\ (+3)$ | $+\,0.101\ (+5)$ | $+\,1.101\ (-3)$ | $+\,1.011\ (-5)$ |
| $0.010\ (+2)$ | $0.100\ (+4)$ | $1.110\ (-2)$ | $1.100\ (-4)$ |
| $0.101=(+5)$ | $1.001=(-7)$ | $1.011=(-5)$ | $0.111=(+7)$ |
| OF = 0 | OF = 1 | OF = 0 | OF = 1. |

Поскольку переполнение полностью искажает результат, то имеется специальная команда прерывания по переполнению, с помощью которой при OF = 1 инициируется программное прерывание, при обработке которого анализируется возникшая ситуация и принимаются все необходимые решения.

Кодирование арифметических флагов имеет вид:

CF = 0 – при отсутствии переноса (заема) из старшего разряда;
CF = 1 – при наличии переноса (заема) из старшего разряда;
PF = 0 – при нечетном количестве единиц в младшем байте результата;
PF = 1 – при четном количестве единиц в младшем байте результата;
AF = 0 – при отсутствии переноса из старшего разряда младшей тетрады AL (разряда D3);
AF = 1 – при наличии переноса из старшего разряда младшей тетрады AL (разряда D3);
ZF = 0 – при ненулевом результате;
ZF = 1 – при ненулевом результате;
SF = 0 – при положительном или нулевом результате;
SF = 1 – при отрицательном результате;
OF = 0 – при отсутствии переполнения разрядной сетки в операциях над знаковыми числами;
OF = 1 – при наличии переполнения разрядной сетки в операциях над знаковыми числами;

К управляющим флагам относятся:

DF – флаг направления, определяющий направление обработки данных в виде цепочек символов. Если DF = 0, то цепочка обрабатывается от первого элемента (с наименьшим адресом) к последнему (с наибольшим адресом). При DF = 1 цепочка обрабатывается от последнего элемента (с наибольшим адресом) к первому (с наименьшим адресом). Выбор направления обработки имеет смысл при пересылке перекрывающихся цепочек. Если начальный адрес приемника меньше начального адреса источника, то пересылку следует начинать с первого элемента. Если же начальный адрес приемника больше начального адреса источника, то пересылка осуществляется с последнего элемента;

IF – флаг разрешения маскируемого прерывания. При IF = 0 маскируемые прерывания запрещены, и МП игнорирует все поступающие на него запросы. При IF = 1 маскируемые прерывания разрешены, и МП обслуживает их при наличии запросов;

TF – флаг трассировки, управляющий отладочным режимом работы МП. При TF = 0 МП выполняет команды основной программы обычным образом, считывая их друг за другом из памяти. При TF = 1 после выполнения каждой команды основной программы автоматически инициируется внутреннее прерывание пошагового режима, в процессе обработки которого отображается содержимое регистров МП и важнейших ячеек памяти. Эта информация позволяет оценить текущее состояние системы, что способствует упрощению отладки программ.

Для управления флагами DF и IF существуют специальные команды. Команды управления флагом TF отсутствуют, и управление им осуществляется через память с помощью логических команд.

При сбросе МП все флаги принимают нулевые значения.

## 2.2. Архитектура памяти

Архитектура памяти (рис.2.2) представляет собой совокупность смежных ячеек. Разрядность всех ячеек одинакова и составляет 8 битов (D7 – D0). В каждой ячейке хранится некоторое число. Это число может быть прочитано из ячейки, а также в нее может быть записано другое число.

Положение каждой ячейки памяти характеризуется ее физическим адресом. Общее количество ячеек определяется разрядностью адреса. В микроЭВМ на базе МП ВМ86/ВМ88 разрядность адреса равна 20, что обеспечивает адресацию $2^{20}$ ячеек или 1 Мбайт. Таким образом, физический адрес ячейки представляет собой 20-разрядное двоичное или 5-разрядное шестнадцатиричное число (см. рис. 2.2).

| Адрес | Значение D7 D0 |
|---|---|
| 00000h | |
| 00001h | |
| 00002h | |
| | |
| FFFFEh | |
| FFFFFh | |
| | |

Рис. 2.2. Архитектура памяти

### 2.2.1. Логическая организация памяти

Для обеспечения легкой переместимости программ в МП ВМ86/ВМ88 используется сегментация памяти. При этом вся память объемом 1 Мбайт рассматривается как совокупность физических сегментов. Физический сегмент представляет собой область смежных ячеек памяти объемом 64 Кбайт. Положение сегмента в адресном пространстве определяется базовым или начальным адресом сегмента segment. Сегментные регистры предназначены для хранения базовых адресов сегментов. Эти адреса характеризуют положение сегментов относительно начала адресного пространства. Положение каждой ячейки в сегменте характеризуется ее смещением offset относительно начала сегмента (рис. 2.3).

При наличии четырех сегментных регистров МП может обращаться к четырем сегментам памяти. Все сегментные регистры имеют специализированное назначение, что определяет специализацию адресуемых ими сегментов памяти, а именно:

CS – сегмент программного кода CS;

DS – основной сегмент данных DS;

ES – дополнительный сегмент данных ES;

SS – сегмент стека SS.

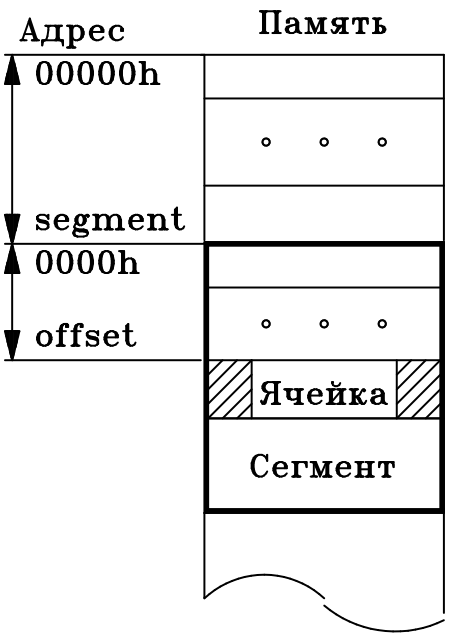


Рис. 2.3. Сегмент в памяти

Следовательно, с логической точки зрения положение ячейки может быть описано базовым адресом сегмента segment и смещением относительно начала этого сегмента offset. Эта пара чисел называется логическим адресом ячейки памяти и записывается в виде segment:offset. База сегмента segment всегда хранится в сегментном регистре, а смещение offset чаще всего берется из регистров указателей адреса IP, SP, BX, BP, SI, DI, либо вычисляется в соответствии со способом адресации операндов, указанным в команде.

Таким образом, логический адрес состоит из двух беззнаковых 16-битных значений и является двойным словом.

Программа всегда оперирует логическими адресами. При обращении к памяти шинный интерфейс МП из заданного логического адреса ячейки памяти вычисляет ее физический адрес, который и выдается на шину адреса.

### Вычисление физического адреса памяти

Физический адрес представляет собой 20-битное число в диапазоне 0...FFFFFh, которое однозначно определяет положение каждого байта в пространстве памяти объемом 1 Мбайт.

При вычислении физического адреса база сегмента segment сдвигается влево на 4 бита и складывается со смещением offset в соответствии со схемой, приведенной на рис. 2.4.

Перенос из старшего разряда, возникающий при суммировании, игнорируется. Это приводит к кольцевой организации памяти, при которой за ячейкой с максимальным адресом FFFFFh следует ячейка с нулевым адресом. Аналогичную кольцевую организацию имеет и каждый сегмент.

Кроме того, на рис. 2.4 указаны различные источники компонент логического адреса, которые могут использоваться для вычисления физического адреса. При этом обозначение EA соответствует эффективному адресу, вычисляемому в соответствии с указанным в команде способом адресации операндов.

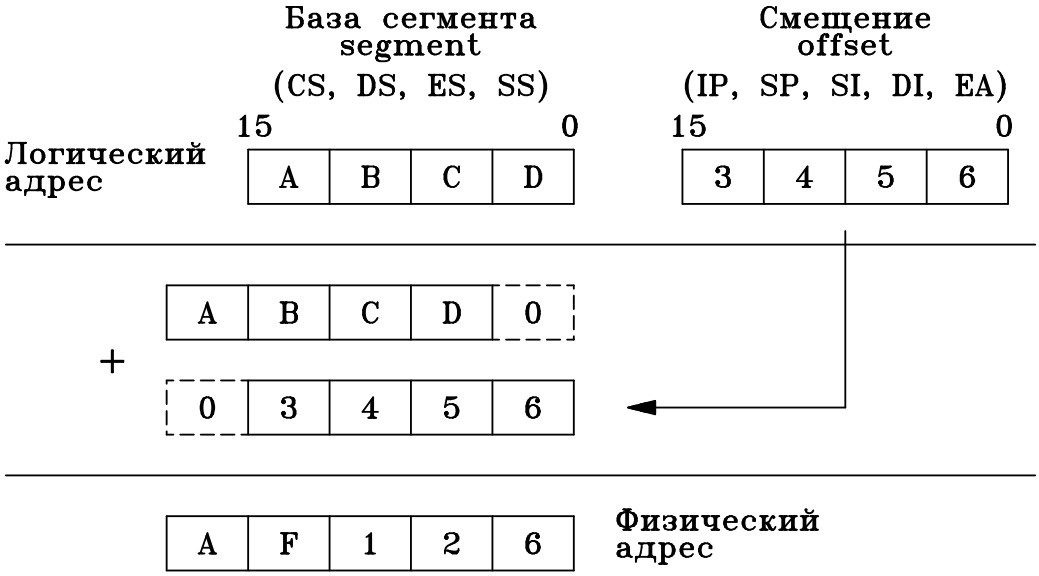


Рис. 2.4. Вычисление физического адреса

При различных типах обращения к памяти компоненты логического адреса выбираются из различных источников. Допустимые варианты источников приведены в табл. 2.2.

Таблица 2.2

Источники компонент логического адреса.

| Тип обращения к памяти | Сегмент (по умолчанию) | Вариант сегмента | Смещение |
|---|---|---|---|
| Выборка команды | CS | Нет | IP |
| Стековая операция | SS | Нет | SP |
| Обращение к данным | DS | CS, ES, SS | EA |
| Обращение к цепочке-источнику | DS | CS, ES, SS | SI |
| Обращение к цепочке-приемнику | ES | Нет | DI |
| Обращение к данным через регистр BP | SS | CS, DS, ES | EA |

Из анализа этой таблицы следует, что команды всегда выбираются из текущего сегмента программного кода по логическому адресу CS:IP, стековые операции всегда выполняются в текущем сегменте стека по логическому адресу SS:SP, а при обращении к данным возможны различные варианты.

Как правило, по умолчанию считается, что данные находятся в основном сегменте данных, и обращение к ним осуществляется по логическому адресу DS:EA. Однако путем явного указания сегмента в описании операнда можно обратиться к данным и из любого другого сегмента.

Разновидностью данных, обрабатываемых МП ВМ86/ВМ88, являются цепочки (строки). При обработке таких данных, одна команда может оперировать с целой цепочкой элементов. При этом элементы цепочки-источника по умолчанию выбираются из основного сегмента данных по логическому адресу DS:SI, но могут выбираться и из любого другого сегмента. Цепочка-приемник может находиться только в дополнительном сегменте данных, и обращение к ее элементам осуществляется по логическому адресу ES:DI.

Если в описании операнда используется регистр BP, то по умолчанию считается, что этот операнд находится в сегменте стека. Однако в этом случае стековый принцип выборки данных игнорируется, и обращение к сегменту стека осуществляется как к ОЗУ с произвольным доступом. Это обеспечивает большую гибкость при обращении к данным, находящимся в стеке.

## Сегментная структура памяти

В соответствии с принципом вычисления физического адреса физические сегменты могут располагаться в памяти различным образом. Они могут быть неперекрывающимися, а также частично или полностью накладываться друг на друга. Различные варианты сегментных структур памяти приведены на рис. 2.5.

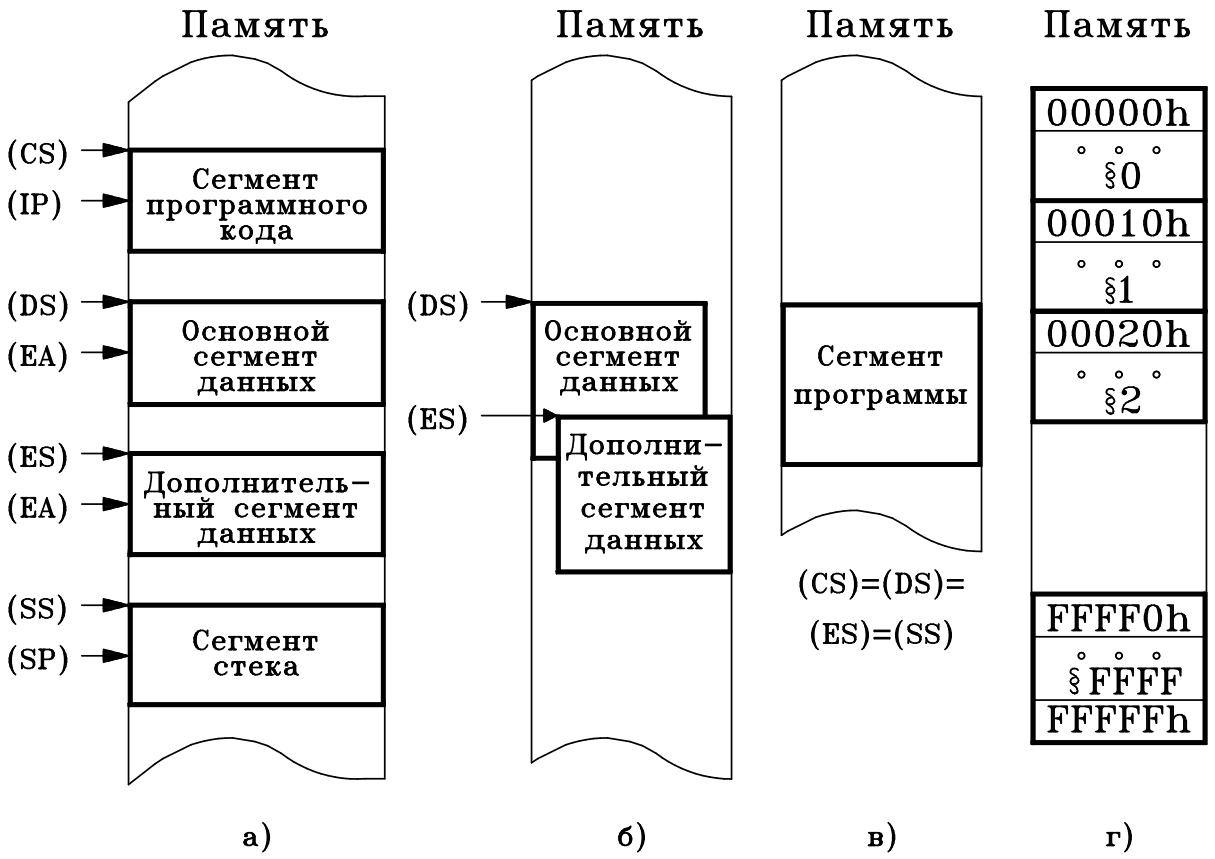


Рис. 2.5. Сегментная структура памяти:
а) неперекрывающиеся сегменты; б) частично перекрывающиеся сегменты;
в) совмещенные сегменты; г) параграфы памяти

Общее количество неперекрывающихся сегментов равно 16. Начало сегмента определяется содержимым соответствующего сегментного регистра. Поэтому всего в памяти может располагаться $2^{16}$ сегментов. При этом начальные адреса сегментов всегда кратны 16, т. к. заканчиваются 16-ричным нулем. Отсюда следует, что соседние сегменты отстоят друг от друга на 16 байтов.

Область памяти объемом в 16 байтов с начальным адресом, кратным 16, называется параграфом. Параграф характеризуется номером, значение которого равно содержимому сегментного регистра.

В частном случае при (CS) = (DS) = (ES) = (SS) = 0 все сегменты совмещены в один сегмент программы объемом 64 Кбайт. При этом организация памяти в МПС на базе ВМ86/ВМ88 идентична организации памяти в МПС на базе МП ВМ80.

При неизменном содержимом сегментных регистров обеспечивается доступ не более чем к 256 Кбайт памяти. Для доступа к полному адресному пространству необходимо изменять содержимое сегментных регистров.

Основным достоинством сегментной организации памяти является возможность свободного перемещения программы по всему объему адресного пространства, что необходимо для эффективного использования памяти в сложных системах. Для перемещения программы достаточно изменить лишь содержимое сегментных регистров CS, DS, ES, SS. Чтобы быть позиционно-независимой, программа не должна модифицировать своих сегментных регистров.

Недостатком сегментной организации памяти является сложность манипуляции физическими адресами при необходимости их программной обработки. Конкретный физический адрес может быть получен из множества логических адресов, т. е. из множества комбинаций segment:offset.

## 2.3. Архитектура интерфейса

Архитектура интерфейса представляет собой совокупность портов ввода/вывода, обеспечивающих сопряжение центральной части микроЭВМ с внешними устройствами по времени. Архитектура интерфейса приведена на рис. 2.6.

Разрядность всех портов может составлять 8 или 16 битов (D7 – D0 или D15 – D0). В каждом порту хранится некоторое число. Запись числа в порт ввода осуществляется внешним устройством, а процессор может прочитать его. Запись числа в порт вывода осуществляется процессором, а внешнее устройство считывает его.

Каждый порт ввода/вывода характеризуется его номером, который может быть одинаковым у порта ввода и порта вывода. Общее количество портов определяется разрядностью номера. В микроЭВМ на базе МПВМ86/ВМ88 разрядность номера порта при прямой адресации равна 8, что обеспечивает обращение к 256 портам ввода и 256 портам вывода. При косвенной адресации раз-

рядность номера порта равна 16, что дает возможность доступа к 65536 однобайтных портов или к 32768 двухбайтных портов.

Порты ввода

| Номер | Значение D15(D7) D0 |
|---|---|
| 00h | |
| 01h | |
| ... | ... |
| FFh | |

Порты ввода

| Номер | Значение D15(D7) D0 |
|---|---|
| 0000h | |
| 0001h | |
| ... | ... |
| FFFFh | |

Порты вывода

| Номер | Значение D15(D7) D0 |
|---|---|
| 00h | |
| 01h | |
| ... | ... |
| FFh | |

Порты вывода

| Номер | Значение D15(D7) D0 |
|---|---|
| 0000h | |
| 0001h | |
| ... | ... |
| FFFFh | |

а) б)

Рис. 2.6. Архитектура интерфейса
а) при прямой адресации, б) при косвенной адресации

Таким образом, номер порта ввода/вывода представляет собой двухразрядное (при прямой адресации) или четырехразрядное (при косвенной адресации) шестнадцатиричное число (см. рис. 2.6).

## 2.4. Основные элементы ассемблера. Формат программной строки

Основными элементами ассемблера являются директивы ассемблера и команды МП.

Команды описывают действия МП в процессе выполнения программы и всегда порождают машинный код в памяти системы.

Директивы служат для сообщения информации, необходимой в процессе трансляции программы. Они обеспечивают резервирование ячеек памяти, назначение имен, распределение сегментных регистров и т. д. Особенностью директив является то, что они, как правило, не порождают машинный код в памяти системы и поэтому часто называются псевдокомандами.

Программа на ассемблере представляет собой совокупность строк. Каждая строка содержит лишь одну команду МП или директиву ассемблера.

Формат командной строки имеет вид:

{МЕТКА:} {ПРЕФИКС} КОМАНДА {ОПЕРАНДЫ} {;КОММЕНТАРИЙ}

Здесь и далее фигурные скобки показывают необязательность заключенного в них элемента.

МЕТКА – это определенное программистом имя, используемое для пометки данной команды. Значением метки является адрес помеченной команды. Метки всегда заканчиваются двоеточием и используются в качестве операндов в командах передачи управления.

ПРЕФИКС – в этом поле помещается один из программных префиксов: префикс повторения REP или префикс блокировки LOCK, которые предшествуют команде и вызывают соответствующие действия.

КОМАНДА – в этом поле указывается мнемоническое имя команды, определяющее выполняемые действия.

ОПЕРАНДЫ – в этом поле описываются данные, над которыми выполняется операция, предписанная командой.

КОММЕНТАРИЙ – любая последовательность символов, начинающаяся с ;. Комментарий не влияет на выполнение команды и предназначен для пояснения выполняемых действий в контексте данной точки программы.

Формат директивной строки ассемблера имеет вид:

{ИМЯ} ДИРЕКТИВА {ОПЕРАНДЫ} {;КОММЕНТАРИЙ}

ИМЯ – это идентификатор каких-либо данных. Имя данных не имеет ничего общего с меткой и двоеточием не заканчивается.

ДИРЕКТИВА – это поле содержит мнемоническое имя директивы, определяющее действия, выполняемые ассемблером в процессе трансляции программы.

Поля ОПЕРАНДЫ и КОММЕНТАРИЙ аналогичны соответствующим полям командной строки.

Все поля программной строки должны разделяться по меньшей мере одним пробелом или знаком табуляции. Программная строка может начинаться не с первой позиции. Ее длина не должна превышать 132 знака.

## 2.5. Директивы управления сегментами

В связи с сегментацией памяти программа представляет собой совокупность логических сегментов.

Логический сегмент – это поименованная область памяти, адресуемая при неизменном значении сегментного регистра. Размеры логического сегмента не могут превышать размеров физического сегмента и составляют от 0 до 64 Кбайт.

Для описания логических сегментов предназначены директивы ассемблера *SEGMENT* и *ENDS*. Формат описания логического сегмента имеет вид:

```
Имя_сегмента    SEGMENT
                . . . . .          ; Содержимое логического
                . . . . .    .     ; сегмента
Имя_сегмента    ENDS
```

Директива *SEGMENT* присваивает логическому сегменту указанное имя и открывает его, а директива *ENDS* закрывает сегмент с указанным именем. Все данные или команды, находящиеся между директивами *SEGMENT* и *ENDS* с одним и тем же именем, принадлежат этому сегменту.

Как правило, логические сегменты не имеют общих частей. Однако допускается их вложенность и не допускается их пересечение.

***Пример 2.1.***

1. Описание неперекрывающихся сегментов:

```
Data1           SEGMENT
                . . . . . .
Data1           ENDS
Data2           SEGMENT
                . . . . . .
Data2           ENDS
```

2. Описание вложенных сегментов:

```
Data1           SEGMENT
                . . . . . .
Data2           SEGMENT
                . . . . . .
Data2           ENDS
                . . . . . .
Data1           ENDS
```

После описания логических сегментов необходимо связать их с сегментными регистрами, которые будут использоваться для их адресации. Для этого предназначена директива *ASSUME*, которая записывается в следующем формате:

*ASSUME* Сегментный_регистр : Имя_сегмента
{,Сегментный_регистр : Имя_сегмента . . .}

где Сегментный_регистр – имя одного из сегментных регистров CS, DS, ES, SS;
Имя_сегмента – имя логического сегмента, указанное в директиве *SEGMENT*.

***Пример 2.2.***

```
ASSUME CS:Code, DS:Data1, ES:Data2, SS:Stack  ; Связывание регистров
                                              ; и сегментов
```

Эта строка указывает, что для адресации сегмента Code будет использоваться сегментный регистр CS, сегмента Data1 – регистр DS, сегмента Data2 – регистр ES и сегмента Stack – регистр SS.

Допускается связывать один и тот же логический сегмент с несколькими сегментными регистрами, что соответствует совмещению сегментов памяти.

***Пример 2.3.***

```
ASSUME CS:Code, DS:Data, ES:Data      ; Совмещение основного и
                                      ; дополнительного сегментов
```

Здесь один и тот же логический сегмент Data связан с регистрами DS и ES и, следовательно, одновременно является и основным, и дополнительным сегментом данных в памяти, что соответствует их совмещению.

Использование директивы *ASSUME* исключает необходимость указания сегментных регистров всякий раз, когда необходимо получить доступ к содержимому того или иного сегмента в памяти. Нужный сегментный регистр выбирается автоматически в соответствии с ее назначениями.

В программе может быть несколько директив *ASSUME*. Как правило, она используется в начале программы и гораздо реже – в ее дальнейшем тексте. Директива *ASSUME* необходима в тех точках программы, начиная с которых используются другие логические сегменты по отношению к текущим сегментам.

Директива *ASSUME* не загружает начальные адреса логических сегментов в соответствующие сегментные регистры. Поэтому во все сегментные регистры, кроме CS, загрузка базовых адресов сегментов должна осуществляться явно. Это реализуется с помощью команд пересылки данных MOV.

***Пример 2.4.***

```
MOV     AX, Data1             ; Инициализация сегментных
                              ; регистров
MOV     DS, AX
ASSUME  DS: Data1
MOV     DX, <Данные из сегмента Data1>
MOV     AX, Data2
MOV     DS, AX
ASSUME  DS: Data2
MOV     DX, <Данные из сегмента Data2>
```

В этом примере сначала осуществляется инициализация регистра DS на базовый адрес сегмента Data1, а затем – на адрес сегмента Data2. Значения Data1 и Data2 являются константами, и их пересылка в DS осуществляется через промежуточный регистр AX. После инициализации DS на базовый адрес некоторого сегмента можно обращаться к находящимся в нем данным.

Доступ к переменным, находящимся в логических сегментах, не связанных с сегментными регистрами, невозможен. Ранее установленную связь между сегментным регистром и логическим сегментом можно разорвать с помощью той же директивы *ASSUME*, записывая директиву *NOTHING* вместо имени сегмента.

***Пример 2.5.***

*ASSUME* DS : *NOTHING*

Эта строка разрывает ранее установленную связь между регистром DS и логическим сегментом Data1 (см. пример 2.3), что делает невозможным обращение к данным из этого сегмента.

## 2.6. Типовая структура текста программы

В общем случае ассемблерная программа представляет собой совокупность логических сегментов данных, стека и программного кода.

Для улучшения сопровождаемости программы ее текст должен иметь структурированную форму. При этом наиболее целесообразно использовать паскалеобразную структуру текста, которая имеет вид:

*NAME* Имя_программы

; Описание констант
Здесь описываются поименованные константы

; Описание нестандартных типов данных
Здесь описываются создаваемые пользователем нестандартные типы данных (структуры и записи)

; Описание переменных (сегмент данных)
Data *SEGMENT*

Здесь описываются переменные стандартных и нестандартных типов данных

Data *ENDS*
; Описание стека (сегмент стека)
Stack *SEGMENT*
*DW* 16 *DUP* (?) ; Резервирование 16 слов под стек
StkTop *LABEL* *WORD* ; Фиксация верхушки пустого стека

```
Stack       ENDS
        ; Описание выполняемых действий (сегмент программного кода)
Code        SEGMENT
            ASSUME    CS:Code, DS:Data, ES:Data, SS:Stack
        ; Описание программных исполнителей
```

Здесь описываются подпрограммы и макрокоманды, используемые в программе

```
        ; Основная программа (исполняемая часть)
        Start:  MOV     AX, Data            ; Инициализация
                MOV     DS, AX              ; сегментных
                MOV     ES, AX              ; регистров и
                MOV     AX, Stack           ; указателя стека
                MOV     SS, AX
                MOV     SP, OFFSET StkTop
```

Здесь приводятся команды, решающие поставленную задачу

```
Code        ENDS
            END       Start
```

Директива *NAME* присваивает внутреннее имя объектному модулю, генерируемому ассемблером при трансляции программы. Это имя используется для обращения к модулю при модульном программировании. В простейшем случае директива *NAME* может отсутствовать.

Далее программа содержит необязательные разделы описания поименованных констант и нестандартных типов данных, которые не вызывают расходования памяти системы.

Состояние и объем используемой памяти определяется логическими сегментами программы Data, Stack и Code, описанных директивами *SEGMENT* и *ENDS*.

В сегменте данных Data описываются переменные стандартных и нестандартных типов, используемые в программе.

В сегменте стека Stack с помощью оператора *DUP* (?) зарезервировано 16 слов. Директива *LABEL* присваивает имя текущей ячейке памяти и определяет его тип. Поскольку *LABEL* расположена за зарезервированной зоной памяти, то имя StkTop будет присвоено следующему слову, что определит верхушку пустого стека.

Сегмент программного кода Code содержит директиву *ASSUME*, связывающую имена сегментов с сегментными регистрами. При этом сегмент данных связан с регистрами DS и ES, что обеспечивает совмещение основного и

дополнительного сегментов данных. Далее описываются подпрограммы и макрокоманды, используемые в программе. Начало исполняемой части программы обязательно помечается стартовой меткой, фиксирующей пусковой адрес программы. Исполняемая часть программы всегда начинается с инициализации сегментных регистров и указателя стека. После этого следуют команды, решающие поставленную задачу.

Текст программы завершается директивой *END*, в которой указывается стартовый адрес программы. Она сообщает ассемблеру о достижении конца исходного текста программы и необходимости ее запуска с команды, помеченной меткой Start.

Для успешного программирования на ассемблере необходимо научиться заполнять все разделы и сегменты типовой структуры программы.

## 2.7. Символические имена

Для идентификации данных и меток в ассемблерных программах используются символические имена.

Символическое имя представляет собой некоторую последовательность символов, в которую могут входить следующие знаки:

– буквы от A до Z (по умолчанию строчные и прописные буквы не различаются);

– цифры от 0 до 9;

– специальные знаки: знак подчеркивания (_), вопросительный знак (?), знак доллара ($) и знак коммерческого at(@).

Символические имена не могут начинаться с цифры. Ассемблер различает имена по первым 32 символам.

***Пример 2.6.***

```
Alpha                    ; Допустимые имена
String_Count
@DELAY
1Array                   ; Недопустимое имя
```

## 2.8. Константы. Их типы и директивы описания

Константы – это элементы данных, не изменяющие своего значения. Все константы делятся на четыре типа:

1) числовые;
2) символьные;
3) поименованные;
4) константы-выражения.

### 2.8.1. Числовые константы

Числовая константа представляет собой число, которое может быть записано в различных системах счисления. Основание системы счисления определяется последним символом этой константы. При этом для обозначения различных систем счисления используются следующие символы: b – двоичная система; q или o – восьмеричная система; d – десятичная система; h – шестнадцатиричная система.

Если в конце числовой константы нет ни одного из перечисленных символов, то ассемблер считает ее представленной в системе счисления, установленной по умолчанию. Изначально, по умолчанию, устанавливается десятичная система счисления.

Числовая константа всегда должна начинаться с десятичной цифры 0–9.

***Пример 2.7.***

| | |
|---|---|
| 01110010101b | – двоичная константа; |
| 17354q | – восьмеричная константа; |
| 255o | – восьмеричная константа; |
| 62897d | – десятичная константа; |
| 62897 | – десятичная константа (по умолчанию); |
| 3ABCh | – шестнадцатиричная константа; |
| F5h | – ошибка – начинается не с цифры 0–9; |
| 0F5h | – шестнадцатиричная константа. |

### 2.8.2. Символьные константы

Символьная константа представляет собой любую последовательность символов, заключенных в апострофы или кавычки. В виде символьных констант в программах представляются различные сообщения. Две формы задания символьных констант обусловлены тем, что в состав самих сообщений часто должны входить кавычки для выделения названий.

***Пример 2.8.***

```
'Введите данные'                    ; Это
"It is string constant"             ; символьные
'Система "Поиск" к работе готова'   ; константы
 Disk full                          ; Ошибка – нет ограничивающих
                                    ; символов
```

### 2.8.3. Поименованные константы

Поименованная константа – это константа, заданная именем. Связь между именем и значением константы устанавливается директивами *EQU* или =.

Эти директивы записываются в следующем формате:

```
Имя     EQU     Строка_символов
Имя     EQU     Числовое_выражение
Имя     =       Числовое_выражение
```

Директивы *EQU* и = аналогичны по назначению, но имеют следующие различия:

1) директива *EQU* может использоваться для присвоения имени как числового, так и символьного значения;

2) имена, определенные директивой =, можно переопределять, а директивой *EQU* – нельзя.

***Пример 2.9.***

```
K       EQU     1024        ; Числовое значение
Speed   EQU     Rate        ; Введение синонима
Count   EQU     CX          ; Переименование регистра
Const   =       56          ; Присвоение числового значения
Const   =       125         ; с возможностью явного или
Const   =       Const + 10  ; неявного переопределения
```

После присвоения имени некоторого значения им можно пользоваться везде, где можно использовать его значение. При трансляции программы ассемблер заменяет имя его значением.

***Пример 2.10.***

```
Table   EQU     DS:[BP][SI]
        MOV     AX, Table           ; Это эквивалентные
        MOV     AX, DS:[BP][SI].    ; строки
```

Использование поименованных констант улучшает сопровождаемость программы и упрощает ее модификацию.

### 2.8.4. Константы-выражения

Константа–выражение представляет собой выражение, содержащее поименованные константы и числовые значения, связанные знаками математических операций. Основными из них являются операции: + (сложение), – (вычитание), ∗ (умножение), / (деление нацело), *MOD* (остаток от деления нацело).

Константа-выражение вычисляется при трансляции программы и имеет числовое значение.

***Пример 2.11.***

```
Count     EQU (35721/10000 + 2)*3      ; после трансляции Count = 15
Alfa      =   (Beta–Gamma) MOD 5
```

Все имена, используемые в константе–выражении, должны быть определены ранее директивами *EQU* или =.

Задание константы в виде выражения дает четкое представление о структуре получаемого значения.

## 2.9. Переменные. Их типы и директивы описания

Переменная – это поименованная область памяти, предназначенная для хранения данных.

Все переменные в ассемблере делятся на переменные стандартных и нестандартных типов. Здесь рассматриваются лишь переменные стандартных типов, а переменные нестандартных типов будут рассмотрены позже.

### 2.9.1. Стандартные типы данных и их размещение в памяти

Для упрощения использования типовых переменных различного размера в ассемблере определены следующие стандартные типы данных:

*BYTE* – данные размером в 1 байт;
*WORD* – данные размером в 1 слово (два байта);
*DWORD* – данные размером в 1 двойное слово (два слова);
*QWORD* – данные размером в 1 квадрослово (четыре слова);
*TWORD* – данные размером в 1 тераслово (пять слов).

Все данные хранятся в памяти. Минимальной адресуемой единицей памяти является один байт, а вся память представляется одномерным массивом байтов, каждый из которых имеет 20-битный физический адрес.

Следовательно, физический адрес, выдаваемый на шину адреса, является адресом байта, т. е. адресом переменной типа *BYTE.*

Любые два смежных байта в памяти образуют слово. Переменная типа *WORD* хранится в слове так, что ее младший байт имеет меньший адрес, а старший байт – больший адрес. Адресом переменной типа *WORD* является адрес ее младшего байта.

Любые два смежных слова образуют двойное слово. Переменная типа *DWORD* хранится в двойном слове так, что ее младшее слово имеет меньший адрес, а старшее слово – больший адрес. В пределах слова данные хранятся так же, как в переменной типа *WORD*. Адресом переменной типа *DWORD* является адрес младшего байта ее младшего слова.

Размещение в памяти и адресация переменных типов *QWORD* и *TWORD* осуществляется аналогично.

Любая последовательность байтов или слов в памяти может рассматриваться как данные типа *ЦЕПОЧКА* (*СТРОКА*). Как правило, в виде цепочек байтов представляются строки текста. В этом случае каждый байт цепочки хранит один символ строки в ASCII-коде. Первый символ цепочки имеет меньший адрес, а последний – больший адрес. Адресом цепочки является адрес ее первого байта.

### 2.9.2. Директивы описания переменных и распределения памяти

Все директивы описания переменных распределяют под них память и инициализируют их на указанные начальные значения.

Директивы описания переменных записываются в следующем формате:

{Имя} *ДИРЕКТИВА* Начальное_значение {,Начальное_значение . . .}
{;Комментарий}

Для описания конкретной переменной вместо формального параметра *ДИРЕКТИВА* должна использоваться одна из следующих директив:

*DB* – определить переменную типа *BYTE*;
*DW* – определить переменную типа *WORD*;
*DD* – определить переменную типа *DWORD*;
*DQ* – определить переменную типа *QWORD*;
*DT* – определить переменную типа *TWORD*.

При наличии имени в директиве описания переменной ячейке памяти по текущему адресу трансляции присваивается это имя. При отсутствии имени выделяется соответствующая область памяти, но она остается безымянной.

В каждой директиве может указываться либо один, либо несколько операндов, определяющих начальные значения распределяемых ячеек памяти. Вместо любого из операндов может указываться вопросительный знак (?). В этом случае начальная инициализация соответствующего элемента данных не производится.

***Пример 2.12.***

```
DataByte    DB   10, 4, ?, 10h        ; Описание переменных
DataWord    DW   100, 100h, –5, ?     ; с инициализацией
DataDWord   DD   3*20, 12345678h      ; на числовые значения
```

Программные строки этого примера вызывают распределение ячеек памяти и инициализацию значений переменных в соответствии с рис. 2.7.

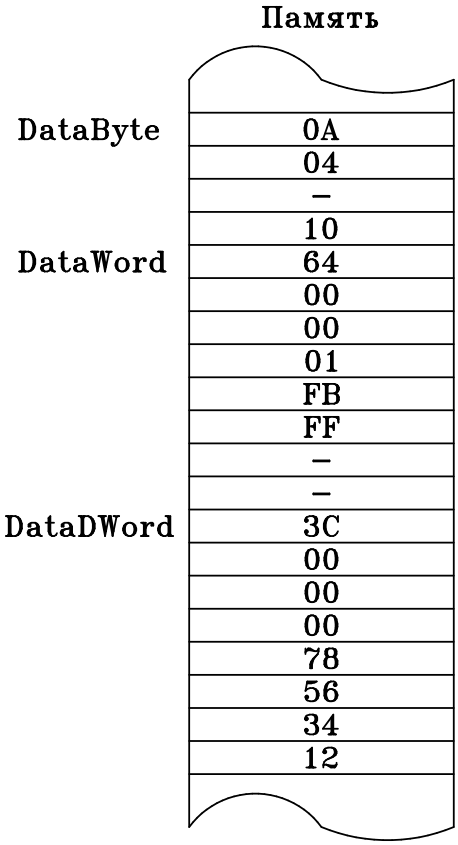


Рис. 2.7. Распределение памяти и инициализация значений переменных

Директива *DB* инициализирует четыре смежных байта на указанные значения и связывает с первым из них имя переменной DataByte. Третий байт не инициализируется, что на рис. 2.7 обозначено прочерком. Директива *DW* инициализирует четыре смежных слова и связывает с младшим байтом первого из них имя переменной DataWord. Директива *DD* распределяет два двойных слова и инициализирует их на значения 3∗20 = 60d = 3Ch и 12345678h. Она также связывает имя DataDWord с младшим байтом первого двойного слова.

Очевидно, что имя переменной фактически является ее адресом.

Переменная типа *ЦЕПОЧКА* (*СТРОКА*) описывается и инициализируется директивой *DB*, в которой заданная цепочка используется в качестве операнда.

***Пример 2.13.***

Message *DB* 'СЕГМЕНТ' ; Инициализация на символьное значение

Эта директива загружает в смежные байты памяти ASCII-коды букв С(91h), Е(85h), Г(83h), М(8Ch), Е(85h), Н(8Dh) и Т(92h), связывая имя Message с байтом, в котором находится буква С.

Инициализация путем перечисления начальных значений отдельных ячеек удовлетворительна при их небольшом количестве и становится громоздкой при инициализации многих ячеек. Для упрощения описания в этом случае допускается использование оператора дублирования *DUP*. Любой из операндов в

списке начальных значений директивы описания переменных можно заменить следующей конструкцией:

N *DUP* (Начальное_значение {,Начальное_значение . . .}),

где N – беззнаковое целое число, определяющее количество повторений набора начальных значений, указанного в скобках.

***Пример 2.14.***

```
Arr1    DB    100   DUP  (0,1)
Arr2    DW    10    DUP  (?)
Arr3    DB    20    DUP  ('Stack')
```

Первая строка этого примера определяет массив Arr1, содержащий сто пар байтов со значениями 0,1; вторая строка – массив Arr2, включающий в себя 10 слов без инициализации; третья строка – массив Arr3, состоящий из 100 байтов (20 цепочек 'Stack').

Допускается вложение операторов *DUP*. Глубина вложения обычно не превышает 8.

***Пример 2.15.***

```
Array   DB    10    DUP  (5,3 DUP (0))
```

Эта строка вызывает резервирование области памяти в 40 байтов, с повторяющимися группами значений 5, 0, 0, 0, . . . под именем Array.

Кроме инициализации на числовые значения, с помощью директив *DW* и *DD* можно инициализировать ячейки памяти на адресные значения. В этом случае их операнды записываются в виде адресного выражения:

Имя_переменной(или Метки) ± Константа–выражение

При этом директива *DW* фиксирует только смещение offset, а директива *DD* – полный логический адрес segment : offset указанного адресного выражения.

***Пример 2.16.***

```
AddrTablNear  DW    Par1     ; Инициализация на смещение
              DW    Par2     ; (внутрисегментный адрес)
              DW    Par3
AddrTablFar   DD    Data1    ; Инициализация на полный логический
              DD    Data2    ; адрес (межсегментный адрес)
```

Директивы *DW* в этом примере создают таблицу внутрисегментных смещений для параметров Par1, Par2, Par3, а директивы *DD* – таблицу полных логических адресов для Data1 и Data2 в соответствии с рис. 2.8.

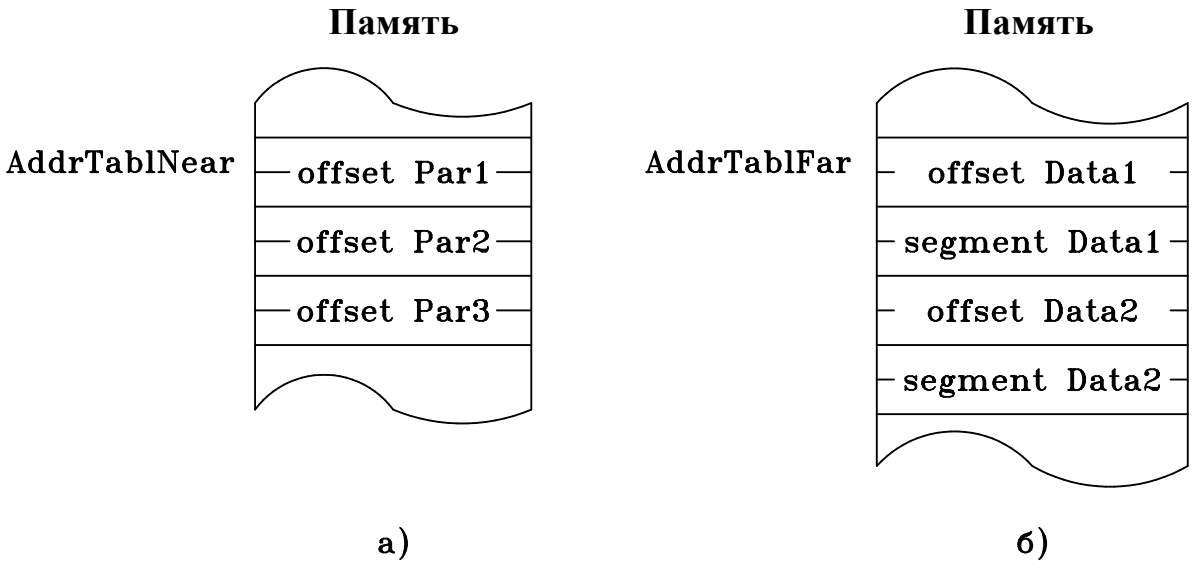


Рис. 2.8. Инициализация памяти на адресные значения:
а) директивой DW; б) директивой DD

На практике, как правило, для описания простых переменных используются директивы *DB*, *DW* и *DD* с единственным начальным значением.

***Пример 2.17.***

```
Alpha   DB    0            ; Описание и инициализация
Beta    DW    ?            ; простых переменных
Gamma   DD    12345678h
```

В этом случае каждый элемент данных имеет собственное имя.

Оператор *DUP* совместно с директивами *DB*, *DW*, *DD* используется для описания массивов.

***Пример 2.18.***

```
Array1  DB    10   DUP (0)          ; Одномерный массив из 10
                                   ; нулевых байт
Array2  DW    20   DUP (?)          ; Одномерный массив из 20
                                   ; слов без инициализации
Array3  DB    10   DUP(15 DUP(?))   ; Двумерный массив (10x15)
                                   ; байтов без инициализации
```

## 2.10. Атрибуты переменных и меток. Директивы, возвращающие атрибутные значения

### 2.10.1. Атрибуты переменных

Любая простая переменная имеет три атрибута: "Сегмент", "Смещение" и "Тип".

Атрибут "Сегмент" представляет собой базовый адрес логического сегмента, в котором описана переменная, и может принимать значение из диапазона 0 . . . FFFFh.

Атрибут "Смещение" представляет собой расстояние в байтах от начала логического сегмента до переменной и может иметь значение в диапазоне 0 . . . FFFFh.

Атрибут "Тип" характеризует длину переменной в байтах и для стандартных переменных может иметь значения 1 (для типа BYTE), 2 (для типа *WORD*), 4 (для типа *DWORD*) и т. д.

Массивы имеют два дополнительных атрибута: "Длина" и "Размер".

Атрибут "Длина" характеризует количество элементов в массиве, независимо от их типа.

Атрибут "Размер" характеризует размер массива в байтах.

Очевидно, что для одного и того же массива справедливо соотношение "Размер" = "Длина" ∗ "Тип".

Для улучшения сопровождаемости программ в ассемблере имеются директивы, возвращающие атрибутные значения переменных, а именно:

*SEG* – возвращает атрибут "Сегмент";
*OFFSET* – возвращает атрибут "Смещение";
*TYPE* – возвращает атрибут "Тип";
*LENGTH* – возвращает атрибут "Длина";
*SIZE* – возвращает атрибут "Размер".

***Пример 2.19.***

Если сегмент Data, начинающийся с физического адреса 05000h, имеет вид:

```
Data            SEGMENT
        Alpha   DB      ?
        Beta    DD      ?
        Gamma   DW      100  DUP (?)
Data            ENDS,
```

то рассмотренные директивы вернут следующие значения:

*SEG* Alpha = *SEG* Beta = *SEG* Gamma = 500h;
*OFFSET* Alpha = 0, *OFFSET* Beta = 1, *OFFSET* Gamma = 5;

*TYPE* Alpha = 1, *TYPE* Beta = 4, *TYPE* Gamma = 2;
*LENGTH* Gamma = 100;
*SIZE* Gamma = 200.

Следует помнить, что директива *LENGTH* всегда возвращает коэффициент повторений при операторе *DUP*, использованный при описании массива. Во всех остальных случаях она возвращает значение 1 без сообщения об ошибке.

Длина массивов, описанных без применения оператора *DUP*, определяется как разность внутрисегментных смещений постпоследней и начальной ячеек массива, деленная на тип массива.

***Пример 2.20.***

```
Str        DB  "Введите исходные данные"          ; Описание строки
EndStr     DB  ?                                  ; Пометка конца строки
LengthStr  = (OFFSET EndStr – OFFSET Str)/TYPE Str ; Определение
                                                  ; длины строки
```

Использование директив, возвращающих атрибутные значения, позволяет писать программы в значительной степени инвариантные к обрабатываемым данным.

***Пример 2.21.***

*Вычислить сумму элементов массива по модулю* $2^{16}$:

```
Array   DW      100 DUP(?)          ; Описание массива слов
        . . . . . . . . . .
        MOV     AX, SEG Array       ; Загрузка полного
        MOV     DS, AX              ; адреса массива
        MOV     BX, OFFSET Array
        MOV     CX, LENGTH Array    ; Загрузка счетчика циклов
        SUB     AX, AX              ; Обнуление накопителя
Next:   ADD     AX, [BX]            ; Прибавление очередного элемента
        ADD     BX, TYPE Array      ; Модификация адреса
        DEC     CX                  ; Модификация счетчика циклов
        JNZ     Next                ; Продолжить, если не все
```

При изменении размера или местоположения массива Array в памяти обрабатывающая его программа не изменится.

## 2.10.2. Атрибуты меток

Метка представляет собой имя ячейки программной памяти и используется в качестве операнда команд передачи управления. Любая метка имеет три атрибута: "Сегмент", "Смещение" и "Дистанция".

Атрибуты "Сегмент" и "Смещение" идентичны соответствующим атрибутам переменной, и их значения при необходимости могут быть получены с помощью тех же директив *SEG* и *OFFSET*.

Атрибут "Дистанция" характеризует расстояние в байтах, в пределах которого метка достижима для команды передачи управления. Этот атрибут аналогичен атрибуту "Тип" переменной.

Если метка достижима из диапазона от (–128) до (+127) байтов от текущей точки программы, то она имеет тип *SHORT* (короткий переход). В этом случае для представления дистанции в команде перехода достаточно одного байта.

Если метка достижима из диапазона от (–32768) до (+32767) байтов, то она имеет тип *NEAR* (ближний переход). В этом случае для представления дистанции необходимы два байта, и управление может передаваться в пределах всего сегмента. При таком переходе модифицируется только указатель команд IP. По умолчанию любая метка имеет тип *NEAR*.

Если метка достижима из другого сегмента программного кода, то она имеет тип *FAR* (дальний переход). В этом случае в команде перехода адрес точки назначения задается ее полным логическим адресом в формате segment : offset, а не дистанцией. При таком переходе модифицируются регистры CS и IP.

Значение атрибута "Дистанция" может быть получено с помощью директивы *TYPE*. При этом для меток типа *SHORT* и *NEAR* возвращается значение (–1), а для меток типа *FAR* – (–2).

## 2.11. Замена атрибутов переменных и меток

После описания переменной или метки в программе все их атрибуты однозначно определены. При трансляции программы они используются по умолчанию для контроля и для формирования машинного кода соответствующих команд. При несоответствии атрибутов переменной или метки их использованию в команде фиксируется ошибка.

***Пример 2.22.***

```
Var1    DW      ?
        MOV     AL, Var1            ; Ошибка несоответствия типов
```

При трансляции команды MOV будет зафиксирована ошибка несоответствия типов, т. к. переменную Var1 типа *WORD* нельзя загрузить в регистр AL типа *BYTE*.

При программировании на ассемблере часто требуется временно изменять атрибуты переменных или меток. В этом случае они указываются явно с помощью директив или конструкций ассемблера. При этом изменяться могут лишь атрибуты "Сегмент", "Смещение" и "Тип".

### 2.11.1. Замена смещения

Для изменения смещения используется конструкция, называемая адресным выражением. В общем случае адресное выражение записывается в следующем формате:

Имя_переменной (или Метки) ± Константа_выражение

При этом атрибуты "Сегмент" и "Тип" адресного выражения не изменяются, а атрибут "Смещение" имеет новое значение, отличающееся на величину указанной константы.

***Пример 2.23.***

Пусть в программе описана некоторая переменная X, тогда:

*SEG* (X + 10) = *SEG* X
*OFFSET*(X + 10) = *OFFSET* X + 10
*TYPE* (X + 10) = *TYPE* X

Одиночные имена переменных или меток являются простейшими адресными выражениями.

### 2.11.2. Замена сегмента

При выполнении программы МП выбирает сегментный регистр для вычисления адреса текущего обращения к памяти в соответствии с используемым способом адресации данных или в соответствии с байтом префикса замены сегмента, предшествующего команде.

При разработке программы программист размещает данные в логических сегментах и с помощью директивы *ASSUME* связывает их с сегментными регистрами, которые по умолчанию будут использоваться для их адресации. Однако, при необходимости можно отменить назначение по умолчанию и явно указать используемый сегментный регистр. Это осуществляется с помощью префикса замены сегмента, который записывается в виде имени сегментного регистра с последующим двоеточием – CS:, DS:, ES: и SS:.

Таким образом, в случае замены сегмента операнд команды записывается в виде:

Сегментный_регистр:Адресное_выражение

Эта конструкция порождает новый адрес, сегментным компонентом которого является указанный сегментный регистр. Если этот регистр не совпадает с сегментным регистром, выбираемым МП по умолчанию в соответствии с используемым способом адресации (см. табл. 2.2), то при трансляции программы ассемблер формирует и помещает перед командой байт префикса замены сегмента (БПЗС). В противном случае БПЗС не формируется. Правила формирования БПЗС рассмотрим на примере.

***Пример 2.24.***

```
Data1          SEGMENT                        ; Описание данных
      Var1     DB          ?
Data1          ENDS
Data2          SEGMENT
      Var2     DB          ?
Data2          ENDS
Code           SEGMENT
               ASSUME      CS:Code, DS:Data1, ES:Data2
               MOV         DL, [BP]           ; По умолчанию SS – БПЗС
                                              ; не формируется
               MOV         DL, DS:[BP]        ; Формируется БПЗС на DS
               MOV         DL, Var1           ; По умолчанию DS, совпадающий
                                              ; с указаниями ASSUME – БПЗС
                                              ; не формируется
               MOV         DL, Var2           ; По умолчанию DS, но из ASSUME
                                              ; следует ES – формируется
                                              ; БПЗС на ES
               ASSUME      DS:NOTHING, ES:Data1
               MOV         DL, Var1           ; По умолчанию DS, но из ASSUME
                                              ; следует ES – формируется
                                              ; БПЗС на ES
               MOV         DL, DS:Var1        ; По умолчанию DS – БПЗС
                                              ; не формируется
```

При явном указании сегментного регистра в описании операнда обращение к нему осуществляется и в случае, если логический сегмент, в котором он находится, не связан с этим регистром.

Префикс замены сегмента всегда относится только к одной команде.

### 2.11.3. Замена типа

Тип переменной или метки может быть временно изменен с помощью директивы *PTR*, которая записывается в следующем формате:

Новый_тип *PTR* Адресное_выражение

В качестве параметра "Новый_тип" могут указываться имена стандартных типов: для переменных – *BYTE*, *WORD*, *DWORD*, а для меток – *NEAR* и *FAR*. В результате адресное выражение приобретает новый тип, а его атрибуты "Сегмент" и "Смещение" не изменяются.

Директива *PTR* должна использоваться для прямого обращения к составным частям переменных типа *DWORD*, *QWORD*, *TWORD*.

***Пример 2.25.***
*Скопировать в памяти переменную типа DWORD*

```
Source   DD    ?                        ; Источник данных
Dest     DD    ?                        ; Приемник данных
; Обращения типа "Память – Память" недопустимы,
; и копирование должно осуществляться через регистры МП
         MOV   AX, Source               ; Ошибка – несоответствие
                                        ; типов данных
         MOV   AX, WORD PTR Source      ; Копирование младшего
         MOV   WORD PTR Dest, AX        ; слова
         MOV   AX, WORD PTR Source + 2  ; Копирование старшего
         MOV   WORD PTR Dest + 2, AX    ; слова
```

Очень часто директива *PTR* используется для устранения неоднозначности при анонимных обращениях к памяти.

***Пример 2.26.***
*Инкрементировать элемент массива байтов (слов), адресуемый регистром BX*

```
INC    [BX]                ; Ошибка – неоднозначность инкремен-
                           ; тируемого элемента: BYTE или WORD?
INC    BYTE  PTR  [BX]     ; Инкрементирование байта
INC    WORD PTR  [BX]      ; Инкрементирование слова
```

Замена типа метки осуществляется аналогично. Например, если сегмент программного кода содержит строку:

```
Begin:    MOV    CX, 100
```

то метка Begin имеет тип *NEAR*, что позволяет ссылаться на нее командам перехода JMP лишь из того же сегмента. Для ссылки на эту метку из других сегментов необходимо явно указывать тип перехода *FAR* с помощью директивы *PTR* – JMP *FAR PTR* Begin, что особенно актуально при ссылках вперед.

Использование директивы *PTR* для частого обращения к переменным или меткам, имеющим как бы два различных типа, неудобно. Для исключения этого неудобства предусмотрена возможность связывания с одними и теми же ячейками памяти различных переменных или меток. Для этого служит директива *LABEL*, записываемая в виде:

Имя *LABEL* Тип

В качестве параметра "Тип" для переменных могут использоваться *BYTE*, *WORD*, *DWORD*, а для меток – *NEAR* и *FAR*.

Часто директива *LABEL* используется для получения доступа к элементам данных различного размера из одной и той же области памяти.

***Пример 2.27.***

*Обеспечить обращение к одной и той же зоне памяти как к области байтов и слов.*

Для решения поставленной задачи описание этой области памяти должно иметь вид:

```
ArrByte    LABEL    BYTE
ArrWord    DW       200 DUP(?)
```

Директива *DW* описывает область памяти как массив слов, присваивая ей имя ArrWord. Директива *LABEL* той же самой области памяти присваивает имя ArrByte, описывая ее как последовательность байтов.

При обращении к этой области памяти через имя ArrByte осуществляется доступ к байту, а при обращении через имя ArrWord – к слову.

***Пример 2.28.***

```
MOV    ArrByte[5], 0      ; Запись 0 в шестой байт области памяти
MOV    ArrWord[5],0       ; Запись 0 в слово (шестой и седьмой
                          ; байты) области памяти
```

С помощью директивы *LABEL* можно для любой команды ввести метку с описанием ее типа.

***Пример 2.29.***

```
Begin    LABEL  FAR        ; Введение метки
MOV      CX, 100
```

Директива *LABEL* связывает метку Begin типа *FAR* с командой MOV. Это позволяет передавать управление на эту команду из других сегментов программного кода командой JMP Begin.

## 2.12. Способы адресации операндов

При выполнении программы МП многократно обращается к памяти за командами и операндами. Положение любой ячейки характеризуется ее полным логическим адресом segment:offset. В качестве сегментного компонента segment используется содержимое соответствующего сегментного регистра, а внутрисегментное смещение offset в общем случае может формироваться различным образом. Часто оно берется из регистров, но может и вычисляться по некоторому закону.

Внутрисегментное смещение offset, используемое для обращения к памяти, называется эффективным или исполнительным адресом EA. Эффективный

адрес формируется в соответствии с используемым способом адресации операнда. Способ адресации определяется формой записи операнда. Поскольку существуют двухоперандные команды, то в одной команде могут использоваться два различных способа адресации.

При любом способе адресации значение, участвующее в операции, представляет собой содержимое указанного операнда.

### 2.12.1. Адресация данных

МП ВМ86/ВМ88 реализует разнообразные способы адресации, позволяющие эффективно обрабатывать и простые, и сложные структуры данных.

**Регистровая адресация**

В этом случае операнд находится в одном из регистров МП и записывается путем указания имени регистра REG.

***Пример 2.30.***

```
MOV     AX, BX          ; Пересылка (BX) в (AX)
MOV     DS, AX          ; Пересылка (AX) в (DS)
```

Здесь и далее обозначение элемента архитектуры в круглых скобках соответствует содержимому этого элемента.

Команды с регистровой адресацией являются наиболее короткими и выполняются за минимальное время.

**Непосредственная адресация**

В этом случае операнд представляет собой числовую или символьную константу Data, записываемую непосредственно в команде.

***Пример 2.31.***

```
MOV     AX, 328         ; Загрузка в AX числа 328
MOV     DX 'МП'         ; Загрузка в DX символов 'МП'
MOV     DS, 100         ; Ошибка – непосредственная загрузка
                        ; в сегментные регистры недопустима
```

Доступ к непосредственным операндам осуществляется очень быстро, т. к. они выбираются из внутренней очереди команд.

**Прямая адресация**

Прямая адресация является простейшим способом адресации памяти. В этом случае операнд записывается прямо в команде в виде имени переменной NameVar (в общем случае – в виде адресного выражения). При этом эффективный адрес EA = offset NameVar.

***Пример 2.32.***

```
MOV     AL, Source          ; Загрузка переменной Source в AL
MOV     Dest + 10, AX       ; Запись (AX) в слово, отстоящее
                            ; от переменной Dest на 10 байтов
                            ; вперед
```

Прямая адресация используется для обращения к простым переменным.

**Косвенная регистровая адресация**

Источником косвенного адреса могут служить только базовые регистры BREG (BX,BP) или индексные регистры IREG (SI,DI). В этом случае операнд записывается в виде имени регистра-источника адреса в квадратных скобках [BREG] или [IREG], а именно: [BX], [BP], [SI], [DI]. При этом эффективный адрес EA = (BX), (BP), (SI) или (DI) соответственно.

***Пример 2.33.***

```
MOV     AX, [BX]    ; Загрузка слова из ячейки, адресуемой (BX)
MOV     [SI], AL    ; Запись байта в ячейку, адресуемую (SI).
```

Косвенная адресация по ячейке памяти отсутствует, но может быть легко реализована при необходимости.

***Пример 2.34.***

```
MOV     SI, Addr            ; Косвенная адресация
MOV     AX, [SI]            ; по ячейке Addr
```

Косвенная адресация используется при циклической организации программ, в которых за счет модификации косвенного адреса с помощью одной и той же команды осуществляется обращение к различным элементам данных.

В более сложных способах адресации в описании операндов, как правило, имеется некоторая константа, которая может задаваться различным образом. Она включается в машинный код команды и используется при вычислении эффективного адреса. Эта константа интерпретируется как число со знаком и вносит смещение относительно эффективного адреса, определяемого другими компонентами описания операнда.

Смещение, содержащееся в команде, принято называть смещением displacement или disp. Смещение disp определяется автоматически по описанию операнда в процессе трансляции программы и может принимать 8- или 16-битные значения.

**Базовая адресация**

Источником базового адреса могут служить только базовые регистры BREG (BX,BP). В этом случае операнд может записываться в следующем виде:

[BREG][Const] или [BREG] + Const или [BREG + Const].

Во всех случаях смещение disp = Const, и эффективный адрес EA = (BREG) + disp.

***Пример 2.35.***

```
MOV     AX,  [BP][10]        ; Загрузка из стека 6-го
MOV     AX,  [BP + 10]       ; слова области, адресуемой BP.
MOV     [BX] + Temp, AL      ; Запись (AL) в байт Temp
                             ; области, адресуемой BX.
```

При этом имя Temp должно быть определено как константа.

Базовая адресация используется в тех случаях, когда номер элемента, к которому осуществляется обращение, известен на этапе разработки программы, а базовый (начальный) адрес области данных вычисляется при ее выполнении.

**Индексная адресация**

Источником индексного адреса могут служить только индексные регистры IREG (SI, DI). В этом случае операнд может записываться в следующем виде:

1) NameVar[IREG] ; disp = offset NameVar
2) NameVar[IREG + Const] ; disp = offset NameVar + Const.

При этом эффективный адрес EA = disp + (IREG).

***Пример 2.36.***

```
MOV     Array[SI], AX        ; Запись AX в текущий элемент
                             ; массива Array
MOV     DX, Array[DI−10]     ;Загрузка DX элементом массива
                             ; Array, отстоящим от элемента,
                             ; адресуемого DI, на 10 байтов назад
```

Индексная адресация используется для обращения к элементам одномерных поименованных массивов в тех случаях, когда имя массива известно на этапе разработки программы, а номер элемента, к которому осуществляется обращение, вычисляется при выполнении программы.

По существу базовая и индексная адресация в МП ВМ86/ВМ88 аналогичны. Это обусловлено тем, что и базовые, и индексные регистры имеют одинаковую длину в 16 бит. При таком рассмотрении оба эти способа адресации считаются одним способом, называемым регистровой относительной адресацией.

**Базово-индексная адресация**

В этом случае операнд записывается в следующем виде:

[BREG][IREG] или [BREG + IREG].

При этом эффективный адрес EA = (BREG) + (IREG).

***Пример 2.37.***

```
MOV     AX,  [BX][SI]       ; Загрузка в AX элемента двумерного
MOV     AX,  [BX + SI]      ; массива с координатами (BX),(SI)
```

Базово-индексная адресация является гибким режимом адресации, т. к. позволяет вычислять и варьировать два компонента адреса. Это может использоваться при обращении к элементам двумерных массивов.

**Относительная базово-индексная адресация**

Этот способ является наиболее сложным способом адресации. Операнд может записываться в следующем виде:

```
1) NameVar[BREG][IREG]              ; disp = offset NameVar
2) [BREG][Const][IREG]              ; disp = Const
3) [BREG][IREG][Const]              ; disp = Const
4) [BREG + Const1][IREG + Const2]   ; disp = Const1 + Const2
```

При этом эффективный адрес EA = (BREG) + (IREG) + disp.

***Пример 2.38.***

```
MOV     Array[BX][SI], AX       ; Запись (AX) в текущий элемент
                                ; двумерного массива Array
MOV     [BX][10*20][DI], AL     ; Запись (AL) в текущий элемент
                                ; 10-ой строки двумерного
                                ; массива с косвенным именем
MOV     [BX][SI][10], AL        ; Запись (AL) в 10-й элемент
                                ; текущей строки двумерного
                                ; массива с косвенным именем
```

Относительная базово-индексная адресация используется для обращения к элементам двумерных массивов с каким-либо одним фиксированным параметром. При этом может быть зафиксировано имя массива, номер выбираемой строки или элемента в ней.

## 2.12.2. Адресация переходов

Способ адресации перехода определяет адрес, по которому передается управление командой перехода. Для задания адреса перехода могут использоваться все те же способы адресации, что и для данных. При этом возможно выполнение четырех различных типов переходов.

### Внутрисегментный прямой переход

В этом случае операнд записывается прямо в команде перехода в виде имени метки LabelNear, находящейся в том же сегменте программного кода, что и эта команда.

При трансляции команды вычисляется смещение disp=offset LabelNear–(IP) и включается в ее машинный код. Смещение disp может иметь 8-битное или 16-битное значение.

При выполнении такой команды вычисляется эффективный адрес перехода EA = (IP) + disp и заносится в регистр IP. При этом управление может передаваться в пределах всего сегмента программного кода. Такой переход называется ближним переходом или переходом типа NEAR.

***Пример 2.39.***

```
JMP     Begin          ; Ближний переход на метку Begin
JMP     Buffer + 10    ; Ближний переход на ячейку, отстоящую
                       ; от метки Buffer на 10 байтов вперед
```

Внутрисегментную прямую адресацию часто называют относительной адресацией, т. к. смещение disp вычисляется относительно (IP).

### Внутрисегментный косвенный переход

Источником косвенного адреса перехода в этом случае может быть любой 16-битный регистр или слово в памяти, адресуемое любым способом. Операнд команды перехода записывается в соответствии с используемым способом адресации данных.

***Пример 2.40.***

```
JMP     AX             ; Ближний переход по адресу из AX
JMP     Alpha          ; Ближний переход по адресу
                       ; из ячейки Alpha (если Alpha DW ?)
JMP     WORD PTR [SI]  ; Ближний переход по адресу
                       ; из ячейки, адресуемой SI
```

При выполнении этих команд содержимое IP заменяется содержимым указанного операнда, что и соответствует ближнему переходу.

### Межсегментный прямой переход

В этом случае операнд записывается прямо в команде перехода в виде имени метки LabelFar, находящейся в другом сегменте программного кода по отношению к этой команде. Для устранения проблем, связанных с обращением вперед, в команде необходимо явно указывать тип перехода с помощью директивы *PTR*.

При трансляции такой команды в ее машинный код включается полный логический адрес точки назначения в формате segment:offset. При выполнении

команды значение segment помещается в CS, а offset – в IP, что и соответствует передаче управления в другой сегмент программного кода.

Такой переход называется дальним переходом или переходом типа FAR.

***Пример 2.41.***

```
JMP     FAR  PTR  Cont     ; Дальний переход на метку Cont
```

**Межсегментный косвенный переход**

Источником косвенного адреса перехода в этом случае служит двойное слово в памяти, адресуемое любым способом. Операнд команды перехода записывается в соответствии с используемым способом адресации данных.

***Пример 2.42.***

```
JMP     Beta                ; Дальний переход по адресу
                            ; из ячейки Beta (если Beta DD ?)
JMP     DWORD PTR [BX]      ; Дальний переход по адресу
                            ; из ячейки, адресуемой BX
```

При выполнении этих команд младшее слово указанного двойного слова помещается в IP, а старшее – в CS, что и соответствует дальнему переходу.

## 2.13. Система команд

Значительную часть ассемблера составляют команды МП [3]. Все команды делятся на шесть групп:

1) команды пересылки данных;
2) арифметические команды;
3) логические команды;
4) команды передачи управления;
5) команды обработки строк;
6) команды управления микропроцессором.

Краткий справочник по системе команд, удобный для повседневной работы, приведен в приложении.

В последующих разделах все команды и особенности их использования будут рассмотрены более подробно.

### 2.13.1 Команды пересылки данных

Команды пересылки данных осуществляют обмен данными или адресами между регистрами МП и ячейками памяти или портами ввода/вывода. Их можно разделить на следующие подгруппы:

1) команды пересылки общего назначения;
2) команды загрузки адресных значений;
3) команды обращения к стеку;

4) команды пересылки флагов;
5) команды ввода/вывода.

### Команды пересылки общего назначения

К этой подгруппе относятся команды MOV, XCHG и XLAT.

Команда MOV является основной командой пересылки общего назначения. Она имеет следующий формат:

MOV приемник, источник

и осуществляет пересылку байта или слова данных из источника в приемник. В качестве приемника и источника данных могут использоваться регистры МП или ячейки памяти. Кроме того, источник может быть и непосредственным значением.

В команде MOV допустимо большинство из возможных сочетаний операндов.

***Пример 2.43.***

```
MOV    AX, Table          ; Пересылка из памяти в регистр
MOV    Table, AX          ; и наоборот
MOV    ES: [BX], DX       ; Пересылка из регистра в память
MOV    DS, AX             ; Пересылка между 16-битными
MOV    BL, AL             ; или 8-битными регистрами
MOV    CL, 30             ; Пересылка константы в регистр
MOV    [BX + SI], 25h     ; или в память
```

Некоторые из сочетаний операндов в команде MOV недопустимы.

1. Нельзя переслать данные из одной ячейки памяти в другую. При необходимости они пересылаются через промежуточный регистр.

***Пример 2.44.***

```
MOV    Alpha, [BX]        ; Ошибка − обращение типа
                          ; "Память − Память"
MOV    AX, [BX]           ; Пересылка данных из ячейки,
MOV    Alpha, AX          ; адресуемой BX, в ячейку Alpha
```

2. Нельзя загрузить непосредственный операнд в сегментный регистр. При необходимости эта загрузка осуществляется через промежуточный регистр.

***Пример 2.45.***

```
MOV    DS, 0ABCDh         ; Ошибка − загрузка константы
                          ; в регистр DS
MOV    AX, 0ABCDh         ; Загрузка константы
MOV    DS, AX             ; в регистр DS
```

3. Нельзя переслать данные между сегментными регистрами. При необходимости такая пересылка осуществляется через промежуточный регистр.

***Пример 2.46.***

```
MOV     ES, DS          ; Ошибка − пересылка между
                        ; сегментными регистрами
MOV     AX, ES          ; Пересылка между сегментными
MOV     DS, AX          ; регистрами (из ES в DS)
```

4. В качестве приемника данных не может использоваться регистр CS, т. к. его изменение эквивалентно передаче управления, что не является пересылкой данных.

Команда обмена XCHG имеет формат:

XCHG приемник, источник

и осуществляет обмен байтами или словами данных между источником и приемником. В качестве источника и приемника данных могут использоваться регистры МП, либо регистр и ячейка памяти.

***Пример 2.47.***

```
XCHG    AX, BX          ; Обмен между 16-битными
XCHG    AL, DH          ; или 8-битными регистрами
XCHG    Alpha, DX       ; Обмен между ячейкой
XCHG    AL, Beta        ; памяти и регистром
```

Ни один из операндов команды XCHG не может быть сегментным регистром.

Команда табличного преобразования XLAT записывается в формате:

XLAT или XLAT таблица−источник

и выполняет преобразование кода из AL в соответствии с выражением (AL) = ((BX) + (AL)).

Преобразование осуществляется по таблице, на которую указывает регистр BX. Следовательно, перед выполнением этой команды начальный адрес таблицы необходимо загрузить в регистр BX. Преобразуемый байт должен быть в регистре AL. При выполнении команды XLAT преобразуемый байт используется как индекс таблицы. Он складывается с начальным адресом таблицы, в результате чего формируется адрес ее строки (BX) + (AL), по которой осуществляется преобразование. По этому адресу считывается элемент таблицы и помещается в регистр AL.

Команда XLAT удобна для выполнения преобразований при отсутствии алгоритмической связи между преобразуемыми кодами.

***Пример 2.48.***

*Выполнить быстрое возведение в квадрат числа, лежащего в диапазоне* 0 – 5*, из ячейки памяти.*

```
TablConv  DB      0,1, 4, 9,16, 25          ; Описание таблицы
                                            ; преобразования
          . . . . . . . . . .
          MOV     AL, X                     ; Чтение операнда
          MOV     BX, OFFSET TablConv       ; Загрузка адреса таблицы
          XLAT    TablConv                  ; Преобразование
          MOV     Y, AL                     ; Запись результата
```

Указание в команде XLAT имени таблицы-источника необязательно, но предпочтительно, т. к. позволяет ассемблеру при трансляции программы проверить наличие таблицы и ее корректность.

**Команды загрузки адресных значений**

К этой подгруппе относятся команды LEA, LDS и LES.

Команда загрузки исполнительного адреса LEA имеет формат:

LEA регистр16, память

и загружает в указанный 16-битный регистр адрес (смещение offset) указанной ячейки памяти, адресуемой любым способом.

Во многих случаях команда LEA идентична команде MOV с директивой *OFFSET*. Однако следует помнить, что при использовании директивы *OFFSET* адрес вычисляется и должен быть известен на этапе трансляции программы, а при использовании команды LEA – при ее выполнении. Следовательно, при обращении к индексированной переменной с регистровым индексом (например, Tabl[SI]), неизвестным при трансляции, команда LEA правильно сформирует адрес элемента данных, а команда MOV *OFFSET* – неправильно. Директива *OFFSET* в подобных условиях возвращает адрес переменной без учета индекса, что приводит к возникновению ошибки, необнаруживаемой в процессе трансляции программы.

***Пример 2.49.***

Пусть имеется сегмент данных:

```
Data            SEGMENT
                DB        10 DUP(?)
        Tabl    DB        20h, 21h, 22h, 23h, 24h, 25h
Data            ENDS
```

Тогда при обращениях к таблице Tabl, получим:

```
LEA     BX, Tabl              ; Идентично MOV BX, OFFSET Tabl
MOV     AL, [BX]              ; (AL) = 20h
LEA     BX, Tabl[4]           ; Идентично MOV BX, OFFSET Tabl[4]
MOV     AL, [BX]              ; (AL) = 24h
MOV     SI, 3
LEA     BX, Tabl[SI]          ; Корректно
MOV     AL, [BX]              ; (AL) = 23h
MOV     BX, OFFSET Tabl[SI]   ; Необнаруживаемая ошибка
MOV     AL, [BX]              ; Ошибка – (AL) = 20h
```

Команды загрузки полных адресных значений LDS и LES имеют формат:

```
LDS     регистр16,  память32
LES     регистр16,  память32
```

и осуществляют загрузку двойного слова из ячейки памяти, адресуемой любым способом, в соответствующий сегментный регистр и указанный 16-битный регистр. При этом младшее слово, расположенное по адресу "память32", загружается в 16-битный регистр, а старшее слово, считываемое по адресу "память 32 + 2", командой LDS загружается в регистр DS, а командой LES − в регистр ES. В соответствии с этим младшее слово области "память32" должно содержать смещение offset, а старшее слово − сегментную компоненту segment выбираемой переменной. Поскольку команды LDS и LES загружают полный логический адрес, то программа может сразу обращаться к объекту, на который указывает этот адрес.

Обычно буфера адресных указателей инициализируются на требуемые значения директивой *DD* при трансляции программы. При необходимости они могут переопределяться двумя командами MOV с директивами *SEG* и *OFFSET*. Это приемлемо, т. к. программа читает адресные указатели гораздо чаще, чем изменяет их.

Команды LDS и LES наиболее часто используются для инициализации регистров при обработке строк.

***Пример 2.50.***

*Скопировать элемент одной строки в две других строки:*

```
String1  DB    80 DUP(?)        ; Описание строк
String2  DB    80 DUP(?)
String3  DB    80 DUP(?)
BuffSrc  DD    String1          ; Инициализация адресного буфера
                                ; источника на String1
BuffDst  DD    String2          ; Инициализация адресного буфера
                                ; приемника на String2
; . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . .
         LDS   SI, BuffSrc      ; Инициализация DS:SI на строку-источник
```

```
        LES     DI, BuffDst     ; Инициализация ES:DI на строку-приемник
        MOV     AL, [SI]        ; Копирование элемента
        MOV     ES: [DI], AL    ; String1 в String2
; . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . .
        MOV     AX, OFFSET String3              ; Переопределение
                                                ; адресного указателя
        MOV     WORD PTR BuffDst, AX            ; приемника на String3
        MOV     AX, SEG String3
        MOV     WORD PTR BuffDst + 2, AX
; . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . .
        LDS     SI, BuffSrc     ; Инициализация DS:SI на строку-источник
        LES     DI, BuffDst     ; Инициализация ES:DI на строку-приемник
        MOV     AL, [SI]        ; Копирование элемента
        MOV     ES: [DI], AL    ; String1 в String3.
```

**Команды обращения к стеку**

Команды обращения к стеку PUSH и POP имеют формат:

```
PUSH    источник
POP     приемник
```

и обеспечивают пересылку слов между стеком системы и регистрами МП или ячейками памяти.

Команда PUSH дважды декрементирует указатель стека SP и помещает содержимое указанного 16-битного регистра или ячейки памяти по этому адресу, т. е. на верхушку стека. В результате стек расширяется в сторону меньших адресов. Регистр SP всегда указывает на последнее слово, включенное в стек.

Команда POP извлекает слово с верхушки стека и помещает его в указанный 16-битный регистр или ячейку памяти. После этого содержимое указателя стека SP дважды инкрементируется, и он вновь указывает на верхушку стека. В команде POP в качестве приемника данных нельзя указывать сегментный регистр CS, т. к. его изменение будет эквивалентно переходу, что не должно реализовываться командами пересылки данных.

Будучи взаимно обратными, команды PUSH и POP обычно используются парами, т. е. каждой команде PUSH в программе должна соответствовать своя команда POP. При этом последовательность команд POP по используемым операндам должна быть обратна последовательности команд PUSH.

***Пример 2.51.***

```
        PUSH    SI              ; Сохранение регистров в стеке
        PUSH    DS
        . . . . . . . . . . .
        POP     DS              ; Восстановление регистров из стека
```

```
POP     SI
PUSH    Counter             ; Сохранение слов памяти в стеке
PUSH    Table[BX][DI]
. . . . . . . . . . .
POP     Alpha               ; Запись в память слов из стека
POP     [BX + SI]
```

Чаще всего стек используется для временного хранения данных. Однако он может использоваться и в качестве буфера при некоторых пересылках.

***Пример 2.52.***

```
PUSH    ES                  ; Копирование ES в DS
POP     DS                  ; через стек
MOV     AX, ES              ; Альтернативное копирование
MOV     DS, AX              ; ES в DS через регистр
```

**Команды пересылки флагов**

К этой подгруппе относятся команды PUSHF, POPF и LAHF, SAHF.

Команды PUSHF и POPF пересылают содержимое полного регистра флагов F в стек и обратно соответственно. Они практически идентичны командам PUSH и POP, но не требуют указания операнда, т. к. таковым для них является только регистр флагов.

После пересылки в стек может быть изменено состояние флага трассировки TF(бит D8), что является единственным способом его формирования, т. к. команды прямого управления этим флагом отсутствуют. После загрузки установленного в единичное значение флага TF в регистр флагов, МП перейдет в режим трассировки.

***Пример 2.53.***

```
PUSHF                           ; Переключение МП
MOV     BP, SP                  ; в режим трассировки
OR      WORD PTR[BP], 0100h
POPF
```

Команды LAHF и SAHF пересылают младший байт регистра флагов F соответственно в регистр AH и обратно. Эти команды введены для обеспечения программной совместимости МП ВМ86/ВМ88 с МП ВМ80/ВМ85.

***Пример 2.54.***

```
LAHF                            ; Эквивалент команды PUSH PSW
PUSH    AX                      ; МП ВМ80/ВМ85
. . . . . . . . . . .
POP     AX                      ; Эквивалент команды POP PSW
SAHF                            ; МП ВМ80/ВМ85
```

**Команды ввода/вывода**

К этой подгруппе относятся команды IN и OUT.

Команда ввода IN осуществляет ввод байта или слова из порта ввода в аккумулятор МП. В ней может использоваться прямая или косвенная адресация порта ввода. В соответствии с этим она записывается в двух форматах:

```
IN    аккумулятор, порт
IN    аккумулятор, DX.
```

В качестве аккумулятора при пересылке байтов должен указываться регистр AL, а при пересылке слов − регистр AX.

При прямой адресации адрес порта "порт" указывается прямо в команде и может иметь десятичное значение от 0 до 255, что позволяет адресоваться к 256 однобайтным или двухбайтным портам ввода.

При косвенной адресации адрес порта выбирается из регистра DX, что позволяет адресоваться к 64 Кбайт однобайтных портов или 32 Кбайт двухбайтных портов.

***Пример 2.55.***

```
IN      AX, 200          ; Ввод слова из порта 200
IN      AL, PortPrint    ; Ввод байта из порта, заданного
                         ; поименованной константой PortPrint
IN      AL, DX           ; Ввод байта из порта по адресу из DX
```

Команда вывода OUT осуществляет вывод байта или слова из аккумулятора МП в порт вывода. С точки зрения используемых аккумуляторов и режимов адресации она идентична команде ввода IN и записывается в следующих форматах:

```
OUT    порт, аккумулятор
OUT    DX,  аккумулятор.
```

***Пример 2.56.***

```
OUT     30h, AX          ; Вывод слова в порт 30h
OUT     DX, AL           ; Вывод байта в порт, указанный в DX
```

Все команды пересылки данных, кроме POPF и LAHF, не изменяют флагов МП.

## 2.13.2. Арифметические команды

Арифметические команды обеспечивают выполнение всех арифметических операций над данными различных типов. Их можно разделить на следующие подгруппы:

1) команды инкрементирования и декрементирования;
2) команды сложения и вычитания;
3) команды расширения знака;
4) команды умножения и деления;
5) команды десятичной коррекции.

Прежде, чем приступить к описанию самих команд, рассмотрим типы данных, которыми они оперируют.

### Типы арифметических данных

МП ВМ86/ВМ88 могут обрабатывать двоичные и десятичные числа, представленные в следующей форме:

1) двоичные беззнаковые целые числа;
2) двоичные знаковые целые числа;
3) десятичные числа в упакованном BCD-формате;
4) десятичные числа в неупакованном ASCII-формате.

Двоичные числа могут иметь длину 8 или 16 битов. У числа без знака все биты представляют его значение. Следовательно, двоичные числа без знака могут принимать значения от 0 до 255 (8-битные) или до 65535 (16-битные).

У чисел со знаком старший бит (D7 или D15) указывает знак числа (0 – число положительно или равно нулю, 1 – число отрицательно), а остальные биты – его значение. Числа со знаком представляются в дополнительном коде и могут принимать значения от (−128) до +127 (8-битные) или от (−32768) до +32767 (16-битные). Более длинные двоичные числа представляются последовательностью байтов или слов в памяти и обрабатываются последовательно.

Десятичные числа могут быть только беззнаковыми целыми числами. При этом десятичные операции выполняются только над байтами, и десятичные числа представляются последовательностью байтов в упакованном или неупакованном формате. В упакованном BCD-формате каждый байт содержит две десятичные цифры в коде 8−4−2−1. Следовательно, один десятичный байт в BCD-формате может содержать значения от 00 до 99.

В неупакованном ASCII-формате каждый байт десятичного числа содержит только одну десятичную цифру в коде 8−4−2−1 в младшей тетраде. Поэтому десятичный байт в ASCII-формате может содержать значения от 0 до 9. При этом старшая тетрада в операциях умножения и деления должна иметь нулевое значение, а в операциях сложения и вычитания ее значение несущественно.

Каким же образом МП узнает с каким типом данных он имеет дело? Он об этом совершенно не заботится и трактует все операнды только как двоичные числа. Необходимая трактовка операндов осуществляется программистом путем соответствующей обработки данных.

### Команды инкрементирования и декрементирования

К этой подгруппе относятся команды INC и DEC.

Команда инкрементирования INC записывается в формате:

INC приемник

и увеличивает на 1 содержимое приемника.

Команда декрементирования DEC имеет формат:

DEC приемник

и уменьшает на 1 содержимое приемника.

В качестве приемника могут использоваться 8-битные или 16-битные регистр или ячейка памяти.

***Пример 2.57.***

```
INC     BX              ; Увеличение на 1 значения 16-битного
INC     AL              ; или 8-битного регистра
INC     BYTE PTR [BX]   ; Увеличение на 1 байта по адресу [BX]
DEC     CX              ; Уменьшение на 1 16-битного регистра
DEC     Alfa            ; Уменьшение на 1 байта или слова
                        ; в памяти в зависимости от типа Alfa
DEC     WORD PTR [DI]   ; Уменьшение на 1 слова по адресу [DI]
```

Особенностью команд INC и DEC является то, что они не изменяют флаг переноса CF. Поэтому эти команды используются для модификации адресных указателей в циклических программах тогда, когда флаг переноса несет полезную информацию от одного прохода тела цикла к другому (например, сложение или вычитание двух длинных чисел).

**Команды сложения и вычитания**

К этой подгруппе относятся команды ADD, ADC, SUB, SBB и NEG.

Команды сложения ADD и сложения с переносом ADC записываются в форматах:

ADD приемник, источник
ADC приемник, источник

и выполняют сложение как 8-битных, так и 16-битных операндов из регистров или из регистра и ячейки памяти. При этом операнды могут быть как знаковыми, так и беззнаковыми числами. В качестве источника может использоваться и непосредственное значение.

Команда ADD реализует операцию

(приёмник) = (приёмник) + (источник),

а команда ADC –

(приёмник) = (приёмник) + (источник) + (CF).

***Пример 2.58.***

```
ADD     AX, DX          ; (AX) = (AX) + (DX)
ADD     Alfa, AL        ; (Alfa) = (Alfa) + (AL), где Alfa DB ?
```

```
ADC     DX, 20          ; (DX) = (DX) + 20 + (CF)
ADC     [BX], AX        ; ((BX)) = ((BX)) + (AX) + (CF)
```

Команда ADD используется для сложения коротких чисел (байтов или слов) или младших элементов более длинных чисел, а команда ADC − для сложения старших элементов длинных чисел, обрабатываемых последовательно. Кроме того, команда ADC может применяться для включения флага переноса CF в байт или слово, например, при формировании старшего элемента арифметической суммы двух чисел.

***Пример 2.59.***

*Вычислить арифметическую сумму* S = A + B*, где* A *и* B *− двойные слова без знака.*

Результат сложения двух операндов в общем случае длиннее одного операнда на 1 бит, который должен быть включен в старший элемент результата.

```
A       DD      ?                       ; Описание операндов
B       DD      ?                       ; в памяти
S       DD      ?                       ; Описание результата
        DW      ?
        . . . . . . . . . . . .
        MOV     AX, WORD PTR A          ; Сложение младших
        ADD     AX, WORD PTR B          ; слов A и B
        MOV     WORD PTR S, AX
        MOV     AX, WORD PTR A + 2      ; Сложение старших
        ADC     AX, WORD PTR B + 2      ; слов A и B
        MOV     WORD PTR S + 2, AX
        MOV     AX, 0                   ; Формирование
        ADC     AX, 0                   ; старшего слова
        MOV     WORD PTR S + 4, AX      ; результата S
```

Команды ADD и ADC формируют все арифметические флаги по результату операции обычным образом. При этом флаг переноса CF равен 1, если результат сложения чисел без знака не помещается в операнде-приемнике. Флаги SF и OF имеют смысл только при сложении чисел со знаком.

Команды вычитания SUB и вычитания с заемом SBB имеют формат:

```
SUB     приемник, источник
SBB     приемник, источник
```

и выполняют вычитание как 8-битных, так и 16-битных операндов из регистров или из регистра и ячейки памяти. При этом операнды могут быть как знаковыми, так и беззнаковыми числами. В качестве источника может использоваться и непосредственное значение.

Команда SUB реализует операцию

(приёмник) = (приёмник) − (источник),

а команда SBB −

(приёмник) = (приёмник) − (источник) − (CF).

Перенос CF в операциях вычитания играет роль заема.

***Пример 2.60.***

```
SUB     AX, DX          ; (AX) = (AX) − (DX)
SUB     AX, MemWord     ; Вычитание из регистра содержимого
SUB     Tabl[SI], AX    ; ячейки памяти или наоборот
SBB     DX, 50          ; (DX) = (DX) − 50 − (CF)
```

Применение команд SUB и SBB аналогично командам ADD и ADC. Команда SUB используется для вычитания коротких чисел (байтов или слов) или младших элементов более длинных чисел, а команда SBB − для вычитания старших элементов длинных чисел, обрабатываемых последовательно.

***Пример 2.61.***

*Вычислить разность* S = A − B*, где* A *и* B *− двойные слова без знака.*

```
A       DD      ?                       ; Описание операндов
B       DD      ?                       ; в памяти
S       DD      ?                       ; Описание результата
        DW      ?
        . . . . . . . . . . . . .
        MOV     AX, WORD PTR A          ; Вычитание младших
        SUB     AX, WORD PTR B          ; слов A и B
        MOV     WORD PTR S, AX
        MOV     AX, WORD PTR A + 2      ; Вычитание старших
        SBB     AX, WORD PTR B + 2      ; слов A и B
        MOV     WORD PTR S + 2, AX
        MOV     AX, 0                   ; Формирование старшего
        SBB     AX, 0                   ; слова результата
        MOV     WORD PTR S + 4, AX      ; (знака разности)
```

Команды SUB и SBB формируют все арифметические флаги по результату операции обычным образом. При этом флаг переноса CF равен 1, если при вычитании требуется заем, т. е. результат вычитания чисел без знака отрицателен. Флаги SF и OF имеют смысл только при вычитании чисел со знаком.

МП BM86/BM88 не имеют устройства вычитания, поэтому вычитание реализуется путем сложения уменьшаемого с вычитаемым, взятым с обратным знаком, т. е. R = X−Y = X + (−Y). Поэтому при выполнении команд SUB и SBB прежде всего обращается знак вычитаемого путем преобразования его в дополнительный код, а затем реализуется сложение.

***Пример 2.62.***

*Найти разность чисел* 7 *и* 10 *в кодовой форме.*

R1 = 10 − 7 = 10 + (−7) = + 3  R2 = 7 − 10 = 7 + (−10) = –3.

Цифровое представление чисел байтом имеет вид:

10 = 00001010,  7 = 00000111.

Обращение знака вычитаемого:

−7 = 11111000 + 1 = 11111001; −10 = 11110101 + 1 = 11110110.

Результирующее сложение:

$$R1 = {}_{+}00001010 \qquad R2 = {}_{+}00000111$$
$$\underline{11111001} \qquad \underline{11110110}$$
$$CF = \overline{1} \leftarrow 00000011 \qquad CF = \overline{0} \leftarrow 11111101$$

Результат R1 = 00000011 = 3, CF = 0; R2 = 11111101 = −3, CF = 1.

Отсюда следует, что:

1) для сохранения правила формирования флага CF ему присваивается инверсное значение по отношению к переносу, рождающемуся в операции сложения;

2) отрицательная разность представляется в дополнительном коде.

Команда обращения знака NEG имеет формат:

NEG приемник

и изменяет знак операнда путем его вычитания из нуля. Таким образом, команда NEG реализует операцию

(приёмник) = −(приёмник).

В качестве операнда может использоваться 8-ми или 16-битные регистр или ячейка памяти.

***Пример 2.63.***

```
NEG    AX                ; Обращение знака 16-битного
NEG    DL                ; или 8-битного регистра
NEG    Alpha             ; Обращение знака ячейки
NEG    WORD PTR [BX]     ; памяти
```

Если операнд равен нулю, то его значение не изменяется и формируется флаг CF = 0. Во всех остальных случаях CF = 1. При попытке обращения знака байта со значением (−128) = 80h или слова со значением (−32768) = 8000h операнд не изменяется, но устанавливается флаг переполнения OF = 1, т. к. обращенные значения не входят в диапазон знаковых чисел.

Команда NEG используется для вычитания значения регистра или ячейки памяти из непосредственного значения.

***Пример 2.64.***

*Вычислить* S = 100 − BM*, где* BM *− байт в памяти.*

Поскольку непосредственное значение не может быть приемником, то команда SUB 100, BM недопустима, и вычисление осуществляется по схеме S = 100 + (−BM).

```
MOV     AL, BM          ; Обращение знака
NEG     AL              ; содержимого BM
ADD     AL, 100         ; Вычисление S = 100 − BM
MOV     S, AL
```

**Команды расширения знака**

К этой подгруппе относятся команды CBW и CWD, не имеющие операндов и предназначенные для преобразования форматов чисел со знаком путем его расширения.

Команда CBW преобразует байт из регистра AL в слово в регистре AX путем дублирования бита D7 из AL во всех битах AH.

Команда CWD преобразует слово из регистра AX в двойное слово в регистрах DX,AX путем дублирования бита D15 из AX во всех битах регистра DX.

Команды CBW и CWD используются для расширения диапазона представления знаковых чисел в случае, если в результате операций над ними возможно переполнение разрядной сетки.

***Пример 2.65.***

*Вычислить арифметическую сумму* S = A + B*, где* A *и* B *– байты со знаком.*

В результате сложения двух операндов в общем случае возможно переполнение и, следовательно, результат может полностью исказиться. Для получения достоверного результата его необходимо формировать в накопителе большей разрядности. Это достигается путем расширения исходных операндов до размера результата перед выполнением операции.

```
MOV     AL, A           ; Расширение первого
CBW                     ; операнда
MOV     DX, AX
MOV     AL, B           ; Расширение второго
CBW                     ; операнда
ADD     AX, DX          ; Сложение операндов
MOV     S, AX
```

Кроме того, команды CBW и CWD используются для преобразования форматов чисел при выполнении арифметических операций над смешанными данными (сложение слова и байта, сложение знакового и беззнакового числа и т. д.).

***Пример 2.66.***

```
; Сложение разноформатных однотипных чисел
        MOV     AL, Alfa            ; Сложение байта со знаком Alfa
        CBW                         ; и слова со знаком Beta
        ADD     AX, Beta            ; из памяти

; Сложение одноформатных разнотипных чисел
        MOV     DL, Alpha           ; Расширение байта
        MOV     DH, 0               ; без знака
        MOV     AL, Beta            ; Расширение байта
        CBW                         ; со знаком
        ADD     AX, DX              ; Сложение разнотипных байтов
```

**Команды умножения и деления**

К этой подгруппе относятся команды MUL,IMUL и DIV,IDIV.

Команда MUL выполняет умножение чисел без знака, а команда IMUL − умножение чисел со знаком. Обе команды могут умножать как байты, так и слова и записываются в формате:

MUL источник
IMUL источник.

В качестве источника может указываться 8-битный или 16-битный регистр общего назначения или ячейка памяти. Содержимое источника служит первым операндом операции умножения. Второй операнд по умолчанию берется из регистра AL (при умножении на байт) или из регистра AX (при умножении на слово). Произведение имеет двойной размер и возвращается в регистрах AX или DX,AX соответственно. Таким образом, команды MUL и IMUL реализуют операции:

(AX) = (AL) ∗ источник8,
(DX,AX) = (AX) ∗ источник16.

***Пример 2.67.***

```
MUL     BX              ; Умножение без знака (DX,AX) = (AX)*(BX)
IMUL    DL              ; Умножение со знаком (AX) = (AL)*(DL)
MUL     BYTE PTR [BX]   ; Умножение без знака на байт
                        ; из памяти:(AX) = (AL)*((BX))
```

Команды MUL и IMUL не позволяют умножать на непосредственное значение. При необходимости непосредственное значение загружается в регистр или ячейку памяти и выполняется умножение на них.

***Пример 2.68.***

```
MOV     DL, 10          ; Умножение на константу
MUL     DL              ; (AX) = (AL) * 10
```

После выполнения команд MUL и IMUL флаги переноса CF и переполнения OF показывают значимость старшей части произведения для последующих вычислений. Флаги CF = OF = 0, если старшая половина произведения равна 0 (для MUL) или содержит лишь расширение знака младшей половины (для IMUL); в противном случае CF = OF = 1. Состояние остальных арифметических флагов изменяется неопределенным образом.

Команда DIV выполняет деление чисел без знака, а команда IDIV − деление чисел со знаком. Обе команды могут делить как на байт, так и на слово и записываются в формате:

```
DIV     источник
IDIV    источник.
```

В качестве источника может указываться 8-битный или 16-битный регистр общего назначения или ячейка памяти. Содержимое источника служит делителем операции деления. Делимое имеет двойной размер и по умолчанию берется из регистра AX (при делении на байт) или из регистров DX, AX (при делении на слово). Результат деления представляет собой частное и остаток и возвращается следующим образом:

− при делении на байт:
  частное − в регистре AL, остаток − в регистре AH;
− при делении на слово:
  частное − в регистре AX, остаток − в регистре DX.

Таким образом, команды DIV и IDIV реализуют операции:

(AL) = (AX)/(источник8), (AH) = (AX) mod (источник8),
(AX) = (DX, AX)/(источник16), (DX) = (DX, AX) mod (источник16),

где mod − операция формирования остатка от целочисленного деления.

***Пример 2.69.***

```
DIV     BX              ; Деление без знака (DX,AX) на (BX)
IDIV    DL              ; Деление со знаком (AX) на (DL)
IDIV    WORD PTR [DI]   ; Деление со знаком (DX,AX) на слово
                        ; из памяти
```

Команды DIV и IDIV не позволяют делить на непосредственное значение. При необходимости непосредственное значение загружается в регистр или ячейку памяти и выполняется деление на них.

***Пример 2.70.***

```
        MOV     BX, 20          ; Деление (DX, AX)
        IDIV    BX              ; на константу 20
```

При знаковом делении теоретически результат операции является неоднозначным.

Например, (−26)/(+7) = −3(остаток= −5) или (−26)/(+7)= −4(остаток = +2). Любой из полученных результатов верен, т. к. справедливо проверочное соотношение для деления: делимое = делитель ∗ частное + остаток.

При выполнении команды IDIV остатку всегда присваивается знак делимого, и для рассмотренного примера результатом будет частное = −3, остаток = −5.

Если при выполнении команд DIV и IDIV делитель равен 0 или частное не помещается в регистре-приемнике (AL или AX), то автоматически генерируется прерывание по ошибке деления типа 0. В этом случае частное и остаток неопределены, а управление передается подпрограмме обработки прерывания, которая принимает все необходимые решения.

Команды DIV и IDIV оставляют все арифметические флаги в неопределенном состоянии.

Для умножения или деления данных, форматы которых не удовлетворяют требованиям соответствующих команд (умножение байта на слово, деление слова на слово), более короткий операнд преобразуется к стандартной форме путем расширения знака (для знаковых чисел) или загрузкой нулей в нужный регистр (для чисел без знака).

***Пример 2.71.***

```
; Умножение знаковых байта и слова
        MOV     AL, Alfa        ; Расширение байта Alfa
        CBW
        IMUL    Beta            ; Умножение на слово Beta

; Умножение байта со знаком на байт без знака
        MOV     AL, S_Byte      ; Расширение байта
        CBW                     ; со знаком
        MOV     DL, Byte        ; Расширение байта
        MOV     DH, 0           ; без знака
        IMUL    DX              ; Умножение

; Деление знаковых слова на слово
        MOV     AX, Alfa        ; Расширение слова
        CWD                     ; со знаком
        IDIV    Beta            ; Деление
```

## Команды десятичной коррекции

Выполнение любой арифметической операции над десятичными числами сводится к выполнению соответствующей двоичной операции с последующей десятичной коррекцией результата или предшествующей коррекцией операнда. При этом десятичные числа могут обрабатываться только побайтно, и все операции должны выполняться в регистре AL, т. к. вся коррекция реализуется только для этого регистра.

## Сложение в BCD-формате

Сложение в BCD-формате выполняется в два этапа: сначала байты операндов суммируются как двоичные числа с помощью команд ADD или ADC, а затем осуществляется коррекция результата командой DAA. Коррекция реализуется следующим образом:

1) если $(AF = 1) \vee (AL_L > 9)$, то $(AL) = (AL) + 06h$;
2) если $(CF = 1) \vee (AL_H > 9)$, то $(AL) = (AL) + 60h$.

Команда DAA преобразует содержимое регистра AL в правильный BCD-формат. Коррекция заключается в последовательном прибавлении корректирующих шестерок к младшей и старшей тетрадам AL при наличии необходимых условий. При каждом корректирующем сложении учитываются возникающие переносы.

***Пример 2.72.***

*Вычислить* Z = X + Y, *где* X, Y – *десятичные четырехразрядные BCD-числа в памяти.*

```
MOV     AL, X           ; Сложение младших
ADD     AL, Y           ; байтов
DAA                     ; Коррекция
MOV     Z, AL
MOV     AL, X + 1       ; Сложение старших
ADC     AL, Y + 1       ; байтов
DAA                     ; Коррекция
MOV     Z + 1, AL
MOV     AL, 0           ; Формирование
ADC     AL, 0           ; старшего байта
DAA                     ; результата
MOV     Z + 2, AL
```

Команда DAA правильно формирует все арифметические флаги, кроме OF, значение которого не определено.

## Вычитание в BCD-формате

Вычитание в BCD-формате также выполняется в два этапа: сначала байты операндов вычитаются как двоичные числа с помощью команд SUB или SBB, а

затем осуществляется коррекция результата командой DAS. Коррекция реализуется следующим образом:

1) если (AF = 1) $\vee$ ($AL_L > 9$), то (AL) = (AL)–06h;

2) если (CF = 1) $\vee$ ($AL_H > 9$), то (AL) = (AL) – 60h.

Команда DAS преобразует содержимое регистра AL в правильный BCD-формат. Коррекция заключается в последовательном вычитании корректирующих шестерок из младшей и старшей тетрад AL при наличии необходимых условий. При каждом корректирующем вычитании учитываются возникающие заемы.

***Пример 2.73.***

*Вычислить* Z = X – Y*, где* X, Y *– десятичные BCD-слова в памяти.*

```
MOV     AL, X           ; Вычитание младших
SUB     AL, Y           ; байтов
DAS                     ; Коррекция
MOV     Z, AL
MOV     AL, X + 1       ; Вычитание старших
SBB     AL, Y + 1       ; байтов
DAS                     ; Коррекция
MOV     Z + 1, AL
MOV     AL, 0           ; Формирование
SBB     AL, 0           ; старшего байта
DAS                     ; результата
MOV     Z + 2, AL
```

Команда DAS правильно формирует все арифметические флаги, кроме OF, значение которого не определено.

При получении отрицательной разности флаг CF = 1, и результат представлен в дополнительном десятичном коде.

Умножение и деление в BCD-формате невозможно. Это обусловлено тем, что в данном формате байт содержит две десятичные цифры, а простых алгоритмов умножения и деления сразу на две цифры неизвестно.

Для умножения или деления чисел в BCD-формате их необходимо либо преобразовать в двоичную форму и воспользоваться двоичной арифметикой, либо представить в ASCII-формате и воспользоваться десятичной арифметикой для этого формата.

### Сложение в ASCII-формате

Сложение в ASCII-формате также выполняется в два этапа: сначала байты операндов суммируются как двоичные числа с помощью команд ADD или ADC, а затем осуществляется коррекция результата командой AAA. Коррекция реализуется в соответствии с алгоритмом, приведенным на рис. 2.9,а.

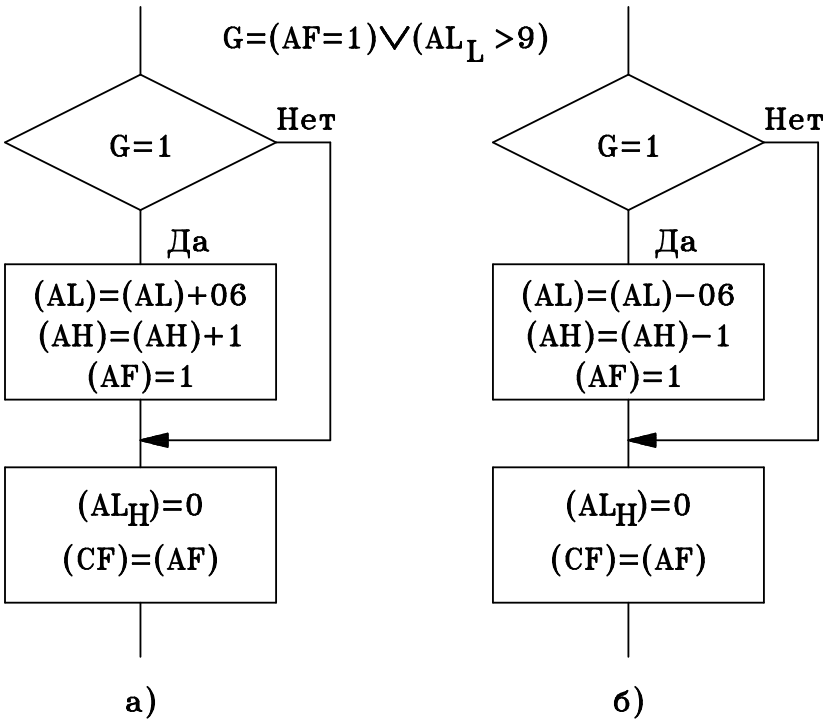


Рис. 2.9. Алгоритмы коррекции ASCII-сложения и ASCII-вычитания:
а) коррекция сложения командой AAA; б) коррекция вычитания командой AAS

Если младшая тетрада регистра AL содержит число, большее 9, или флаг AF = 1, то выполняется корректирующее сложение и увеличивается на 1 содержимое регистра AH. Далее обнуляется старшая тетрада регистра AL и флагу основного переноса CF присваивается значение флага AF. Таким образом, перенос, возникающий при сложении, учитывается дважды: в регистре AH и флаге CF. Значение CF используется при сложении длинных чисел, а инкрементирование AH при сложении не имеет смысла и используется при реализации некоторых алгоритмов умножения в ASCII-формате. Однако необходимо помнить, что команда AAA изменяет содержимое регистра AH, и, если оно существенно, то его необходимо сохранять.

***Пример 2.74.***

*Вычислить* Z = X + Y*, где* X, Y *– десятичные двухразрядные ASCII-числа в памяти.*

```
MOV     AL, X           ; Сложение младших
ADD     AL, Y           ; байтов
AAA                     ; Коррекция (внимание – (AH)!)
MOV     Z, AL
MOV     AL, X + 1       ; Сложение старших
ADC     AL, Y + 1       ; байтов
AAA                     ; Коррекция (внимание – (AH)!)
MOV     Z + 1, AL
MOV     AL, 0           ; Формирование
```

```
ADC     AL, 0           ; старшего байта
AAA                     ; результата
MOV     Z + 2, AL
```

Команда AAA правильно формирует флаг CF. Все остальные арифметические флаги после ее выполнения не определены.

### Вычитание в ASCII-формате

Вычитание в ASCII-формате также выполняется в два этапа: сначала байты операндов вычитаются как двоичные числа с помощью команд SUB и SBB, а затем осуществляется коррекция результата командой AAS. Коррекция реализуется в соответствии с алгоритмом, приведенным на рис. 2.9,б.

Если младшая тетрада регистра AL содержит число большее 9 или флаг AF = 1, то выполняется корректирующее вычитание и уменьшается на 1 содержимое регистра AH. Далее обнуляется старшая тетрада регистра AL и флагу основного переноса CF присваивается значение флага AF. Таким образом, заем, возникающий при вычитании, учитывается дважды: в регистре AH и флаге CF. Значение CF используется при вычитании длинных чисел, а декрементирование регистра AH при вычитании не имеет смысла и используется при реализации некоторых алгоритмов деления в ASCII-формате. Однако необходимо помнить, что команда AAS изменяет содержимое регистра AH, и, если оно существенно, то его необходимо сохранять.

***Пример 2.75.***

*Вычислить* Z = X – Y*, где* X*,* Y *– десятичные двухразрядные ASCII-числа в памяти.*

```
MOV     AL, X           ; Вычитание младших
SUB     AL, Y           ; байтов
AAS                     ; Коррекция (внимание – (AH)!)
MOV     Z, AL
MOV     AL, X + 1       ; Вычитание старших
SBB     AL, Y + 1       ; байтов
AAS                     ; Коррекция (внимание – (AH)!)
MOV     Z + 1, AL
MOV     AL, 0           ; Формирование
SBB     AL, 0           ; старшего байта
AAS                     ; результата
MOV     Z + 2, AL
```

Команда AAS правильно формирует флаг CF. Все остальные арифметические флаги после ее выполнения не определены.

При получении отрицательной разности флаг CF = 1, и результат представлен в дополнительном десятичном коде.

## Умножение в ASCII-формате

Умножение в ASCII-формате выполняется в два этапа: сначала байты операндов умножаются как двоичные числа с помощью команды MUL, а затем осуществляется коррекция результата командой AAM.

При умножении двух неупакованных чисел, находящихся в диапазоне 0 – 9, командой MUL будет получен результат в диапазоне 0 – 81, представленный двоичным числом. Команда коррекции AAM преобразует полученный результат из двоичной формы в ASCII-формат. Для этого она осуществляет деление содержимого регистра AL на 10 с размещением частного в регистре AH, а остатка – в регистре AL. Таким образом, команда AAM реализует операции:

$$(AH) = (AL)/10, \quad (AL) = (AL) \bmod 10.$$

Отсюда следует, что команду AAM можно рассматривать как специализированную команду деления беззнакового числа из регистра AL на 10.

Команда AAM правильно формирует флаги SF, ZF и PF по результату в регистре AL. При этом всегда SF = 0, т. к. результат коррекции представлен в ASCII-формате, а ZF = 1, если исходное число кратно 10. Флаги CF, AF и OF изменяются неопределенным образом.

***Пример 2.76.***

*Вычислить* Z = X ∗ Y*, где* X *– десятичное двухразрядное ASCII-число, а* Y *– ASCII-байт в памяти.*

Для умножения в ASCII-формате необходимо прежде всего представить схему умножения, которая для заданного примера имеет вид:

$$\begin{array}{l} \quad X_1\ X_0 \\ * \qquad Y_0 \\ \hline + \quad Z'_{00}\ Z_{00} \\ Z'_{01}\ Z_{01} \\ \hline Z_2\ \ Z_1\ \ Z_0 \end{array} \qquad\qquad \begin{array}{r} 89 \\ * \quad 6 \\ \hline + \ 54 \\ 48\ \ \\ \hline 534 \end{array}$$

Из анализа этой схемы следует, что вычисление произведения сводится к вычислению частичных произведений с последующим их суммированием с учетом соответствующих весов.

Программа, реализующая эту схему, имеет вид:

```
MOV     AL, X          ; Загрузка X0
MUL     Y              ; Вычисление Z'00Z00 = X0*Y0
AAM                    ; Коррекция умножения
MOV     Z, AX          ; Запись Z'00Z00 в память
MOV     AL, X + 1      ; Загрузка X1
MUL     Y              ; Вычисление Z'01Z01 = X1*Y0
```

```
AAM                     ; Коррекция умножения
ADD     AL, Z + 1       ; Сложение Z'00 + Z01
AAA                     ; Коррекция сложения
MOV     Z + 1, AX       ; Запись Z2Z1 в память
```

Рассмотренные алгоритм и программа умножения чисел в ASCII-формате могут быть обобщены для умножения чисел произвольной разрядности.

### Деление в ASCII-формате

Деление в ASCII-формате выполняется в два этапа: сначала осуществляется десятичная коррекция делимого с помощью команды AAD, а затем выполняется деление операндов командой DIV.

Команда коррекции AAD преобразует неупакованное делимое, находящееся в регистрах AX, AL из ASCII-формата в двоичную форму и загружает его в регистр AL. При этом она предполагает, что регистр AH содержит старшую цифру, а регистр AL – младшую цифру делимого с нулевым значением старших тетрад в обоих регистрах.

Таким образом, при выполнении команды AAD реализуются следующие действия:

1) вычисляется значение (AL) = (AH) ∗ 10 + (AL);

2) обнуляется регистр AH.

После коррекции делимое находится в регистре AX в двоичной форме и готово для деления на одноразрядный десятичный делитель в ASCII-формате, которое и выполняется командой DIV.

Команда AAD правильно формирует флаги SF,ZF и PF по результату в регистре AL. Остальные арифметические флаги становятся неопределенными.

***Пример 2.77.***

*Вычислить Z = X/Y, где X – десятичное двухразрядное ASCII-число, а Y – ASCII-байт в памяти.*

В соответствии с правилами арифметики деление начинается со старшей цифры.

```
MOV     AH, 0           ; Загрузка старшей цифры
MOV     AL, X + 1       ; делимого в AX
DIV     Y               ; Деление старшей цифры
MOV     Z + 1, AL       ; Запись старшей цифры частного
MOV     AL, X           ; Загрузка младшей цифры делимого
AAD                     ; Коррекция делимого
DIV     Y               ; Деление младшей цифры
MOV     Z, AL           ; Запись младшей цифры частного
MOV     R, AH           ; Запись остатка
```

Рассмотренные алгоритм и программа деления чисел в ASCII-формате могут быть обобщены для деления многоразрядного делимого на одноразрядный делитель.

При необходимости деления чисел в ASCII-формате на многоразрядный делитель все операнды должны быть преобразованы в двоичную форму и выполнено двоичное деление с последующим преобразованием результата в ASCII-формат.

### 2.13.3 Логические команды

Логические команды предназначены для побитовой обработки данных. Их можно разделить на следующие подгруппы:

1) команды логических операций;
2) команды сравнения;
3) команды сдвигов.

**Команды логических операций**

К этой подгруппе относятся команды AND, OR, XOR и NOT, реализующие поразрядные логические операции над операндами. Первые три команды записываются в формате:

AND приемник, источник
OR приемник, источник
XOR приемник, источник

и выполняют соответственно операции "логическое И", "логическое ИЛИ" и "исключающее ИЛИ" над соответствующими битами операндов приемника и источника. Результаты исполнения этих команд для однобитных операндов приведены в табл. 2.3.

Таблица 2.3

Таблица истинности команд AND, OR, XOR

| Операнды | | Результат | | |
|---|---|---|---|---|
| Приемник | Источник | AND | OR | XOR |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 |

Операндами команд AND, OR и XOR могут быть байты или слова. В качестве приемника и источника данных могут использоваться регистры МП или регистр и ячейка памяти. Кроме того, источник может быть и непосредственным значением.

Команда конъюнкции AND используется для сброса битов операнда-приемника. После выполнения этой команды сбрасываются те биты, которым в операнде-источнике соответствуют 0. Команда AND, как правило, применяется для выделения указанных полей операнда-приемника. Выделяемые поля указываются значениями битов операнда-источника, называемого маской. Маска должна содержать единицы в разрядах, соответствующих выделяемым полям, и нули во всех остальных. После выполнения команды AND выделяемые поля приемника сохраняют свое значение, а все остальные обращаются в нуль.

***Пример 2.78.***

```
AND    AX, DX            ; Логическое И над двумя регистрами
AND    AL, MemByte       ; или регистром и ячейкой памяти
AND    [BX], DX
AND    AL, 00001111b     ; Выделение младшей тетрады регистра AL
```

Команда дизъюнкции OR предназначена для объединения полей операндов приемника и источника. После выполнения этой команды те биты операнда-приемника, в которых хотя бы один из операндов содержит 1, устанавливаются в 1. Команда OR обычно используется для установки указанных полей операнда-приемника в 1. Устанавливаемые поля указываются значениями битов операнда-источника (маски). Маска должна содержать единицы в разрядах, соответствующих устанавливаемым полям, и нули во всех остальных. После выполнения команды OR устанавливаемые поля приемника имеют единичное значение, а все остальные не изменяются.

***Пример 2.79.***

```
OR     AL, DL            ; Объединение двух регистров
OR     Table[SI], DX     ; или регистра и ячейки памяти
OR     AX, 8000H         ; Установка разряда D15 регистра AX в 1
```

***Пример 2.80.***

*Переключить МП в режим трассировки и обратно.*

Эта задача решается путем установки и сброса флага трассировки TF. Это реализуется через память с помощью команд AND или OR с соответствующей маской:

```
  ; Включение режима трассировки
PUSHF                          ; Запись в стек регистра флагов
MOV    BP, SP                  ; Настройка адреса на верхушку стека
OR     WORD PTR [BP], 100h     ; Установка флага TF
POPF                           ; Загрузка регистра флагов
. . . . . . . . . . . . . .
  ; Выключение режима трассировки
  . . . . . . . . . . . . . .
  AND    WORD PTR [BP], 0FEFFh    ; Сброс флага TF
```

Команда XOR служит для инвертирования указанных полей операнда-приемника. Инвертируемые поля указываются значениями битов операнда-источника (маски). Маска должна содержать единицы в разрядах, соответствующих инвертируемым полям, и нули во всех остальных. После выполнения этой команды инвертируемые поля изменяют свое значение на противоположное, а все остальные не изменяются.

Кроме того, команда XOR используется, если необходимо определить, в каких битах значения операндов не совпадают. После ее выполнения в разрядах, в которых биты операндов не совпадают, устанавливаются единицы, а во всех остальных – нули.

Команда XOR часто применяется для очистки регистров. В этом случае в качестве приемника и источника используется один и тот же регистр.

Команда XOR используется и для правильного формирования флага четности PF при необходимости характеристики слова или более длинных элементов данных. Это достигается путем выполнения этой команды по отношению к младшему и старшему байтам контролируемого слова. При этом из слова попарно удаляются единицы, совпадающие в разрядах составляющих его байтов, что не влияет на признак четности, но сводит слово к байту, который правильно характеризуется флагом PF.

***Пример 2.81.***

```
XOR     AX, 8000h        ; Инвертирование разряда D15 регистра AX
XOR     AX, DX           ; Выявление несовпадающих значений
                         ; в битах регистров
XOR     DX, [BX + SI]    ; или регистра и ячейки памяти
XOR     MemByte, AL
XOR     AX, AX           ; Очистка регистра AX
XOR     AL, AH           ; Формирование флага PF для слова из AX
```

Команды AND, OR и XOR правильно формируют флаги SF, ZF, PF и сбрасывают флаги OF = CF = 0.

Команда отрицания NOT записывается в формате:

NOT приемник

и осуществляет инвертирование каждого бита операнда–приемника. В качестве приемника могут использоваться 8-битные или 16-битные регистры или ячейки памяти. Команда NOT не изменяет значения флагов.

***Пример 2.82.***

```
NOT     AX                   ; Инвертирование регистра
NOT     BYTE PTR [BX + 2]    ; или ячейки памяти
```

## Команды сравнения

К этой подгруппе относятся команды CMP и TEST. Они записываются в формате:

```
CMP     приемник, источник
TEST    приемник, источник
```

и осуществляют соответственно арифметическое и логическое сравнение операндов с фиксацией результатов сравнения только на флагах. Значения операндов при выполнении этих команд не изменяются.

Операндами команд CMP и TEST могут быть байты или слова. В качестве приемника и источника данных используются регистры МП или регистр и ячейка памяти. Кроме того, источник может быть и непосредственным значением.

Команда арифметического сравнения CMP выполняет сравнение операндов путем их вычитания подобно команде SUB, но результат вычитания не сохраняется, а лишь фиксируются его признаки на флагах. Таким образом, эта команда реализует операцию

(F) = (приемник) – (источник).

По значению флагов можно определить соотношения между операндами. После команды сравнения всегда находится команда условного перехода, анализирующая состояние флагов и передающая управление при истинности условия перехода. На практике следующая за CMP команда условного перехода выбирается по табл. 2.4 в зависимости от требуемого условия перехода (соотношения между сравниваемыми операндами).

***Пример 2.83.***

```
CMP     AX, DX          ; Сравнение регистра с регистром
CMP     [BX], DX        ; или с ячейкой памяти
CMP     AL, 53          ; или с числом
```

Команда TEST осуществляет логическое сравнение операндов путем выполнения над ними операции поразрядной конъюнкции подобно команде AND. При этом результат операции не сохраняется, а лишь фиксируются его признаки на флагах, которые формируются идентично команде AND. Таким образом, эта команда реализует операцию

(F) = (приемник) • (источник).

Как правило, команда TEST используется для проверки значения заданного бита. В этом случае за ней следует команда условного перехода JNZ.

***Пример 2.84.***

```
        IN      AL, Port        ; Переход по метке M1,
        TEST    AL, 80h         ; если бит D7 порта
        JNZ     M1              ; ввода Port установлен в 1
        . . . . . . . . . . . . .
M1:     . . . . . . . . . . . . .
```

## Команды сдвигов

Команды этой подгруппы предназначены для сдвига байтов или слов на указанное количество разрядов влево или вправо. Все команды сдвигов записываются в формате:

ROTATE приемник, счетчик,

где ROTATE – формальная мнемоника команды сдвига;
счетчик – формальный параметр, определяющий количество сдвигов операнда-приемника.

Параметр "счетчик" может принимать два фактических значения: 1 или CL. В первом случае сдвиг осуществляется на один разряд и называется статическим сдвигом. Во втором случае количество сдвигов операнда определяется содержимым регистра CL, рассматриваемым как беззнаковое целое число. Такой сдвиг называется динамическим.

В качестве приемника могут использоваться 8-битные или 16-битные регистр МП или ячейка памяти.

По сути выполняемых действий все команды сдвигов можно разделить на три подгруппы:

1) команды логических сдвигов;
2) команды арифметических сдвигов;
3) команды циклических сдвигов.

## Команды логических сдвигов

К этой подгруппе относятся команды SHR и SHL, предназначенные для сдвига нечисловых значений (например, масок), а также для сдвига чисел без знака.

Команда SHR сдвигает операнд-приемник вправо, а команда SHL – влево в соответствии со схемами, приведенными на рис. 2.10,а. Эти команды помещают в освобождающиеся после сдвига биты нулевые значения.

***Пример 2.85.***

```
SHR     AX, 1                  ; Сдвиг регистра на 1 бит вправо
SHL     BYTE PTR [BX], CL      ; Сдвиг байта в памяти на (CL)
                               ; разрядов влево
```

Команды логического сдвига также используются для сдвига операндов при преобразовании форматов десятичных чисел.

***Пример 2.86.***

*Преобразовать десятичные ASCII-числа из регистров AL, BL в BCD-формат.*

```
MOV     CL, 4                  ; Загрузка счетчика сдвигов
SHL     AL, CL                 ; Сдвиг старшей цифры
OR      AL, BL                 ; Формирование BCD-числа
```

Поскольку сдвиг операнда на один бит влево эквивалентен умножению его на 2, а сдвиг на один бит вправо – его делению на 2, то команды сдвига можно использовать для быстрого умножения и деления чисел. При этом умножение возможно на любое число и сводится к некоторой комбинации операций сдвигов влево и сложения, а деление возможно лишь на степень числа 2 (2, 4, 8, 16, . . .) и сводится к выполнению соответствующего количества сдвигов вправо.

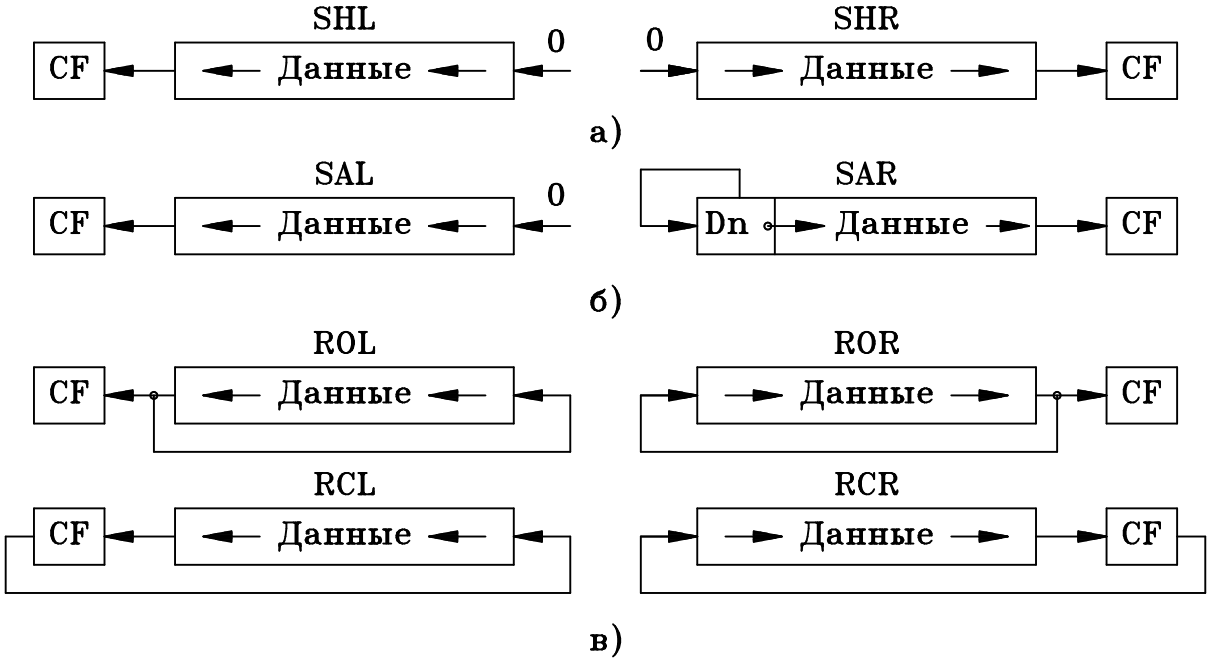


Рис. 2.10. Операции сдвига:
а) логические сдвиги; б) арифметические сдвиги; в) циклические сдвиги

***Пример 2.87.***

*Умножить число без знака из регистра AX на 53.*

Для определения требуемой комбинации операций сдвига и сложения в общем случае необходимо константу–множитель представить в виде двоичной схемы Горнера. Для числа 53 это представление имеет вид:

$$53 = 00110101b = ((((((0 \cdot 2 + 0)2 + 1)2 + 1)2 + 0)2 + 1)2 + 0)2 + 1$$

Реализация этой схемы начинается с внутренних скобок.

```
MOV     DX, AX          ; Копирование для последующих
                        ; сложений
SHL     AX, 1           ; Умножение на 2
ADD     AX, DX          ; Умножение на 3
SHL     AX, 1           ; Умножение на 6
SHL     AX, 1           ; Умножение на 12
ADD     AX, DX          ; Умножение на 13
```

```
SHL     AX, 1          ; Умножение на 26
SHL     AX, 1          ; Умножение на 52
ADD     AX, DX         ; Умножение на 53
```

Хотя в этой последовательности 8 команд, но выполняется она в 7 раз быстрее, чем одна команда MUL!

Команды логического сдвига правильно формируют флаги SF, ZF и PF. Флаг переноса CF формируется в соответствии со схемой сдвига (см. рис. 2.10,а). При сдвиге беззнаковых чисел влево флаг CF = 1 показывает, что результат вышел за допустимый диапазон, а при сдвиге вправо флаг CF содержит остаток от деления на 2.

Флаг переполнения OF имеет достоверное значение лишь при однократном сдвиге. При этом OF = 0, если два старших бита операнда равны, и OF = 1 – в противном случае.

**Команды арифметических сдвигов**

К этой подгруппе относятся команды SAL и SAR, предназначенные для сдвига знаковых чисел. Арифметический сдвиг отличается от логического сдвига тем, что знаковый разряд не сдвигается и сохраняет свое значение.

Команда SAL сдвигает операнд-приемник влево, а команда SAR – вправо в соответствии со схемами, приведенными на рис. 2.10,б. При этом при сдвиге вправо знаковый (старший) бит операнда дублируется в соседнем правом разряде, сохраняя свое значение, т.е. знак числа. Арифметический сдвиг влево совпадает с логическим сдвигом влево. Это обусловлено тем, что в дополнительном коде все старшие разряды числа, не хранящие его модуль, хранят его знак. Поэтому команды SAL и SHL идентичны, и обе мнемоники сохранены лишь для точного отражения содержательного смысла выполняемой операции.

Команды SAL и SAR используются для быстрого умножения и деления знаковых чисел на константу аналогично командам SHL и SHR (см. пример 2.87). Однако необходимо помнить, что результат деления отрицательного числа с помощью команды SAR не совпадает с результатом его деления с помощью команды IDIV. Например, результат операции (–7)/2 (–7=11111001b), полученный командой SAR, равен (–4), а командой IDIV – (–3). Это объясняется тем, что команда IDIV усекает и положительное и отрицательное частное к нулю путем отбрасывания дробной части, а команда SAR усекает положительные числа к нулю, а отрицательные – к минус бесконечности.

Команды SAL и SAR формируют флаги идентично командам SHL и SHR. Однако правилу формирования флага OF можно дать другую трактовку. Флаг переполнения OF = 1, если в результате сдвига изменился знак числа (при умножении на 2 результат вышел из диапазона, т.е. возникло переполнение). При сдвиге вправо это условие никогда не выполняется, и всегда OF = 0.

***Пример 2.88.***

```
MOV     DL, –25     ; Сдвиг числа (–25) = 11100111b влево:
SAL     DL, 1       ; (DL) = –50 = 11001110b, OF = 0, CF = 1,
                    ; SF = 1, ZF = 0, PF = 0
MOV     DL, –25     ; Сдвиг числа (–25) = 11100111b вправо на 2 бита:
MOV     CL, 2       ; (DL) = –7 = 11111001b, OF = ?, CF = 1, SF = 1,
SAR     DL, CL      ; ZF = 0, PF = 1
```

**Команды циклических сдвигов**

К этой подгруппе относятся команды ROL, ROR и RCL, RCR. Они осуществляют сдвиг в соответствии со схемами, приведенными на рис. 2.10,в. Особенностью этих команд является сохранение выдвинутых за пределы операнда битов путем включения их обратно в операнд. Очевидно, что команды RCL и RCR включают флаг переноса CF в кольцо сдвига, а команды ROL и ROR – нет.

Команды циклического сдвига изменяют только флаги CF и OF. При этом CF формируется в соответствии со схемами, а OF – идентично командам логического сдвига.

***Пример 2.89.***

*Пусть перед выполнением каждой команды (AL) = 11011010b и CF = 0. Тогда после выполнения команд циклического сдвига получим:*

```
ROL     AL, 1           ; (AL) = 10110101, CF = 1
ROR     AL, 1           ; (AL) = 01101101, CF = 0
RCL     AL, 1           ; (AL) = 10110100, CF = 1
RCR     AL, 1           ; (AL) = 01101101, CF = 0
```

Команды циклического сдвига позволяют изменять порядок следования битов в операнде.

***Пример 2.90.***

*Выполнить зеркальную перестановку битов в регистре* AL.

Если расположение битов в регистре AL до операции имеет вид $A_7A_6A_5A_4A_3A_2A_1A_0$, то после нее оно должно быть $A_0A_1A_2A_3A_4A_5A_6A_7$.

Подобные задачи решаются путем выделения очередного бита операнда на флаге CF за счет сдвига в одну сторону с последующим включением его в накопитель результата сдвигом в другую сторону.

Фрагмент программы для решения этой задачи имеет вид:

```
RCL     AL, 1           ; Выделение бита операнда на CF
RCR     DL, 1           ; Включение бита операнда
                        ; в накопитель результата
```

После 8-кратного повторения этого фрагмента в регистре DL будет получен требуемый результат.

### 2.13.4. Команды передачи управления

Команды передачи управления предназначены для изменения естественного порядка выполнения команд. Их можно разделить на следующие подгруппы:

1) команды безусловных переходов;
2) команды условных переходов;
3) команды управления циклами;
4) команды для организации подпрограмм;
5) команды прерываний.

Все команды передачи управления не изменяют состояния флагов.

**Команды безусловных переходов**

К этой подгруппе относится единственная команда JMP, которая передает управление по указанному адресу независимо от каких-либо условий. При этом переход может совершаться по прямому или косвенному адресу.

Команда прямого перехода записывается в формате JMP Labl и передает управление команде, помеченной меткой Labl. В зависимости от положения метки Labl относительно команды JMP могут совершаться короткий, ближний и дальний переходы.

Формат JMP Labl соответствует ближнему переходу (тип NEAR), когда команда JMP и метка Labl находятся в одном сегменте программного кода. В этом случае машинный код команды JMP содержит два байта для хранения дистанции disp до метки. При выполнении команды вычисляется адрес перехода (IP) = (IP) + disp(Labl) и управление передается в пределах всего сегмента.

Если метка находится в диапазоне от (–128) до (+127) байтов от команды JMP, то для хранения дистанции может использоваться один байт. Для этого команда должна записываться в формате JMP *SHORT* Labl, что соответствует выполнению короткого перехода. Короткий переход является разновидностью ближнего перехода и выполняется за то же время. При коротком и ближнем переходах модифицируется только указатель команд IP.

В случае, если команда JMP и метка Labl находятся в различных сегментах программного кода, то переход является дальним (тип FAR). Для выполнения дальнего перехода команда в общем случае записывается в виде JMP *FAR PTR* Labl. В машинном коде этой команды находится полный логический адрес точки назначения в формате segment:offset, который и загружается в регистры CS:IP при ее выполнении.

Дальний переход может быть выполнен и командой JMP Labl, если где-то выше метка описана директивой *LABEL* как дальняя.

***Пример 2.91.***

```
; Прямые переходы
JMP     Next              ; Ближний  переход на метку Next
JMP     SHORT Cont        ; Короткий переход на метку Cont
JMP     FAR PTR Start     ; Дальний переход на метку Start
JMP     Begin             ; Дальний переход на метку Begin,
                          ; если Begin LABEL FAR
```

Команда косвенного перехода записывается в формате:

JMP  источник

и передает управление по адресу, являющемуся содержимым операнда-источника. В качестве источника могут использоваться 16-битные РОН или ячейка памяти, адресуемая любым способом. Если адрес перехода задается регистром или словом в памяти, то команда JMP выполняет ближний косвенный переход. Если адрес перехода задается двойным словом в памяти, то выполняется дальний косвенный переход. При этом содержимое младшего слова указанного двойного слова загружается в регистр IP, а старшего – в регистр CS.

***Пример 2.92.***

```
; Косвенные переходы
JMP     AX                ; Ближний косвенный переход
                          ; по адресу (AX)
JMP     Alfa              ; Ближний косвенный переход по
                          ; ячейке памяти (Alfa DW ?)
JMP     DWORD PTR [BX]    ; Дальний косвенный переход
```

**Команды условных переходов**

Команды условных переходов передают управление в зависимости от значения некоторого условия. Они записываются в формате:

Jcond  Labl,

где cond – формальное условие перехода, и передают управление по метке Labl в случае истинности заданного условия. В противном случае выполняется очередная команда программы.

Все условные переходы являются короткими (тип SHORT). Это обусловлено тем, что в машинном коде команд условных переходов лишь один байт выделен для хранения дистанции disp8 до метки, который и используется для вычисления адреса перехода (IP) = (IP) + disp8(Labl). Отсюда любая команда условного перехода передает управление в пределах от (–128) до (+127) байтов.

Большинство команд условных переходов имеют две мнемоники. Выбор той или иной мнемоники позволяет подчеркнуть смысл проверяемого условия перехода и сделать программу более понятной.

Команды условных переходов могут передавать управление в зависимости от различных условий. Все они могут быть разделены на следующие подгруппы:

1) команды перехода по состоянию арифметических флагов;
2) команды перехода по соотношению между числами;
3) команды перехода по состоянию регистра CX.

### Команды перехода по состоянию арифметических флагов

Команды этой подгруппы используют в качестве условия перехода состояние арифметических флагов. При этом проверяемое условие перехода cond может принимать следующие фактические значения:

| | |
|---|---|
| Z/E – нуль/равно (ZF = 1), | NZ/NE – не нуль/не равно (ZF = 0), |
| C – наличие переноса (CF = 1), | NC – отсутствие переноса (CF = 0), |
| S – отрицательный результат (SF = 1), | NS – положительный результат (SF = 0), |
| P/PE – четный паритет (PF = 1), | NP/PO – нечетный паритет (PF = 0), |
| O – наличие переполнения (OF = 1), | NO – отсутствие переполнения (OF = 0). |

Здесь через дробную черту приведены альтернативные названия одного и того же условия.

Таким образом, команды условного перехода по состоянию арифметических флагов в зависимости от проверяемого условия могут записываться в следующем виде: JZ/JE, JNZ/JNE; JC, JNC; JS, JNS; JP/JPE, JNP/JPO; JO, JNO.

Эти команды могут использоваться после любой команды, формирующей соответствующий флаг.

***Пример 2.93.***

```
DEC     CL
JNZ     Begin           ; Переход, если (CL) не нуль
. . . . . . . . . . .
ADD     AX, Word        ; Сложение знаковых чисел
JO      Overflow        ; Переход, если переполнение
```

### Команды перехода по соотношению между числами

Команды этой подгруппы используются только после команды сравнения CMP. В результате сравнения формируются флаги, позволяющие проверить все соотношения между числами. Однако рассматриваемые команды реализуют операторы отношений <, >, <=, >=.

Для анализа результатов сравнения беззнаковых и знаковых чисел служат различные команды условного перехода. Это обусловлено тем, что использование дополнительного кода для представления знаковых чисел вносит сущест-

венное различие в интерпретацию результатов сравнения чисел без знака и со знаком. Например, код FFh = 11111111b как число без знака имеет значение 255 > 0, а как число со знаком – (–1) < 0.

Для устранения этой путаницы при сравнении беззнаковых чисел используются термины "ниже" (Below) – "выше" (Above), а при сравнении чисел со знаком – "меньше" (Less) – "больше" (Greater), выбор которых поясняется рис. 2.11.

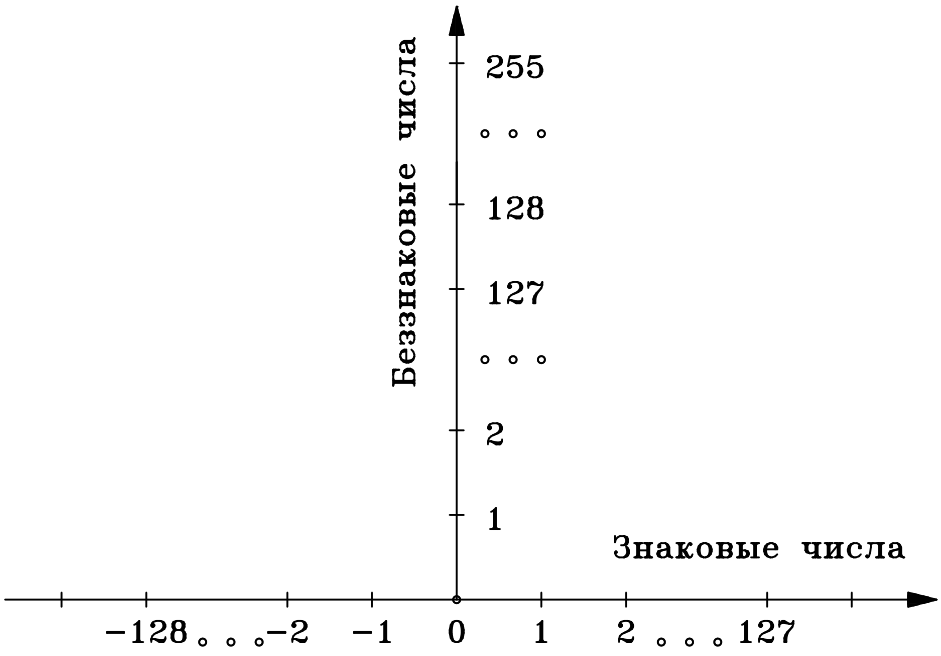


Рис. 2.11. Числовые оси для представления чисел

В соответствии с этим формальное условие cond команд условных переходов по соотношению между беззнаковыми числами может принимать следующие фактические значения:

B/NAE – ниже/не выше и не равно, NB/AE – не ниже/выше или равно, BE/NA – ниже или равно/не выше, NBE/A – не ниже и не равно /выше.

Отсюда команды условных переходов по соотношению между беззнаковыми числами имеют вид: JB/JNAE, JNB/JAE, JBE/JNA, JNBE/JA.

Аналогично условие cond команд условных переходов по соотношению между знаковыми числами может принимать значения:

L/NGE – меньше/не больше и не равно, NL/GE – не меньше/больше или равно, LE/NG – меньше или равно/не больше, NLE/G – не меньше и не равно//больше.

Отсюда команды условных переходов по соотношению между знаковыми числами имеют вид: JL/JNGE, JNL/JGE, JLE/JNG, JNLE/JG.

На практике для выбора команды условного перехода, следующей за командой сравнения (CMP приемник, источник), используется табл. 2.4.

Таблица 2.4

Выбор команд условных переходов в сочетании с командой CMP

| Условие перехода | Следующая за CMP команда | |
|---|---|---|
| | для чисел без знака | для чисел со знаком |
| Приёмник > источник | JA/JNBE | JG/JNLE |
| Приёмник = источник | JE | JE |
| Приёмник < > источник | JNE | JNE |
| Приёмник < источник | JB/JNAE | JL/JNGE |
| Приёмник < = источник | JBE/JNA | JLE/JNG |
| Приёмник > = источник | JAE/JNB | JGE/JNL |

Для анализа операндов на равенство, т. е. для реализации операторов отношений = и < >, необходимо использовать команды JE и JNE из первой подгруппы. В этом случае нет различий в сравнении знаковых и беззнаковых чисел.

***Пример 2.94.***

*Реализовать ветвление программы в зависимости от соотношения между знаковым числом из регистра* AL *и числом* 100.

```
        CMP     AL, 100     ; Сравнение (AL) с числом 100
        JGE     GE100       ; Переход, если (AL) > = 100
        . . . . . . . . . . ; Команды для выполнения при (AL)  < 100
GE100:  JG      G100        ; Переход, если (AL) > 100
        . . . . . . . . . . ; Команды для выполнения при (AL) = 100
G100    . . . . . . . . . . ; Команды для выполнения при (AL) > 100
```

**Команды перехода по состоянию регистра CX**

К этой подгруппе относится единственная команда JCXZ, которая передает управление по указанной метке, если (CX) = 0.

Эта команда используется для организации циклов в тех случаях, когда возможна ситуация, в которой тело цикла не выполняется ни разу. При этом команда JCXZ помещается перед телом цикла и передает управление за его пределы.

***Пример 2.95.***

```
        MOV     CX, Count       ; Чтение счетчика циклов
        JCXZ    EndLoop         ; Переход, если счетчик = 0
        . . . . . . . . . . . . ; Тело цикла
        . . . . . . . . . . . .
EndLoop: . . . . . . . . . . . ; Продолжение программы
                                ; при (Count) = 0
```

Очень часто в программах необходимо совершать условные переходы за пределы короткого диапазона (полные внутрисегментные переходы, межсегментные переходы). Такие условные переходы реализуются командами JMP, выполняемыми при истинности противоположного условия.

***Пример 2.96.***

*Выполнить межсегментный условный переход на метку* Zero *при флаге нуля* ZF = 1*:*

```
        JNZ     Cont            ; Анализ противоположного условия ZF = 0
        JMP     FAR PTR Zero    ; Межсегментный переход при ZF = 1
Cont:   . . .   . . . . . .     ; Продолжение программы при ZF = 0
```

**Команды управления циклами**

Команды управления циклами используются для условной передачи управления при организации циклов. Они записываются в формате:

LOOPcond Labl,

где cond – дополнительное формальное условие перехода.

При этом основным проверяемым условием для всех команд этой подгруппы является содержимое счетчика циклов, в качестве которого служит регистр CX. Каждая команда управления циклами прежде всего декрементирует регистр CX, а затем анализирует его. Если (CX) < > 0, то управление передается на начало цикла по указанной метке Labl. Если (CX) = 0, то выполняется очередная команда программы.

Все переходы, реализуемые командами управления циклами, являются короткими. Это также обусловлено тем, что их машинный код содержит лишь один байт для хранения дистанции disp8, который используется при вычислении адреса перехода (IP) = (IP) + disp8(Labl).

Дополнительное условие перехода cond либо отсутствует в команде, либо сводится к анализу флага ZF и принимает значения Z/E или NZ/NE.

Основной командой этой подгруппы является команда LOOP, в которой дополнительное условие перехода отсутствует. Она записывается в формате:

LOOP Labl.

Команда LOOP повторяет цикл, пока (CX) < > 0 и завершает его при (CX) = 0. Она используется для организации циклов с постусловием. При этом фрагмент программы имеет вид:

```
        MOV     CX, Count       ; Загрузка счетчика циклов
Begin:  . . . . . . . . . . .   ; Тело цикла
        . . . . . . . . . . .
        LOOP    Begin           ; Зацикливание, если (CX) < > 0
        . . . . . . . . . . .   ; Продолжение программы
```

***Пример 2.97.***

*Выполнить зеркальную перестановку битов в регистре AL (см. пример 2.90).*

```
        MOV     CX, 8           ; Загрузка счетчика циклов
Next:   RCL     AL, 1           ; Выделение бита
        RCR     DL, 1           ; Формирование результата
        LOOP    Next
        MOV     AL, DL          ; Пересылка результата
```

Во многих приложениях требуются такие циклы, которые должны завершаться не только после заданного количества повторений, но и при выполнении какого-либо другого условия. В качестве дополнительного условия прекращения цикла чаще всего используется значение флага ZF.

Для организации таких циклов предназначены команды LOOPZ/LOOPE и LOOPNZ/LOOPNE. Как правило, они используются после команды сравнения CMP.

Команда LOOPZ/LOOPE повторяет цикл до тех пор, пока (CX) < > 0 и ZF = 1. Следовательно, цикл завершается, если либо (CX) = 0, либо ZF = 0. Обычно эта команда применяется для организации циклов с целью поиска первого несовпадающего элемента в некотором массиве.

Команда LOOPNZ/LOOPNE повторяет цикл до тех пор, пока (CX) < > 0 и ZF = 0. Следовательно, цикл завершается, если либо (CX) = 0, либо ZF = 1. Обычно эта команда применяется для организации циклов с целью поиска первого совпадающего элемента в некотором массиве.

Таким образом, команды LOOPZ/LOOPE и LOOPNZ/LOOPNE допускают два варианта завершения цикла. Вариант завершения легко определить путем анализа состояния флага ZF после выхода из цикла. Например, если после завершения цикла командой LOOPNZ/LOOPNE оказывается, что ZF = 1, то цикл завершен по совпадению элементов, а при ZF = 0 – по счетчику цикла, т. к. совпадений не было.

***Пример 2.98.***

*Найти первый нулевой элемент в массиве байтов Array:*

```
        LEA     BX, Array –1            ; Загрузка адреса и
        MOV     CX, LENGTH Array        ; счетчика циклов
Next:   INC     BX                      ; Модификация адреса
        CMP     BYTE PTR [BX], 0        ; Сравнение с нулем
        LOOPNZ  Next
        JNZ     NotFound                ; Переход, если нуль не найден
        . . . .  . . . . . . .          ; Продолжение при наличии
        . . . .  . . . . . . .          ; нулевого элемента
NotFound: . . . .  . . . . . . .        ; Продолжение при отсутствии
        . . . .  . . . . . . .          ; нулевого элемента
```

В данном примере команда модификации адреса INC расположена перед командой CMP, что вызывает его инкрементирование перед первым сравнением. Поэтому для получения достоверного адреса первого элемента начальный адрес массива Array загружен в регистр BX на единицу меньшим значением. Команда INC не может находиться за командой CMP, т. к. она изменяет флаг ZF, искажая результат сравнения, полученный CMP и требующийся для LOOPNZ.

### Команды для организации подпрограмм

Подпрограмма (процедура) представляет собой совокупность команд, которая, располагаясь в памяти однократно, может быть исполнена сколь угодно раз в любом месте программы. Для использования подпрограмм необходимы команды для их вызова из текущей точки программы и команды для возврата из них в точку вызова. Очевидно, что для обеспечения корректного возврата необходимо сохранять адрес текущей точки программы. Этот адрес называется адресом возврата, и он автоматически сохраняется в стеке при вызове процедуры. При возврате из процедуры адрес возврата извлекается из стека и загружается в регистры-указатели адреса команды, что обеспечивает возврат в точку вызова. Все эти функции выполняются двумя командами: CALL и RET.

Команда CALL обеспечивает вызов подпрограмм. При этом вызов может осуществляться по прямому и косвенному адресу.

Команда прямого вызова записывается в формате:

CALL Name

и передает управление процедуре с именем Name. В качестве имени Name может использоваться любая метка или имя процедуры, введенное ее описанием с помощью директив *PROC* и *ENDP*. В зависимости от положения имени Name относительно команды CALL может совершаться ближний или дальний вызов процедуры.

Формат CALL Name соответствует ближнему вызову (тип NEAR), когда команда CALL и команда, помеченная именем Name, находятся в одном сегменте программного кода. В этом случае машинный код команды CALL содержит два байта для хранения дистанции disp16 до имени Name. При выполнении такой команды в качестве адреса возврата в стеке сохраняется лишь содержимое регистра IP, а затем вычисляется и записывается в него начальный адрес (IP) + disp16(Name), что и соответствует ближнему вызову указанной процедуры.

В случае, если команда CALL и имя Name находятся в различных сегментах программного кода, то вызов является дальним (тип FAR). Для выполнения дальнего вызова команда в общем случае записывается в виде CALL *FAR PTR* Name. В машинном коде этой команды находится полный логический адрес точки назначения в формате segment:offset. При выполнении такой команды CALL в качестве адреса возврата в стеке сохраняется содержимое регистров

CS:IP, а затем в них записывается адрес Name, что и соответствует дальнему вызову указанной процедуры.

***Пример 2.99.***

```
; Прямой вызов подпрограммы
CALL    Delay                ; Ближний прямой вызов подпрограммы Delay
CALL    FAR PTR Time         ; Дальний прямой вызов подпрограммы Time
```

Команда косвенного вызова подпрограмм записывается в формате:

CALL источник

и вызывает процедуру по адресу, являющемуся содержимым операнда-источника. В качестве источника могут использоваться 16-битный РОН или ячейка памяти, адресуемая любым способом. Если адрес вызова задается регистром или словом в памяти, то команда CALL выполняет ближний косвенный вызов. Если адрес перехода задается двойным словом в памяти, то выполняется дальний косвенный вызов. При этом содержимое младшего слова указанного двойного слова загружается в регистр IP, а старшего – в регистр CS.

***Пример 2.100.***

```
; Косвенные вызовы подпрограмм
CALL    BX                   ; Ближний косвенный вызов
                             ; по адресу (BX)
CALL    WORD PTR [BX]        ; Ближний вызов по ячейке
                             ; памяти ((BX))
CALL    Beta                 ; Дальний вызов по ячейке
                             ; памяти (Beta DD ?)
```

Команда RET обеспечивает возврат из подпрограммы в точку вызова. Эта команда должна завершать любую процедуру. При этом она может находиться и не в конце процедуры, но исполняться будет последней.

Команда RET не имеет операндов. Она извлекает из стека адрес возврата. Если процедура имеет тип *NEAR*, то RET извлекает из стека одно слово и загружает его в указатель команд IP, что и обеспечивает внутрисегментный возврат. Если процедура имеет тип *FAR*, то команда RET извлекает из стека два слова и помещает их в регистры CS, IP, что и обеспечивает межсегментный возврат.

В команде RET допускается указывать непосредственное 16-битное значение, интерпретируемое как целое без знака, то есть она может записываться в виде:

RET число.

Эта команда выполняется идентично обычной RET, но после восстановления адреса возврата указанное число прибавляется к указателю стека SP, т. е. (SP) = (SP) + число. Это используется для удаления входных параметров из стека при стековом способе передачи параметров в подпрограмму.

**Команды прерываний**

Для организации подпрограмм существует еще одна подгруппа команд, реализующая несколько иные действия по сравнению с командами CALL и RET. Команды этой подгруппы выполняют действия, практически идентичные действиям по обслуживанию аппаратных прерываний, что и обусловило их название – команды прерываний.

Основной командой этой подгруппы является команда INT, которая записывается в формате:

INT тип_прерывания,

где операнд "тип_прерывания" представляет собой некоторый номер из диапазона 0 – 255, идентифицирующий конкретное прерывание.

В отличие от команды CALL команда INT осуществляет только дальний косвенный вызов процедуры по номеру прерывания. При этом косвенный адрес вызова считывается из специальной таблицы, называемой таблицей векторов прерываний.

Таблица векторов прерываний включает в себя 256 элементов, каждый из которых представляет собой двойное слово. Элемент таблицы выбирается по номеру прерывания и содержит логический адрес начала подпрограммы в формате segment: offset. Этот адрес называется вектором прерывания.

При выполнении команды INT реализуются следующие действия:

1) записывается в стек содержимое регистра флагов F;

2) обнуляются флаги трассировки TF и разрешения прерываний IF;

3) записывается в стек адрес возврата CS:IP;

4) вычисляется адрес вектора прерывания путем умножения типа прерывания на 4;

5) загружается вектор прерывания в регистры CS,IP.

Таким образом, после выполнения команды INT в стеке окажутся значения регистров F, CS, IP, флаги TF и IF будут равны нулю, а регистры CS:IP будут указывать на начало подпрограммы, что и соответствует ее вызову.

При выполнении команды INT флаг трассировки TF автоматически сбрасывается с целью исключения пошагового режима выполнения вызванной подпрограммы, а флаг разрешения прерывания IF – с целью запрещения несанкционированных вложенных прерываний.

Для вызова прерываний с типами 3 и 4 могут использоваться альтернативные команды INT и INTO, которые записываются без операндов.

Однобайтная команда INT вызывает прерывание типа 3, которое определено как прерывание по контрольной точке. Контрольной точкой может быть

любое место в программе, где нормальное ее выполнение прерывается и производятся некоторые специальные действия. Контрольные точки обычно вводятся при отладке с целью отображения состояния системы в критических местах программы. Эта команда также используется для наложения "заплат" на программу в процессе отладки, что позволяет исправлять ее без повторной трансляции.

Однобайтная команда INTO вызывает прерывание типа 4, которое определено как прерывание по переполнению. В отличие от всех остальных эта команда является командой условного прерывания, которое инициируется при OF = 1. Если OF = 0, то выполняется очередная команда программы. Команда INTO используется для контроля за переполнением и передачи управления в случае его возникновения соответствующей подпрограмме, которая анализирует сложившуюся ситуацию и принимает все необходимые решения. Команда INTO размещается за командами, при выполнении которых возможно возникновение переполнения (например, ADD, SUB, SAL, и т.п.).

Для возврата в основную программу из процедур, вызванных сигналами запроса или командами прерываний, служит команда IRET. Она отличается от команды RET лишь тем, что кроме адреса возврата CS:IP восстанавливает из стека еще и содержимое регистра флагов F. Это обеспечивает продолжение выполнения основной программы с точки прерывания с восстановлением состояния системы прерывания (флага IF) и пошагового режима (флага TF).

### 2.13.5. Команды обработки строк

Строка представляет собой последовательность элементов, расположенных в смежных ячейках памяти. Элементами строки могут быть байты или слова, а ее длина ограничивается лишь размерами логического сегмента (до 64 Кбайт).

Команды обработки строк выполняют пять основных операций: пересылка, сравнение, сканирование, загрузка и сохранение элементов строк. Обработка строки всегда осуществляется последовательно элемент за элементом. Поэтому команды обработки строк часто называют строковыми примитивами. Однако с помощью одного примитива может быть обработана вся строка путем его многократного повторения.

Все строковые примитивы в общем случае могут быть записаны в следующих форматах:

```
COPS   строка_приемник, строка_источник;
COPSB;
COPSW;
```

где COPS – формальная мнемоника строкового примитива.

Отсюда следует, что строковый примитив не имеет операндов. Операнды, приведенные в первом формате, имеют фиктивный характер, т. к. они исполь-

зуются не для адресации данных, а лишь для определения типа обрабатываемых элементов. В этом случае тип элементов определяется предшествующим описанием операндов. Если операнды описаны как последовательность байтов (директивой DB), то строковый примитив выполняет операцию над байтами, а если как последовательность слов (директивой DW), то – над словами.

Во втором и третьем форматах тип обрабатываемых элементов указывается явно в мнемонике строкового примитива: B – байты, W – слова.

При трансляции программы ассемблер заменяет первый формат строкового примитива вторым или третьим форматами в зависимости от размера операндов. Однако первый формат записи является основным и рекомендуется к применению в большинстве случаев. Это обусловлено тем, что его использование позволяет ассемблеру проверить возможность адресации операндов при трансляции программы и способствует выявлению ошибок на данном этапе.

Для адресации данных строковые примитивы могут использовать единственный способ: косвенную адресацию по регистрам SI, DI. При этом адрес строки-источника всегда находится в регистре SI, а строки-приемника – в регистре DI. По умолчанию предполагается, что строка-источник находится в основном сегменте данных, и адрес ее элемента берется из регистров DS:SI, а строка-приемник находится в дополнительном сегменте данных, и адрес ее элемента берется из регистров ES:DI. Однако, сегмент, соответствующий регистру SI, может быть заменен на любой другой с помощью префикса замены сегмента. Для этого строковый примитив должен записываться в первом формате. Сегмент, соответствующий регистру DI, заменить нельзя ни при каких условиях. Отсюда следует, что строка-источник может находиться в любом сегменте, а строка-приемник – лишь в дополнительном сегменте данных.

Обработка строки-приемника в основном сегменте данных обеспечивается совмещением сегментов путем загрузки (ES) = (DS). При этом формально будет осуществляться обращение к строке-приемнику в дополнительном сегменте, а фактически – в основном сегменте данных.

Для ускорения обработки строк при выполнении любого строкового примитива автоматически аппаратным путем модифицируются регистры указатели адреса SI и DI, что подготавливает обращение к следующему элементу строки без всяких затрат времени. При модификации адресов содержимое регистров SI и DI увеличивается или уменьшается в зависимости от состояния флага направления DF. При DF = 0 строка обрабатывается от первого элемента к последнему, и адреса увеличиваются. При DF = 1 строка обрабатывается от последнего элемента к первому, и адреса уменьшаются. При этом при обработке байтов содержимое регистров SI, DI изменяется на 1, а при обработке слов – на 2.

### Строковые примитивы

К строковым примитивам относятся команды MOVS, CMPS, SCAS, LODS и STOS.

Строковый примитив пересылки MOVS записывается в форматах:

```
MOVS   строка_приемник, строка_источник;
MOVSB ;
MOVSW;
```

и обеспечивает пересылку элемента строки-источника (содержимого ячейки памяти по адресу (DS:SI)) в строку-приемник (ячейку памяти по адресу (ES:DI)).

Таким образом, команда MOVS в отличие от команды MOV реализует обращение типа "Память-Память".

***Пример 2.101.***

*В основном сегменте данных переслать* 100 *элементов из строки* Source *в строку* Dest:

```
        PUSH    DS              ; Совмещение основного и допол-
        POP     ES              ; нительного сегментов данных
        LEA     SI, Source      ; Загрузка адресов источника
        LEA     DI, Dest        ; и приемника данных
        MOV     CX, 100         ; Загрузка счетчика циклов
Next:   MOVS    Dest, Source    ; Пересылка элементов
        LOOP    Next
```

Строковый примитив MOVS не изменяет никаких флагов.

Строковый примитив сравнения CMPS записывается в форматах:

```
CMPS    строка_приемник, строка_источник;
CMPSB ;
CMPSW;
```

и обеспечивает арифметическое сравнение элемента строки-источника (содержимого ячейки памяти по адресу (DS : SI)) с элементом строки-приемника (содержимым ячейки памяти по адресу (ES : DI)). Сравнение осуществляется путем вычитания операндов, но результат вычитания не сохраняется, а лишь фиксируются его признаки на флагах. При этом реализуется операция

(F) = (строка_источник) – (строка_приемник)

Команда CMPS вычитает из операнда–источника операнд-приемник в отличие от команды CMP, вычитающей из операнда-приемника операнд-источник. В связи с этим команды условных переходов, следующие за командой CMPS, должны отличаться от команд, которые следовали бы в аналогичной ситуации за командой CMP. На практике для выбора команды условного перехода, следующей за примитивом CMPS, используется табл. 2.5.

Таблица 2.5

Выбор команд условных переходов в сочетании с командой CMPS

| Условие перехода | Следующая за CMPS команда | |
|---|---|---|
| | для чисел без знака | для чисел со знаком |
| Приемник > источник | JB/JNAE | JL |
| Приемник = источник | JE | JE |
| Приемник < > источник | JNE | JNE |
| Приемник < источник | JA/JNBE | JG/JNLE |
| Приемник <= источник | JAE/JNB | JGE |
| Приемник > = источник | JBE/JNA | JLE |

***Пример 2.102.***

*Сравнить строки слов со знаком* Str1 *и* Str2. *В случае, если хоть один элемент строки* Str1 *больше соответствующего элемента строки* Str2, *установить флаг в ячейке памяти* Great1.

```
        CLD                     ;Флаг  DF = 0 для обработки вперед
        MOV     Great1, 0       ; Обнуление флаговой ячейки
        LEA     SI, Str1        ; Загрузка адреса источника
        LEA     DI, Str2        ; и адреса приемника
        MOV     CX, LENGTH Str1 ; Загрузка счетчика циклов
Next:   CMPSW                   ; Сравнение элементов строк
        JG      Exit            ; Переход, если
                                ; приемник < источник
        LOOP    Next            ; Зацикливание,
                                ; если не все элементы
        . . . .  . . . . . . .  ; Продолжение
                                ; при отсутствии заданного
        . . . .  . . . . . . .  ; соотношения элементов
Exit:   MOV     Great1, 0FFh    ; Установка флаговой ячейки
```

Строковый примитив сканирования SCAS записывается в форматах:

```
SCAS    строка_приемник;
SCASB ;
SCASW;
```

и обеспечивает сканирование строки из дополнительного сегмента данных (содержимого ячеек памяти по адресу (ES : DI)) с целью поиска заданного значения. Искомые значения помещаются в регистры-аккумуляторы AL или AX в зависимости от типа элементов строки. Операция сканирования представляет собой арифметическое сравнение текущего элемента строки с содержимым ак-

кумулятора. Сравнение реализуется путем вычитания из содержимого регистра-аккумулятора текущего элемента строки. При этом результат вычитания не сохраняется, а лишь фиксируются его признаки на флагах аналогично команде сравнения CMP, т. е. реализуется операция

(F) = (ac) – (строка_приемник)

Если считать содержимое аккумулятора при выполнении SCAS операндом-приемником команды CMP, а текущий элемент строки – ее операндом-источником, то эта аналогия будет полной. Тогда, в общем случае, команда условного перехода, следующая за примитивом SCAS, может выбираться по табл. 2.4.

***Пример 2.103.***

*В строке* Str *длиной* 80 *символов найти первый символ, отличный от пробела.*

```
        CLD                     ; Флаг DF = 0 для обработки вперед
        LEA     DI, Str         ; Загрузка адреса строки
        MOV     CX, 80          ; и счетчика циклов
        MOV     AL, ' '         ; Загрузка символа для поиска
Next:   SCASB                   ; Сравнение
        LOOPE   Next
        JE      NotFound        ; Переход, если в строке все пробелы
        . . . . . . . . . . .   ; Продолжение программы при
        . . . . . . . . . . .   ; наличии символа (по адресу (ES:DI – 1))
```

Строковый примитив загрузки LODS записывается в форматах:

```
LODS    строка_источник;
LODSB;
LODSW;
```

и загружает текущий элемент строки из основного сегмента данных (содержимое ячейки памяти по адресу (DS:SI)) в регистр-аккумулятор AL или AX в зависимости от размера элемента.

***Пример 2.104.***

*Сравнить строки* Dest *и* Source. *Элемент строки* Source *в первой паре несовпадающих элементов загрузить в регистр* AL.

```
        CLD                     ; Флаг DF = 0 для обработки вперед
        LEA     DI, Dest        ; Загрузка адресов и
        LEA     SI, Source      ; счетчика циклов
        MOV     CX, LENGTH Source
Next:   CMPSB
        JNE     Exit            ; Переход при несовпадении
                                ; элементов
```

```
        LOOP    Next
        . . . . . . . . . . .          ; Продолжение программы
                                       ; при отсутствии
        . . . . . . . . . . .          ; несовпадающих элементов
Exit:   DEC     SI                     ; Коррекция автоинкремента SI
        LODSB                          ; Загрузка элемента строки
```

Строковый примитив сохранения STOS записывается в форматах:

```
STOS    строка_приемник;
STOSB;
STOSW
```

и обеспечивает запись байта из регистра AL или слова из регистра AX в текущий элемент строки дополнительного сегмента данных (ячейку памяти по адресу (ES : DI)).

Команда STOS удобна для заполнения строки заданным значением.

***Пример 2.105.***

*Обнулить массив слов* WordArray.

```
        CLD                            ; Флаг DF = 0 для обработки вперед
        LEA     DI, WordArray          ; Загрузка адреса массива,
        MOV     CX, LENGTH WordArray   ; счетчика цикла
        MOV     AX, 0                  ; и заданного значения
Next:   STOSW                          ; Обнуление массива
        LOOP    Next
```

Примитивы LODS и STOS применяются совместно для пересылки строк с их промежуточной обработкой.

***Пример 2.106.***

*В дополнительном сегменте данных переслать последовательность байтов из строки* Source *в строку* Dest *с промежуточным изменением их знака.*

```
        LEA     DI, Dest               ; Загрузка адресов
        LEA     SI, ES: Source
        MOV     CX, LENGTH Source      ; и счетчика циклов
Next:   LODS    ES: Source             ; Считывание элемента
        NEG     AL                     ; Изменение знака
        STOSB                          ; Сохранение элемента
        LOOP    Next
```

Особенностью рассмотренных примеров 2.101–2.106 является программная организация циклов для обработки строк, что приводит к большим затратам

времени на решение этих задач. Однако, при обработке строк цикл может быть организован не только программным, но и аппаратным путем, что существенно сокращает временные затраты на его реализацию. Для аппаратной организации циклов используются префиксы повторения.

### Префиксы повторения

Префикс повторения предшествует строковому примитиву и заставляет микропроцессор повторять его заданное количество раз. При этом количество повторений берется из регистра CX.

При наличии префикса обработка строки осуществляется в следующем порядке:

1)анализируется содержимое регистра CX;

2)если (CX) = 0, то обработка строки прекращается и управление передается очередной команде;

3)если (CX) < > 0, то его значение декрементируется с последующим выполнением строкового примитива и переходом к этапу 1.

Основным префиксом повторения является префикс REP, действие которого точно соответствует вышеописанному. Как правило, он используется с примитивами MOVS и STOS.

***Пример 2.107.***

*Скопировать строку* Source *в строку* Dest *в основном сегменте данных.*

```
        PUSH    DS                      ; Совмещение основного
        POP     ES                      ; и дополнительного
                                        ; сегментов данных
        LEA     SI, Source              ; Загрузка адресов
        LEA     DI, Dest
        MOV     CX, LENGTH Source       ; и счетчика циклов
REP     MOVS    Dest, Source            ; Копирование строки
```

В результате выполнения этого фрагмента программы все элементы строки Source скопируются в строку Dest.

***Пример 2.108.***

*Очистить зону памяти размером* 512 *байтов, начиная с физического адреса* B8000h.

```
        MOV     AX, 0B800h              ; Загрузка физического
        MOV     ES, AX                  ; адреса
        MOV     DI, 0
        MOV     CX, 512                 ; Загрузка счетчика циклов
        MOV     AL, 0                   ; Загрузка числа для очистки
REP     STOSB                           ; Очистка зоны памяти
```

В этом примере физический адрес B8000h представлен в виде логического адреса B800h : 0000, компоненты которого и загружены в соответствующие регистры.

Очевидно, что префикс REP функционально идентичен команде управления циклами LOOP, но отличается от нее тем, что сводит к минимуму время обработки строки.

Аналогично командам управления циклами префиксы повторения могут использовать значение флага ZF в качестве дополнительного условия для прекращения цикла.

Префикс REPZ/REPE повторяет строковый примитив до тех пор, пока (CX) < > 0 и ZF = 1. Следовательно, обработка строки прекращается, если либо (CX) = 0 либо ZF = 0, либо оба эти условия выполняются одновременно.

Префикс REPNZ/REPNE повторяет строковый примитив до тех пор, пока (CX) < > 0 и ZF = 0. Следовательно, обработка строки прекращается, если либо (CX) = 0, либо ZF = 1, либо оба эти условия выполняются одновременно.

Префиксы REPZ/REPE и REPNZ/REPNE, как правило, используются с примитивами CMPS и SCAS. При этом примитив CMPS в сочетании с префиксом REPZ/REPE соответствует поиску первой пары несовпадающих элементов, а с префиксом REPNZ/REPNE – поиску первой пары совпадающих элементов двух строк. Примитив SCAS в сочетании с префиксом REPZ/REPE используется для поиска первого элемента строки, отличающегося от заданного, а с префиксом REPNZ/REPNE – для поиска первого элемента, совпадающего с заданным.

Префиксы повторения REPZ/REPE и REPNZ/REPNE допускают два варианта завершения обработки строки. При необходимости вариант завершения может быть определен путем анализа флага ZF после выхода из цикла. Например, если после выхода из-под префикса REPNZ флаг ZF = 1, то выход осуществлен по совпадению элементов, а если после выхода ZF = 0, то – по окончанию строки, т. к. совпадения не было.

***Пример 2.109.***

*Сравнить две строки в памяти. При равенстве всех их элементов установить флаговую ячейку памяти.*

```
        LEA     SI, Source              ; Загрузка адресов строк
        LEA     DI, Dest
        MOV     CX, LENGTH Source       ; и счетчика циклов
        MOV     Flag, 0                 ; Очистка флаговой ячейки
REPE    CMPS    Dest, Source            ; Сравнение строк
        JNZ     Cont                    ; Переход, если несовпадение
        MOV     Flag, 0FFh              ; Установка флаговой ячейки
        . . . . . . . . . . . .         ;Продолжение при равенстве строк
Cont:   . . . . . . . . . . . .         ; Продолжение при неравенстве строк
```

Особым случаем является тот, когда последний элемент строки удовлетворяет условию выхода из-под префикса повторения. При этом и флаг ZF имеет активное значение, и регистр (CX) = 0. При необходимости эта ситуация раскрывается путем анализа не только флага ZF, но и состояния регистра CX.

***Пример 2.110.***

*Подсчитать количество символов* $ *в строке* Str *длиной* 80 *байтов.*

```
        MOV     DL, 0           ; Очистка накопителя
        CLD                     ; DF = 0 для обработки вперед
        LEA     DI, Str         ; Загрузка адреса строки
        MOV     CX, 80          ; и счетчика циклов
        MOV     AL, '$'         ; Загрузка искомого символа
Next:   REPNE   SCASB           ; Сканирование строки
        JNZ     Cont            ; Переход, если конец строки
        INC     DL              ; Инкремент накопителя при
                                ; наличии $ (по адресу (DI)–1)
        JCXZ    Exit            ; Переход, если конец строки
        JMP     SHORT Next      ; Продолжить обработку строки
Cont:   . . . . . . . . . . . . ; Продолжение при отсутствии $
Exit:   . . . . . . . . . . . . ; или наличии $ в последнем
                                ; элементе строки
```

Примитив LODS с префиксами не используется, т. к. загрузка последовательности элементов в один и тот же аккумулятор без какого-либо анализа не имеет смысла.

Обработка строки путем использования некоторого примитива с префиксом может быть прервана внешним запросом прерывания. Переход к обработке прерывания осуществляется после выполнения операции над текущим элементом. После возврата из прерывания обработка строки продолжается с очередного элемента. Однако, если для строкового примитива, кроме префикса повторения, заданы еще какие-либо префиксы (замены сегмента или блокировки), то продолжение обработки будет неправильным. Это обусловлено тем, что при переходе к обработке прерывания МП автоматически сохраняет лишь один префикс, непосредственно предшествующий команде, а все остальные теряются. Поэтому после возврата из прерывания любые дополнительные префиксы не действуют.

Если со строковым примитивом необходимо использовать несколько префиксов, то на время его выполнения прерывания должны быть запрещены. При этом следует учитывать, что при обработке длинных строк временной интервал с выключенной системой прерывания может стать недопустимо большим.

## 2.13.6. Команды управления микропроцессором

Команды этой группы позволяют управлять работой МП из программы. Они делятся на две подгруппы:

1) команды управления флагами;

2) команды синхронизации.

### Команды управления флагами

Все команды этой подгруппы не имеют операндов. Они позволяют изменять состояния трех флагов: флага переноса CF, флага направления DF и флага разрешения прерывания IF.

Команды CLC, CMC и STC выполняют соответственно сброс (CF = 0), инвертирование (CF = $\overline{CF}$), и установку (CF = 1) флага переноса. Они используются для установки нужного состояния флага CF перед циклическим исполнением команд сложения ADC и вычитания SBB, а также перед командами циклического сдвига.

Команды CLD и STD обеспечивают соответственно сброс (DF = 0) и установку (DF = 1) флага направления DF, определяющего направление обработки строк. В зависимости от состояния этого флага осуществляется автоинкрементирование или автодекрементирование регистров SI и DI при выполнении строковых примитивов.

Команды CLI и STI управляют состоянием флага разрешения прерывания IF, осуществляя соответственно его сброс (IF = 0) или установку (IF = 1).

При IF = 0 внешние маскируемые прерывания (по входу INTR) игнорируются МП, а немаскируемые прерывания (по входу NMI) не могут быть запрещены программным путем и обрабатываются обычным образом. После команды STI флаг IF устанавливается в 1, разрешая маскируемые прерывания. Однако ожидающее прерывание не распознается до завершения команды, находящейся за STI. При переходе к обслуживанию прерывания флаг IF автоматически сбрасывается, что предотвращает несанкционированные вложенные прерывания по входу INTR.

Команды управления флагом трассировки TF отсутствуют. Его значение изменяется через память с помощью логических команд AND или OR с соответствующей маской (см. пример 2.80).

### Команды синхронизации

Эти команды позволяют синхронизировать работу МП с внешними событиями. К ним относятся команды HLT, WAIT, ESC, LOCK и NOP.

Команда останова HLT переводит МП в состояние останова, выход из которого возможен либо по сигналу сброса RESET, либо по запросу прерывания INTR или NMI. Эта команда используется вместо бесконечного программного цикла для ожидания какого-либо внешнего события, о наступлении которого сообщает сигнал прерывания.

Команда ожидания WAIT проверяет уровень сигнала на входе $\overline{TEST}$ и переводит МП в состояние ожидания, если $\overline{TEST}$ = 1. В состоянии ожидания уровень сигнала на входе $\overline{TEST}$ проверяется через каждые 5 тактов синхронизации. При обнаружении значения $\overline{TEST}$ = 0 МП выходит из состояния ожидания и продолжает выполнение программы. Команда WAIT помещается в те точки программы, начиная с которых для ее продолжения необходимо наступление какого-либо внешнего события, о котором сообщает сигнал $\overline{TEST}$. Команда WAIT часто используется совместно с командой ESC для взаимодействия с арифметическим сопроцессором.

Команда расширения функций ESC записывается в формате:

ESC внешняя_операция, источник,

где внешняя_операция – это код внешней операции, которая должна выполняться не самим МП, а каким-либо другим устройством системы; источник – это адресуемый любым способом операнд, над которым выполняется заданная операция.

При выполнении команды ESC МП извлекает из памяти на шину данных указанные в ней операнды, но не обрабатывает их.

Извлеченные операнды предназначены для другого устройства многопроцессорной системы (как правило, сопроцессора) и считываются им с последующим выполнением указанной операции. Таким образом, команда ESC обеспечивает подключение внешних устройств системы для реализации расширенных функций.

Команда блокировки LOCK является префиксом и может предшествовать любой команде. При наличии префикса LOCK перед некоторой командой в течение всего времени ее выполнения на выходе $\overline{LOCK}$ МП устанавливается активный нулевой уровень. Этот сигнал должен использоваться для аппаратной блокировки, т. е. предотвращения доступа к системной шине другим устройствам многопроцессорной системы.

Команда холостой операции NOP не выполняет никаких действий. Однако она весьма полезна и может использоваться для различных целей: для организации временных задержек; для замены команд, удаляемых из программы в процессе отладки, что исключает необходимость ее повторной трансляции, в качестве носителя метки и т. д. и т.п.

## 2.13.7. Принципы кодирования команд

| Поле КОП | Поле адреса |
|---|---|

Рис. 2.12. Обобщенный формат команды одноадресной ЭВМ

В общем случае формат команд одноадресной ЭВМ имеет вид, приведенный на рис. 2.12. Поле кода операции КОП определяет функциональное назначение команды, а

поле адреса указывает адрес операнда или способ его вычисления.

Длина машинного кода команды в основном определяется используемым способом адресации. В МП ВМ86/ВМ88 длина команд может составлять от одного до шести байтов. В первых (одном или двух) байтах команды находится код операции и указывается способ адресации операндов. Последующие байты являются необязательными и содержат смещение disp или непосредственный операнд.

Команды МП ВМ86/ВМ88 могут адресовать один или два операнда и записываются на ассемблере в общем виде как COP oper или COP oper1, oper2, где COP – обобщенная мнемоника, а oper – операнд команды.

В двухместных командах один из операндов обязательно должен быть регистром. Следовательно, они допускают обращения типа регистр-регистр, регистр-память или память-регистр, а обращения типа память-память, т. е. одновременная адресация двух ячеек памяти в одной команде, недопустимы. Это определяется принятым способом кодирования команд. Наиболее общий формат машинного кода двухместной команды (без непосредственного операнда) приведен на рис. 2.13, где штриховыми линиями обозначены необязательные байты команды, содержащие смещение disp.

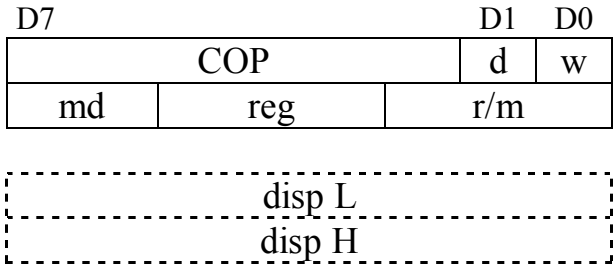


Рис. 2.13. Формат двухместной команды (без непосредственного операнда)

Первый байт команды содержит код операции COP и два однобитных поля: поле направления d и поле ширины операнда w. Операнды, участвующие в операции, определяются вторым байтом команды, называемом постбайтом режима адресации. Постбайт включает в себя три поля: двухбитное поле режима адресации md, трехбитное поле операнда-регистра reg и трехбитное поле операнда r/m, который может быть и регистром и ячейкой памяти.

Поле ширины операнда w определяет разрядность операндов, обрабатываемых командой. При w = 0 операнды являются байтами, а при w = 1 – словами.

Поле направления d определяет операнд-приемник результата. При d = 0 приемником результата является операнд, описываемый полем r/m, а при d = 1 – полем reg.

Поле режима адресации md указывает способ интерпретации поля r/m при адресации операнда. Значение md = 11 соответствует регистровой адресации, и поле r/m рассматривается как код регистра. Значение md < > 11 соответствует адресации памяти. При этом конкретное значение md определяет ширину смещения disp, используемого при вычислении адреса операнда:

$$md = \begin{cases} 00, disp = 0 & \text{– смещение отсутствует;} \\ 01, disp = dispL & \text{– 8-битное смещение, расширяемое по знаку до 16 битов;} \\ 10, disp = dispH, dispL & \text{– 16-битное смещение.} \end{cases}$$

При md < > 11 реализуется косвенная адресация памяти, и поле r/m определяет способ формирования эффективного адреса EA операнда. Кодирование способа адресации операнда, определяемого полем r/m, в общем случае описывается табл.2.6, в которой обозначено D8 = dispL – однобайтное смещение, а D16 = dispH, dispL – двухбайтное смещение в команде.

Таблица 2.6

Кодирование способов адресации операнда

| r/m \ md | 00 | 01 | 10 | 11 | |
|---|---|---|---|---|---|
| | | | | w = 0 | w = 1 |
| 000 | (BX) + (SI) | (BX)+(SI)+D8 | (BX)+(SI)+D16 | AL | AX |
| 001 | (BX) + (DI) | (BX)+(DI)+D8 | (BX)+(DI)+D16 | CL | CX |
| 010 | (BP) + (SI) | (BP)+(SI)+D8 | (BP)+(SI)+D16 | DL | DX |
| 011 | (BP) + (DI) | (BP)+(DI)+D8 | (BP)+(DI)+D16 | BL | BX |
| 100 | (SI) | (SI) + D8 | (SI) + D16 | AH | SP |
| 101 | (DI) | (DI) + D8 | (DI) + D16 | CH | BP |
| 110 | D16 | (BP) + D8 | (BP) + D16 | DH | SI |
| 111 | (BX) | (BX) + D8 | (BX) + D16 | BH | DI |

Из этой таблицы следует, что всего имеется 24 варианта вычисления эффективного адреса EA памяти и 16 вариантов адресации регистров.

Приведенные в табл. 2.6 правила кодирования имеют одно исключение.

При md = 0 и r/m = 110 смещение disp не равно 0, как следует из общего правила, а равно двухбайтному смещению D16. Это исключение позволяет реализовать прямую адресацию памяти, не предусмотренную явно форматом постбайта адресации. Смещение D16 в этом случае представляет собой абсолютный адрес операнда, т. е. смещение соответствующей ячейки памяти внутри сегмента.

Однако это исключение приводит к невозможности косвенной адресации по регистру BP (на уровне машинных кодов). Для реализации косвенной адресации по BP ассемблер кодирует ее как базовую адресацию с нулевым смещением, т. е. на уровне машинных кодов [BP] = [BP + 0].

Наиболее общий формат двухместной команды с непосредственным операндом приведен на рис. 2.14.

В этом случае необходимость адресации второго операнда отсутствует, и поле reg постбайта режима адресации используется для расширения кода операции. Кроме того, здесь не нужен бит d, т. к. результат можно загрузить только

на место одного операнда. Вместо бита d используется бит s, определяющий совместно с битом w размер непосредственного операнда data:

$$sw = \begin{cases} *0, data = dataL & \text{– 8-битный операнд;} \\ 01, data = dataH, dataL & \text{–16-битный операнд;} \\ 11, data = dataL & \text{– 8-битный операнд, расширяемый по знаку до 16 битов.} \end{cases}$$

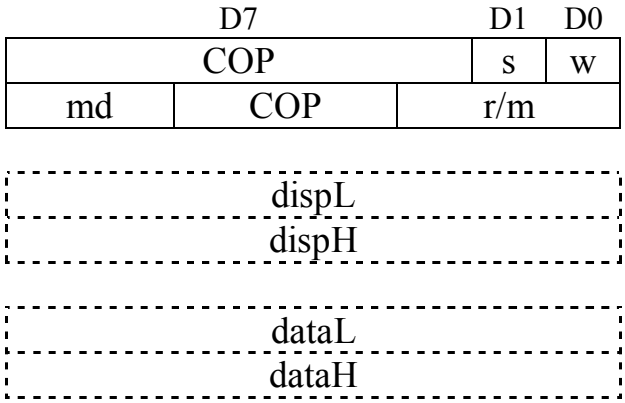


Рис. 2.14. Формат двухместной команды с непосредственным операндом

Например, поле sw в машинном коде нижеприведенных команд имеет следующие значения:

MOV AL,5 (sw = 00); MOV AX,5555h (sw = 01); MOV AX,5 (sw = 11).

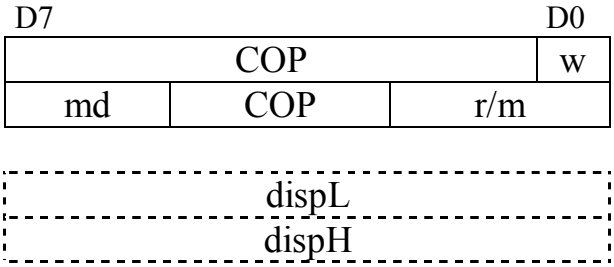


Рис. 2.15. Формат одноместной команды

Поля md, r/m, и disp интерпретируются также, как и в предыдущем формате.

Наиболее общий формат одноместной команды приведен на рис. 2.15. Все поля этого формата рассмотрены выше.

При вычислении физического адреса ячейки памяти компоненты ее логического адреса выбираются в соответствии с табл. 2.2. При обращении к данным сегментный регистр по умолчанию может быть заменен с помощью префикса замены сегмента на любой другой. На уровне машинных кодов префикс замены сегмента представляется предшествующим команде байтом с форматом, изображенным на рис. 2.16. При этом сегментные регистры sreg кодируются следующим образом:

ES – 00, CS – 01, SS – 10, DS – 11.

| D7 | | | D4 D3 | | | D0 |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | sreg | 1 | 1 | 0 |

Рис. 2.16. Формат префикса замены сегмента

| COP |
|---|

Рис. 2.17. Формат команды с неявной адресацией

| COP | w | reg |
|---|---|---|

Рис. 2.18. Специальный укороченный формат команды

Команды с неявной адресацией являются однобайтными и их формат приведен на рис. 2.17.

Кроме рассмотренных базовых форматов, используемых для кодирования команд, имеются специальные укороченные форматы, позволяющие на один байт сократить длину машинного кода команды. Укороченный формат используется для кодирования команд, оперирующих лишь регистрами, и имеет вид, приведенный на рис. 2.18. В нем поле reg из постбайта режима адресации перенесено в первый байт команды, а постбайт исключен. Если команда является двухместной, то для ее кодирования укороченным форматом один из регистров должен быть аккумулятором AL или AX, который описывается в поле кода операции COP. Отсюда следует, что использование аккумуляторов AL или AX в качестве операндов команд способствует оптимизации длины машинного кода программы.

При трансляции программы ассемблер автоматически выбирает более короткий формат команды. Более полная информация о кодировании команд приведена в [4].

Знание форматов команд необходимо при анализе дампов памяти в процессе отладки и ремонта программ МПС.

### 2.13.8. Время выполнения команд

В общем случае полное время выполнения команды включает в себя две составляющие:

1) базовое время выполнения команды;

2) время вычисления эффективного адреса EA (при обращении к памяти).

Время вычисления EA зависит от используемого способа адресации операнда и приведено в табл. 2.7.

Базовое время выполнения команды определяется типом команды и приводится в системе команд МП как ее характеристика [4]. Например, команда MOV reg,mem имеет базовое время Тбаз = 8 тактов. При этом предполагается, что выполняемая команда уже выбрана и находится в очереди команд.

Приведенные значения базового времени всегда соответствуют обработке байтов или слов по четному адресу. Если слово имеет нечетный адрес, то МП обращается к нему за два цикла шины. Следовательно, для МП ВМ86 при обращении к каждому слову по нечетному адресу, а для МП ВМ88 при обраще-

нии к каждому слову необходимо прибавить к приведенному значению базового времени 4 такта синхронизации на каждое обращение к памяти.

Таблица 2.7

Время вычисления эффективного адреса

| Способ адресации | Значение EA | Число тактов |
|---|---|---|
| Прямая | disp | 6 |
| Косвенная | (BX), (SI), (DI) | 5 |
| Базовая | (BX), (BP) + disp | 9 |
| Индексная | disp + (SI), (DI) | 9 |
| Базово-индексная | (BX), (BP) + (SI), (DI) | 8 |
| Относительная базово-индексная | (BP) + (DI) + disp<br>(BX) + (SI) + disp<br>(BP) + (SI) + disp<br>(BX) + (DI) + disp | 11<br>11<br>12<br>12 |

Наличие в команде префикса замены сегмента увеличивает базовое время ее выполнения на 2 такта.

Необходимо учитывать, что при выполнении команд типа ADD mem,reg обращение к памяти осуществляется дважды: за операндом и для записи результата, что увеличивает время выполнения еще на 4 такта.

***Пример 2.111.***

*Вычислить время выполнения команд*

```
MOV     ES:[BX], DX,
ADD     ES:[BX + SI], DX
```

$T_{MOV}$ = Тбаз.$_{MOV}$ + $T_{EA}$ + $T_{ES}$ = 8 + 5 + 2 = 15 тактов – для МП ВМ86 по четному адресу;

$T_{MOV}$ = Тбаз.$_{MOV}$ + $T_{EA}$ + $T_{ES}$ + $T_W$ = 8 + 5 + 2 + 4 = 19 тактов – для МП ВМ88 по любому адресу, для МП ВМ86 по нечетному адресу;

$T_{ADD}$ = Тбаз.$_{ADD}$ + $T_{EA}$ + $T_{ES}$ + $2T_W$ = 16 + 8 + 2 + 2∗4 = 34 такта – для МП ВМ88 по любому адресу, для МП ВМ86 – по нечетному адресу.

Первоначально время выполнения команды вычисляется в тактах сигнала синхронизации CLK МПС. Для его преобразования в секунды количество тактов необходимо умножить на длительность такта.

Знание времени выполнения команд необходимо при проектировании времязависимых программ.

## 2.14. Организация подпрограмм

В качестве подпрограмм оформляются либо повторяющиеся фрагменты программы, либо независимые программные исполнители. Как правило, перед использованием подпрограммы приводится описание, определяющее ее имя и тип.

### 2.14.1. Директивы описания подпрограмм

Для описания подпрограммы используются директивы ассемблера *PROC* и *ENDP*. Формат описания имеет вид:

```
Имя_подпрограммы    PROC    {Тип_подпрограммы}
                    . . . . . . . ; Тело подпрограммы
                    . . . . . . . ; (описание выполняемых действий)
                    RET           ; Возврат
Имя_подпрограммы    ENDP
```

Директива *PROC* открывает текст подпрограммы, присваивая ей имя и тип, а директива *ENDP* закрывает его. Формальный параметр "Тип_подпрограммы" может принимать фактические значения *NEAR* или *FAR*.

Вызов подпрограммы и возврат из нее осуществляется командами CALL и RET. Выбор типа подпрограммы определяется положением команды вызова CALL относительно описания подпрограммы.

Тип *NEAR* используется, если описание подпрограммы и команда ее вызова CALL находятся в одном и том же сегменте программного кода. В этом случае реализуется внутрисегментный вызов процедуры. По умолчанию (без указания типа) подпрограмма имеет тип *NEAR*.

Тип *FAR* используется, если описание подпрограммы и команда ее вызова CALL находятся в различных сегментах программного кода. В этом случае реализуется межсегментный вызов процедуры.

Команда RET обеспечивает соответствующий вызову возврат в основную программу и, как правило, завершает текст подпрограммы.

***Пример 2.112.***

*Написать подпрограмму для вычисления* S = A – B, *где* A *и* B – *слова в памяти.*

```
Code        SEGMENT
            . . . . . . . . .
      SubW  PROC      NEAR        ; Описание подпрограммы
            MOV       AX, A
            SUB       AX, B
            MOV       S, AX
            RET
      SubW  ENDP
```

```
                . . . . . . . . . .
                CALL        SubW        ; Внутрисегментный вызов
                                        ; подпрограммы
                . . . . . . . . . .
Code            ENDS
```

В зависимости от типа вызова и возврата команды CALL и RET реализуют различные действия. Поэтому при трансляции программы ассемблер формирует для них машинные коды, соответствующие типу подпрограммы, указанному в ее описании.

В случае если подпрограмма имеет тип *FAR*, то и внутрисегментный вызов ее реализуется как межсегментный.

***Пример 2.113.***

*Написать подпрограмму для вычисления* S = A – B *с возможностью вызова ее из различных сегментов программного кода.*

```
Code1           SEGMENT
                . . . . . . . . . .
        SubWF   PROC        FAR         ; Описание процедуры
                MOV         AX, A       ; для межсегментного
                SUB         AX, B       ; вызова
                MOV         S, AX
                RET
        SubWF   ENDP
                . . . . . . . . . .
                CALL FAR PTR SubWF      ; Внутрисегментный вызов
                . . . . . .
Code1           ENDS
Code2           SEGMENT
                . . . . . . . . . .
                CALL FAR PTR SubWF      ; Межсегментный вызов
                . . . . . . . . . .
Code2           ENDS
```

## 2.14.2. Передача параметров в подпрограммы

Любая подпрограмма выполняет свои действия по отношению к одним и тем же элементам архитектуры, поскольку они в ней жестко определены. Поэтому без принятия дополнительных мер подпрограмма всегда обрабатывает одни и те же данные. Например, при любом вызове процедуры SubW из примера 2.112 всегда будет вычисляться разность S = A – B.

Однако в прикладных программах часто требуется выполнять одни и те же действия над различными данными. Для обработки различных данных с по-

мощью одной подпрограммы необходимо каким-либо образом сообщить ей о размещении обрабатываемых данных и формируемых результатов. Это обеспечивается с помощью передачи параметров в подпрограмму. При этом элементы архитектуры, через которые передаются параметры, называются формальными параметрами.

Для обработки некоторых данных формальные параметры перед вызовом подпрограммы должны быть определены соответствующими фактическими значениями. При другом вызове этой же процедуры формальные параметры определяются другими фактическими значениями, что и обеспечивает обработку других данных.

Передача входных параметров всегда осуществляется перед вызовом подпрограммы, а выходных – после возврата из нее. Основная программа, из которой вызывается процедура, часто называется макроуровнем программы. Таким образом, передача параметров в подпрограмму всегда осуществляется на макроуровне, а их обработка – в самой подпрограмме.

Существуют различные способы передачи параметров в подпрограмму, накладывающие на нее свой отпечаток.

### Передача параметров через регистры

В этом случае входные параметры перед вызовом подпрограммы загружаются в РОНы МП, с которыми она работает. После вызова подпрограмма выполняет необходимые действия и оставляет результат тоже в регистрах. Пересылка результата в память осуществляется после возврата из подпрограммы.

Общая структура программы в этом случае будет иметь вид:

```
        Name      PROC
                  [          ]          ; Обработка данных
                  RET
        Name      ENDP
; Макроуровень программы
        Start:    . . . . . . .
                  [          ]          ; Передача входных параметров
                  CALL      Name        ; Обработка данных
                  [          ]          ; Передача выходных параметров
```

***Пример 2.114.***

*Вычислить* S = A + B, *где* A *и* B – *некоторые слова в памяти.*

Текст программы для решения этой задачи будет иметь вид:

```
AddWRP  PROC
        ADD     AX, DX          ; Сложение слов
        RET
AddWRP  ENDP
```

```
; Макроуровень программы
Start:      . . . . . . .
            MOV     AX, A1          ; Передача входных
            MOV     DX, B1          ; параметров
            CALL    AddWRP          ; Вычисление S1 = A1 + B1
            MOV     S1, AX          ; Передача выходного
            . . . . . . .           ; параметра
            MOV     AX, A2          ; Вычисление S2 = A2 + B2
            MOV     DX, B2
            CALL    AddWRP
            MOV     S2, AX
```

Достоинством этого способа является простейшая реализация подпрограмм, а недостатком – жесткое ограничение на количество передаваемых параметров.

**Передача параметров через общие ячейки памяти**

В этом случае параметры передаются через общие ячейки памяти, к которым обращаются и программа, и подпрограмма. В одних ячейках макроуровень программы располагает входные данные, а в других подпрограмма оставляет полученные результаты.

При этом общая структура программы будет иметь вид:

```
Name        PROC
            [          ]    ; Чтение входных параметров из общих ячеек
            [          ]    ; Обработка данных
            [          ]    ; Запись выходных параметров в общие ячейки
            RET
Name        ENDP

; Макроуровень программы
Start:      . . . . . . .
            [          ]    ; Передача входных параметров в общие ячейки
            CALL Name       ; Обработка данных
            [          ]    ; Передача выходных параметров из общих ячеек
```

Для реализации этого способа должны быть описаны общие ячейки памяти, которые являются формальными параметрами.

***Пример 2.115.***

*Вычислить* S = A + B, *где* A *и* B – *некоторые слова в памяти.*

Для передачи параметров в данном случае потребуется три слова в памяти, и текст программы для решения этой задачи будет иметь вид:

```
FormA       DW      ?           ; Описание общих
FormB       DW      ?           ; ячеек памяти
```

```
FormS     DW      ?
          . . . . . . .
          . . . . . . .
AddWMP    PROC
          MOV     AX, FormA          ; Чтение параметра
          ADD     AX, FormB          ; Сложение слов
          MOV     FormS, AX          ; Запись параметра
          RET
AddWMP    ENDP

 ; Макроуровень программы
Start:    . . . . . . .
          MOV     AX, A1             ; Передача входных
          MOV     FormA, AX          ; параметров
          MOV     AX, B1
          MOV     FormB, AX
          CALL    AddWMP             ; Вычисление S1 = A1 + B1
          MOV     AX, FormS          ; Передача выходного
          MOV     S1, AX             ; параметра
          . . . . . . .
          MOV     AX, A2             ; Вычисление S2 = A2 + B2
          MOV     FormA, AX
          MOV     AX, B2
          MOV     FormB, AX
          CALL    AddWMP
          MOV     AX, FormS
          MOV     S2, AX
```

Достоинством этого способа является неограниченное количество передаваемых параметров, а недостатком – необходимость описания общих ячеек и усложнение макроуровня программы.

**Передача параметров через зоны параметров**

Этот способ целесообразно использовать, если передаваемые параметры можно расположить в смежных ячейках памяти. Эта область памяти называется зоной параметров (рис. 2.19). Для передачи параметров в подпрограмму достаточно передать адрес Zone зоны параметров.

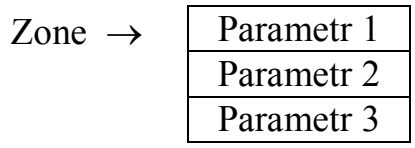


Рис. 2.19. Зона параметров

Общая структура программы в этом случае будет иметь вид:

```
Name        PROC
            [          ]   ; Чтение входных параметров из зоны параметров
            [          ]   ; Обработка данных
            [          ]   ; Запись выходных параметров в зону параметров
            RET
Name        ENDP

; Макроуровень программы
Start:      . . . . . . . .
            LEA     BX, Zone    ; Передача адреса зоны
            CALL    Name        ; Обработка данных
```

Для реализации этого способа также необходимо описывать зоны параметров, которые являются формальными областями:

***Пример 2.116.***

*Вычислить* S = A + B, *где* A *и* B – *некоторые слова в памяти*.

Для передачи параметров в данном случае потребуется область памяти, состоящая из трех слов. Разместим параметры в области так, чтобы слагаемое A занимало первое слово, B – второе слово, а сумма S – третье слово зоны.

Текст программы для решения этой задачи будет иметь вид:

```
Zone1     DW      ?, ?, ?          ; Описание зон
          . . . . . . .            ; параметров
Zone2     DW      ?, ?, ?
          . . . . . . .
AddWZP    PROC
          MOV     AX, [BX]         ; Чтение параметра
          ADD     AX, [BX + 2]     ; Сложение слов
          MOV     [BX + 4], AX     ; Запись параметра
          RET
AddWZP    ENDP

 ; Макроуровень программы
Start:    . . . . . . .
          LEA     BX, Zone1        ; Передача адреса зоны 1
          CALL    AddWZP           ; Вычисление S1 = A1 + B1
          . . . . . . .
          LEA     BX, Zone2        ; Вычисление S2 = A2 + B2
          CALL    AddWZP
```

В рассмотренном примере предполагается, что зоны параметров уже сформированы и содержат необходимые фактические значения.

Достоинством этого способа является простота макроуровня программы, а недостатком – необходимость описания и формирования зон параметров.

## Передача параметров через стек

Очень часто передача параметров осуществляется через стек. Это обусловлено тем, что стек всегда существует и не требует дополнительного описания.

В этом случае перед вызовом подпрограммы все входные параметры передаются в стек. После вызова подпрограммы они считываются из стека и обрабатываются. Результат также помещается в стек, из которого считывается на макроуровне программы.

Считывание и запись параметров в стек осуществляется через регистр BP, позволяющий обращаться к стеку, как к ОЗУ с произвольной выборкой. Поскольку адресация должна осуществляться относительно верхушки стека, то необходимо прежде всего выполнить привязку к ней. Для этого в регистр BP копируется содержимое указателя стека SP. Перед этим содержимое регистра BP сохраняется в стеке для его дальнейшего восстановления.

Общая структура программы в этом случае будет иметь вид:

```
Name    PROC
        PUSH    BP                      ; Сохранение BP
        MOV     BP, SP                  ; Привязка к верхушке стека

        MOV     REG, [BP + Смещение]    ; Чтение входных параметров из стека

                                        ; Обработка данных

        MOV     [BP + Смещение], REG    ; Запись выходных параметров в стек

        POP     BP                      ; Восстановление BP
        RET     DATA                    ; Возврат с исключением входных
                                        ; параметров из стека

        ; Макроуровень программы
Start:    . . . . . . . . . . . . . .
        SUB     SP, CONST               ; Резервирование стека под результат

        PUSH    Параметр                ; Передача входных параметров в стек

        CALL    Name                    ; Обработка данных

        POP     Параметр                ; Передача выходных параметров
                                        ; из стека
```

Величина CONST представляет собой число, определяющее размер резервируемой под результат зоны стека в байтах. Поскольку все операции со стеком выполняются словами, то значение CONST всегда должно быть четным.

Рассмотренной структуре программы соответствует схема размещения параметров в стеке, приведенная на рис. 2.20,а.

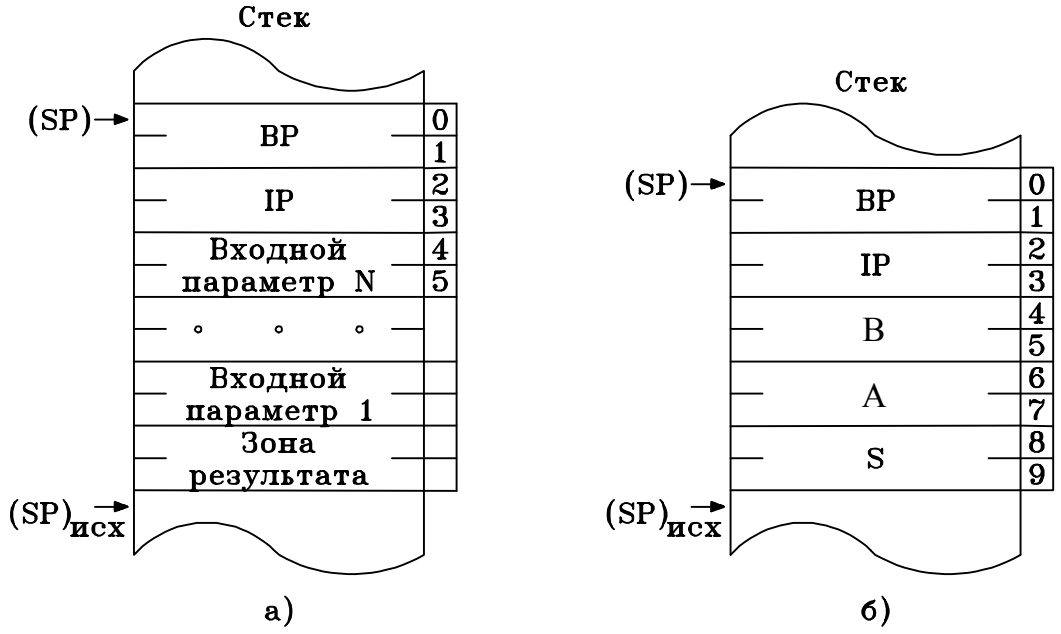


Рис. 2.20. Схема размещения параметров в стеке:
а) в общем случае; б) для примера 2.117

Для определения смещения, добавляемого к содержимому регистра BP при чтении входных и записи выходных параметров в стек, все байты используемой зоны стека должны быть пронумерованы.

После считывания входных параметров в подпрограмму они остаются в стеке. Однако после возврата из подпрограммы они должны быть удалены из стека, а указатель стека SP должен указывать на зону результата. Для исключения входных параметров из стека служит команда возврата RET DATA. Она отличается от обычной RET тем, что после восстановления адреса возврата прибавляет к регистру SP число DATA. Очевидно, что для исключения входных параметров из стека значение DATA должно быть равно количеству байтов, занимаемых в стеке входными параметрами. Поскольку все операции со стеком выполняются словами, то число DATA должно быть четным.

***Пример 2.117.***

*Вычислить* S = A + B*, где* A *и* B *– некоторые слова в памяти.*

Поскольку результат этой операции считаем словом, то в стеке под него необходимо зарезервировать два байта.

Текст программы для решения этой задачи будет иметь вид:

```
AddWSP  PROC
        PUSH    BP              ; Сохранение BP
        MOV     BP, SP          ; Привязка к верхушке стека
        MOV     AX, [BP + 6]    ; Чтение входного параметра
```

```
            ADD     AX, [BP + 4]        ; Сложение слов
            MOV     [BP + 8], AX        ; Запись выходного параметра
            POP     BP                  ; Восстановление BP
            RET     4                   ; Возврат с исключением
AddWSP      ENDP                        ; входных параметров

  ; Макроуровень программы
Start:      . . . . . . . . . . .
            SUB     SP, 2               ; Резервирование слова под результат
            PUSH    A1                  ; Передача входных
            PUSH    B1                  ; параметров
            CALL    AddWSP              ; Вычисление S1 = A1 + B1
            POP     S1                  ; Передача выходного параметра
            . . . . . . . . . . .
            SUB     SP, 2               ; Вычисление S2 = A2 + B2
            PUSH    A2
            PUSH    B2
            CALL    AddWSP
            POP     S2
```

Этой программе соответствует схема размещения параметров в стеке, приведенная на рис. 2.20,б. Нумерация ячеек стека позволяет определить все необходимые смещения.

При написании такой программы сначала пишется ее макроуровень и составляется схема размещения параметров в стеке. Далее ячейки стека нумеруются, начиная от верхушки, что определяет смещения для доступа к передаваемым параметрам. После этого пишется подпрограмма с использованием известных смещений.

В вышерассмотренных примерах 2.114 – 2.117 все параметры передавались в подпрограммы и из них своими значениями. Это приемлемо, когда количество передаваемых чисел невелико, и недопустимо при их большом количестве (например, при обработке массивов). Для обработки массивов они передаются в подпрограммы не значениями элементов, которых может быть очень много, а своими атрибутами, основными из которых являются адрес и длина.

***Пример 2.118.***

*Написать подпрограмму для вычисления арифметической суммы элементов некоторого массива байтов без знака. С помощью этой подпрограммы обработать массив* Array1.

Поскольку подпрограмма должна обеспечивать обработку произвольного массива, то для решения поставленной задачи атрибуты массива Array1 должны передаваться в нее в виде параметров.

В качестве примера используем передачу параметров через общие ячейки памяти. В данной задаче для передачи параметров потребуется три общих слова

памяти: первое – для передачи адреса массива, второе – для передачи его длины, третье – для передачи результата.

С учетом этого текст программы для решения поставленной задачи будет иметь вид:

```
Data        SEGMENT
    Array1  DB      100 DUP(?)      ; Описание входного массива
    SUM1    DW      ?               ; Описание ячейки результата
            . . . . . . . . . . .
    FormAddr DW     ?               ; Описание общих
    FormLen DW      ?               ; ячеек памяти
    FormSum DW      ?
Data        ENDS
            . . . . . . . . . . .
    SumArr  PROC    NEAR
            MOV     BX, FormAddr    ; Чтение входных
            MOV     CX, FormLen     ; параметров
            MOV     AX, 0           ; Очистка накопителя
    Next:   ADD     AL, [BX]        ; Прибавление очередного
                                    ; байта
            ADC     AH, 0           ; Учет переноса
            INC     BX              ; Модификация адреса
            LOOP    Next
            MOV     FormSum, AX     ; Передача результата
            RET
    SumArr  ENDP

    ; Макроуровень программы
    Start:  . . . . . . . . . . .
            LEA     AX, Array1          ; Передача адреса
            MOV     FormAddr, AX        ; массива Array1
            MOV     AX, LENGTH Array1   ; Передача длины
            MOV     FormLen, AX         ; массива Array1
            CALL    SumArr              ; Вычисление суммы
            MOV     AX, FormSum         ; Передача суммы
            MOV     Sum1, AX            ; массива Array1
```

С помощью этой подпрограммы аналогичным образом может быть вычислена сумма элементов и других массивов.

## 2.15. Организация системы прерываний

МП ВМ86/ВМ88 имеют простую, но эффективную систему прерываний. Каждому прерыванию поставлен в соответствие некоторый номер TYPE, иден-

тифицирующий его тип. Тип прерывания может принимать значения от 0 до 255, что обуславливает возможность обработки 256 различных прерываний. Все прерывания делятся на внешние и внутренние.

### 2.15.1. Внешние прерывания

Внешние прерывания инициируются внешними по отношению к МП устройствами по входам INTR или NMI.

Прерывания по входу INTR являются маскируемыми и могут быть программно запрещены путем сброса флага IF. Как правило, к входу INTR МП подключается контроллер прерывания BH59A, на входы которого подаются запросы прерывания от внешних устройств. Контроллер прерывания воспринимает эти запросы и генерирует сигнал INTR, поступающий в МП. Если прерывания разрешены (IF = 1), то МП переходит к обслуживанию этого прерывания. При этом прежде всего генерируются циклы подтверждения прерывания INTA, в результате которых из контроллера прерывания считывается код типа прерывания TYPE, соответствующий активному запросу с максимальным приоритетом. Этот код используется для определения начального адреса соответствующей подпрограммы обработки прерывания.

Прерывания по входу NMI являются немаскируемыми и не могут быть запрещены программным путем. Как правило, они используются для реакции на катастрофические события в системе (аварии в системе питания, ошибки в ОЗУ и т.п.). Прерывания по входу NMI имеют фиксированный тип TYPE = 2. Код этого типа автоматически формируется в МП. Поэтому по запросу NMI циклы подтверждения прерывания INTA не генерируются, и подпрограмма обработки прерывания сразу вызывается по начальному адресу, соответствующему типу 2.

### 2.15.2. Внутренние прерывания

Внутренние прерывания являются немаскируемыми и не могут быть запрещены программным путем. Они возникают либо по инициативе МП при наличии некоторых ситуаций, либо по инициативе программиста с помощью команд прерываний INT TYPE.

Первые пять типов прерываний (от 0 до 4) зарезервированы для системных целей и использовать их как-либо иначе не рекомендуется.

Прерывания с типами 0 и 1 вызываются по инициативе МП.

Тип 0 имеет прерывание по ошибке деления, которое инициируется автоматически при выполнении команд деления в случае деления на 0 или в случаях, когда формат частного превышает формат приемника результата.

Тип 1 имеет прерывание пошагового режима. Оно инициируется автоматически после выполнения каждой команды при установленном флаге трассировки (TF = 1).

Прерывания с типами 3 и 4 вызываются по инициативе программиста с помощью команд прерываний INT и INTO.

Прерывания с типами от 5 до 255 являются прерываниями пользователя и служат различным целям. Тип прерывания может указываться в командах прерывания INT TYPE (например, INT 10h, INT 21h, INT 150) и всегда используется для определения начального адреса подпрограммы его обработки.

### 2.15.3. Обслуживание прерываний

При одновременном возникновении нескольких запросов на прерывания они обслуживаются в порядке их приоритетов, представленных в табл. 2.8.

Таблица 2.8

Приоритет прерываний

| Вид прерывания | Тип прерывания | Приоритет |
|---|---|---|
| По ошибке деления | 0 | 1 |
| По команде INT | 3 | 1 |
| По команде INTO | 4 | 1 |
| По команде INT TYPE | 5 – 255 | 1 |
| По входу NMI | 2 | 2 |
| По входу INTR | 5 – 255 | 3 |
| По флагу TF | 1 | 4 |

В этой таблице уровень 1 является высшим, а 4 – низшим уровнем приоритета. Наличие нескольких источников прерываний высшего приоритета обусловлено тем, что все они являются следствием команд, которые не могут выполняться одновременно. Очевидно, что прерывание пошагового режима (по флагу TF) имеет низший приоритет, и, следовательно, в пошаговом режиме все остальные прерывания исполняются обычным образом.

Переходя к обслуживанию некоторого прерывания, МП прекращает выполнение текущей программы и переходит к выполнению подпрограммы обработки, соответствующей типу прерывания. Связь между типом прерывания и начальным адресом подпрограммы устанавливается таблицей векторов прерываний, приведенной на рис. 2.20.

Таблица векторов прерываний расположена в самом начале адресного пространства памяти и включает в себя 256 элементов. Каждый элемент таблицы содержит двойное слово, называемое вектором прерывания. Вектор прерывания представляет собой начальный логический адрес подпрограммы обработки прерывания в формате segment:offset. В каждом векторе слово с большим адресом содержит базовый адрес сегмента segment, а слово с меньшим адресом – смещение подпрограммы offset относительно начала этого сегмента.

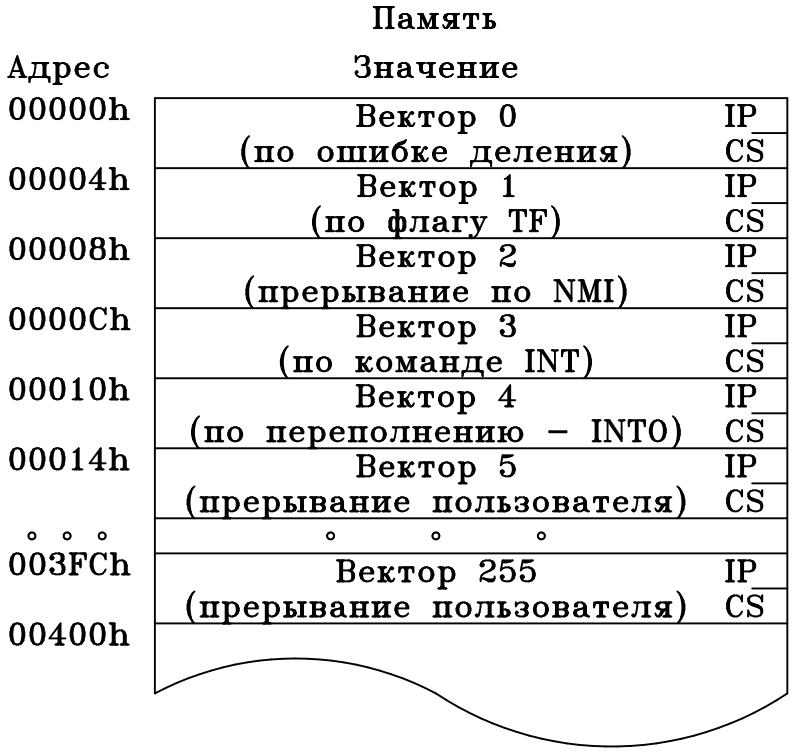


Рис. 2.20. Таблица векторов прерываний

При переходе к обслуживанию любого прерывания МП прежде всего сохраняет в стеке содержимое регистров F, CS, IP и сбрасывает флаги IF, TF. Тем самым исключаются пошаговый режим выполнения подпрограммы и возможность возникновения несанкционированных вложенных прерываний.

После этого формируется адрес вектора прерывания путем умножения его типа на 4, т. к. каждый элемент таблицы векторов составляет 4 байта. По этому адресу считывается вектор прерывания и загружается в регистры CS : IP, что и соответствует переходу к подпрограмме обработки прерывания. При обращении к таблице векторов прерываний сегментные регистры не используются.

Таким образом, подпрограмма обработки прерывания всегда имеет тип *FAR* и вызывается косвенно из любой точки адресного пространства.

Новое содержимое регистров CS : IP определяет начальный адрес подпрограммы, по которому выбирается ее первая команда и т. д. Любая подпрограмма обработки прерывания должна начинаться с сохранения в стеке изменяемых ею регистров МП. В конце подпрограммы эти регистры восстанавливаются из стека.

Подпрограмма обработки прерывания должна завершаться командой IRET, восстанавливающей из стека содержимое регистров IP, CS, F, что обеспечивает полное восстановление состояния МП и возврат к команде основной программы, которая выполнялась бы при отсутствии прерывания.

Для корректного вызова подпрограмм обработки прерываний все используемые вектора должны быть инициализированы на соответствующие адресные

значения. При инициализации векторов маскируемые прерывания должны быть запрещены.

***Пример 2.119.***

*Вычислить* S = A + B, *где* A *и* B *некоторые слова в памяти, по прерыванию с типом* 83.

Для общности введем поименованную константу IntType = 83. С учетом этого текст программы для решения поставленной задачи будет иметь вид:

```
IntType   EQU     83
          . . . . . . . . . . .
AddWInt   PROC    FAR                 ; Подпрограмма обработки
                                      ; прерывания
          PUSH    AX                  ; Сохранение используемых
                                      ; регистров
          MOV     AX, A               ; Вычисление S = A + B
          ADD     AX, B
          MOV     S, AX
          POP     AX                  ; Восстановление регистров
          IRET                        ; Возврат из подпрограммы
AddWInt   ENDP
          . . . . . . . . . . .
Start:    . . . . . . . . . . .
          ; Инициализация вектора прерывания
          CLI                         ; Запрет маскируемых прерываний
          PUSH    DS                  ; Сохранение регистра DS
          XOR     AX, AX              ; Подготовка обращения
          MOV     DS, AX              ; к таблице векторов прерываний
          MOV     BX, 4*IntType       ; Загрузка адреса вектора
          LEA     AX, AddWInt         ; Запись смещения offset
          MOV     [BX], AX            ; подпрограммы
          MOV     AX, SEG AddWInt     ; Запись адреса segment
          MOV     [BX + 2], AX        ; подпрограммы
          POP     DS                  ; Восстановление регистра DS
          STI                         ; Разрешение маскируемых
                                      ; прерываний
```

После выполнения этого фрагмента при инициализации прерывания с типом 83 (по входу INTR или командой INT 83) управление будет передано подпрограмме AddWInt, после выполнения которой осуществится возврат в основную программу.

В персональных компьютерах некоторые вектора прерываний инициализированы операционной системой (например, прерывания от клавиатуры, таймера и т.п.). При необходимости выполнения нетиповых действий по этим прерываниям интересующие прерывания перехватываются. Суть перехвата заклю-

чается в переопределении соответствующего вектора прерывания так, чтобы он указывал не на стандартную процедуру операционной системы, а на подпрограмму пользователя. Тогда при возникновении этого прерывания будет вызываться подпрограмма пользователя, выполняющая необходимые действия.

## 2.16. Нестандартные типы данных

Переменные стандартных типов (простые или элементы массивов) имеют единственное поле для хранения данных, состоящее из одного или нескольких байтов. Однако, при обработке сложной информации необходимо, чтобы с одним именем связывалось несколько полей различного типа. Это обеспечивается с помощью нестандартных типов данных, создаваемых пользователем, к которым относятся структуры и записи.

### 2.16.1. Структуры

Структура – это нестандартный тип данных, представляющий собой совокупность байтовых полей, объединенных одним именем.

Для использования этого типа данных необходимо:

1) объявить шаблон (тип) структуры;

2) описать переменные объявленного типа.

Для описания шаблона структуры используются директивы ассемблера *STRUC* и *ENDS*. Формат объявления шаблона имеет вид:

| | |
|---|---|
| Имя_структуры | *STRUC* |
| | Описание полей структуры |
| | директивами *DB,DW,DD* |
| Имя_структуры | *ENDS* |

Директива *STRUC* открывает шаблон структуры, объявляя его имя как нестандартный тип данных. Директива *ENDS* закрывает шаблон, определяя размер структуры. Описание полей структуры осуществляется директивами распределения памяти *DB*, *DW*, *DD* обычным образом. Следовательно, каждое поле может иметь уникальное имя, которому может быть присвоено некоторое начальное значение.

Структуры не могут быть вложенными, т. к. для описания их полей нельзя использовать директивы *STRUC* и *ENDS*.

***Пример 2.120.***

*Объявить шаблон структуры для описания даты некоторого события.*

Дата определяется тремя полями: день, месяц и год, – каждое из которых может принимать значение из известного диапазона. Шаблон такой структуры объявляется следующим образом:

```
DATE        STRUC
        Day     DB      12      ; День
        Month   DB      4       ; Месяц
        Year    DW      1961    ; Год
DATE        ENDS
```

В данном примере полям структуры присвоено предварительное начальное значение 12 апреля 1961 года.

Объявление типа структуры не вызывает распределения памяти и инициализации полей на указанные значения, а лишь определяет шаблон для дальнейшего использования.

Резервирование ячеек памяти с их инициализацией на начальные значения осуществляется путем описания переменных объявленного типа. Описание переменных структурированного типа выполняется в следующем формате:

{Имя_переменной} Имя_структуры <{Начальное_значение, . . .}>

По этому описанию в памяти создается переменная структурированного типа с распределением полей в соответствии с указанным шаблоном. Имя переменной связывается с началом структуры. Если имя отсутствует, то ассемблер распределяет память в соответствии с шаблоном, но эта область памяти остается безымянной.

В угловых скобках < > указывается список начальных значений полей структурированной переменной. Последовательность начальных значений должна строго соответствовать последовательности полей шаблона по типу и размеру. Каждое начальное значение может быть целым числом, строковой константой или константой-выражением. Для каждого поля может быть задано лишь одно начальное значение. Задавать начальные значения для всех полей необязательно. Однако отсутствующие элементы в начале списка указываются запятыми. При отсутствии начального значения в описании переменной ее соответствующее поле инициализируется на значение, заданное в шаблоне. Если оно и там не определено, то это поле остается неопределенным и в переменной. Угловые скобки < > обязательны и в том случае, когда не задано ни одного начального значения.

***Пример 2.121.***

*Описать структурированные переменные типа* DATE *из примера* 2.120.

```
Data        SEGMENT
        Dt1     DATE    <25 ,10 ,1991h>     ;
        Dt2     DATE    <, , 1996h>         ;
        Dt3     DATE    < >                 ;
Data        ENDS
```

Это описание вызывает распределение и инициализацию ячеек памяти в соответствии с рис. 2.21.

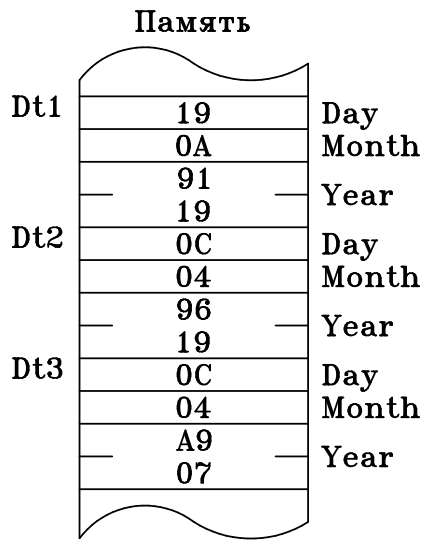


Рис. 2.21. Распределение и инициализация памяти под структурированные переменные

При описании переменных структурированного типа могут быть переопределены начальные значения полей, заданные шаблоном структуры. Однако это справедливо лишь для простых полей шаблона, в которых задано единственное значение. Составные поля не могут быть переопределены, что не позволяет использовать оператор *DUP* для их описания при необходимости переопределения.

Если поле шаблона содержит строку, то она может быть переопределена только строкой, длина которой не превышает длину строки в шаблоне. Если подавляющая строка короче подавляемой, то ассемблер дополняет ее пробелами справа.

Таким образом, для переопределения полей структуры путем описания переменных необходимо, чтобы в ее шаблоне каждое поле имело единственный элемент максимально возможного размера.

***Пример 2.122.***

```
        ; Описание переопределяемых и непереопределяемых полей
S1          STRUC
      Name  DB        'Строка_символов_максимального_размера'
      Year  DW        ?
S1          ENDS
S2          STRUC
      F1    DB        1, 2                ; Неподавляемое поле
      F2    DW        5                   ; Подавляемое поле
      F3    DB        10 DUP(?)           ; Неподавляемое поле
S2          ENDS
Data        SEGMENT
      M1    S1        < 'Иванов,Петров', 1978 >
      M2    S1        < 'Сидоров', 1992 >
      V1    S2        < ,10 >             ; Нет ошибки
                                          ; переопределения
      V2    S2        < 5 >               ; Ошибка переопределения
      V3    S2        < ,,'Строка'>       ; неподавляемого поля
Data        ENDS
```

При описании переменных структурированного типа допускается использование оператора дублирования *DUP*.

***Пример 2.123.***

DateArray DATE 100 *DUP* (< >)

Это описание вызовет 100-кратное дублирование шаблона DATE из примера 2.120 в памяти, что соответствует созданию массива DateArray структурированных элементов.

Атрибуты переменных структурированного типа определяются обычным образом. Например, для примеров 2.121, 2.123 имеем:

*SEG* Dt1 = *SEG* Dt2 = *SEG* Dt3 = Data;
*OFFSET* Dt1 = 0, *OFFSET* Dt2 = 4, *OFFSET* Dt3 = 8;
*TYPE* Dt1 = *TYPE* Dt2 = *TYPE* Dt3 = *TYPE* DateArray = 4;
*LENGTH* DateArray = 100; *SIZE* DateArray = 400.

Обращение к различным полям структурированной переменной осуществляется путем записи операнда в следующем формате:

Имя_структурированной_переменной . Имя_поля

С точки зрения ассемблера часть операнда ( . Имя_поля) представляет собой смещение в байтах относительно начала структурированной переменной. При вычислении эффективного адреса для обращения к полю структурированной переменной ассемблер суммирует все смещения.

***Пример 2.124.***

```
; Обращения к полям структурированных переменных
  MOV   AL, Dt1 . Day              ; Загрузка поля Day переменной Dt1
  MOV   Dt2 . Month, 12            ; Запись числа в поле Month
                                   ; переменной Dt2
  MOV   AX, DateArray[10] . Year   ; Загрузка AX из поля Year
                                   ; 10-го элемента массива
```

Обработка структурированных данных осуществляется обычным образом.

***Пример 2.125.***

*Сведения о студентах включают в себя имя, пол и год рождения. Вычислить количество девушек не старше* 20 *лет в группе.*

Очевидно, что для решения этой задачи необходимо использовать структуры, и текст программы будет иметь вид:

```
        ; Описание нестандартных типов данных
STUD            STRUC                           ; Информация о студенте
        Name    DB  'Строка длиной 25 символов'     ; Имя
        Sex     DB      'Ж'                     ; Пол (или 'М')
        Birthyear DW    ?                       ; Год рождения
STUD            ENDS

        ; Описание переменных
Data            SEGMENT
        Grup    STUD    25 DUP(< >)             ; Описание группы
        Sum     DB      ?                       ; Результат
        Year    DW      ?                       ; Текущий год
Data            ENDS
                . . . . . . . . . . .
        Start:  . . . . . . . . . . .
                . . . . . . . . . . .
        ; Программа заполнения массива Grup
        ; и переменной текущего года Year
        . . . . . . . . .
        ; Вычисление количества девушек
                MOV     DL, 0                   ; Очистка накопителя
                MOV     SI, 0                   ; и индекса
                MOV     CX, LENGTH Grup         ; Загрузка счетчика циклов
        Next:   CMP     Grup[SI].Sex, 'Ж'       ; Проверка пола
                JNE     Cont                    ; Переход, если юноша
                MOV     AX, Year                ; Вычисление возраста
                SUB     AX, Grup[SI].Birthyear
                CMP     AX, 20                  ; Анализ возраста
                JA      Cont                    ; Переход, если старше 20 лет
                INC     DL                      ; Формирование результата
        Cont:   ADD     SI, TYPE Grup           ; Модификация индекса
                LOOP    Next                    ; Зацикливание, если не все
                MOV     Sum,DL                  ; Запись результата
```

## 2.16.2. Записи

Запись – это нестандартный тип данных, представляющий собой совокупность битовых полей, объединенных одним именем.

Для использования этого типа данных необходимо:

1) объявить шаблон (тип) записи;

2) описать переменные объявленного типа.

Для описания шаблона записи используется директива ассемблера *RECORD*. Формат объявления шаблона имеет вид:

Имя_записи *RECORD* Спецификация_поля {,Спецификация_поля . . .},

где каждая спецификация поля записывается следующим образом:

Имя_поля : Длина_поля {= Начальное_значение}

Директива *RECORD* объявляет имя записи как нестандартный тип данных. Спецификация каждого поля определяет его имя, длину в битах и начальное значение, на которое оно может инициализироваться. Общая длина полей записи может составлять от 1 до 16 битов.

***Пример 2.126.***

*Объявить шаблон записи для описания информации о количественном составе классов школы города.*

Для представления этой информации необходимы 3 поля: номер класса, индекс класса и количество учащихся в нем. Для определения размера полей записи будем считать, что номер класса может принимать значения от 1 до 11, индекс класса – от "А" до "Ж"", а количество учеников в любом из них не превышает 40 человек.

С учетом этого размер поля Number для хранения номера класса должен быть 4 бита, поля Index для его индекса – 3 бита ( индекс имеет 7 значений: "А", "Б", "В", "Г", "Д", "Е", "Ж"), поля Amount для хранения количества учеников – 6 битов. Тогда объявление шаблона такой записи будет иметь вид:

CLASS *RECORD* Number : 4 = 6, Index : 3 = 3, Amount : 6.

В данном примере полям записи присвоено предварительное начальное значение номера и индекса класса – 6"Г". При этом необходимо учесть, что индексы классов представляются кодированными значениями: 0 – "А", 1 – "Б", 2 – "В", 3 – "Г", . . . , 6 – "Ж". Поле Amount оставлено без инициализации.

Индекс класса можно задать и в символьной форме (. . ., Index : 8 = 'Г',. . .). Однако, при этом размер поля должен быть 8 битов, т. к. информация об индексе хранится в ASCII-коде.

Объявление типа записи не вызывает распределения памяти и инициализации полей на указанные значения, а лишь определяет ее шаблон для дальнейшего использования.

Резервирование ячеек памяти с их инициализацией на начальные значения осуществляется путем описания переменных объявленного типа. Описание переменных типа"Запись" выполняется в следующем формате:

{Имя_переменной} Имя_записи <{Начальное_значение,. . .}>.

По этому описанию в памяти создается переменная типа "Запись" с распределением полей в соответствии с указанным шаблоном. Имя переменной связывается с началом записи. При отсутствии имени ассемблер распределяет

память в соответствии с шаблоном, но эта область памяти остается безымянной.

В угловых скобках < > указывается список начальных значений полей записи. Последовательность начальных значений должна строго соответствовать последовательности полей шаблона по размеру. Каждое начальное значение может быть числом или символьной константой. Для каждого поля может быть задано лишь одно начальное значение. Задавать начальные значения всех полей записи необязательно. Однако отсутствующие элементы в начале списка указываются запятыми. При отсутствии начального значения в описании переменной ее соответствующее поле инициализируется на значение, заданное в шаблоне записи. Если оно и там не определено, то это поле останется неопределенным и в переменной. Угловые скобки < > обязательны и в том случае, когда не задано ни одного начального значения.

***Пример 2.127.***

*Описать переменные типа "Запись" по шаблону* CLASS *из примера* 2.126.

```
Data        SEGMENT
      Cl1   CLASS      <2,5,25>
      Cl2   CLASS      <, ,31>
      Cl3   CLASS      < >
Data        ENDS
```

Это описание вызывает распределение и инициализацию ячеек памяти в соответствии с рис. 2.22.

Если суммарная длина полей шаблона записи не равна 8 или 16 битов, то переменная выравнивается вправо с расширением до целого байта или слова нулевыми битами слева. Следовательно, младший бит последнего поля всегда является самым младшим битом записи.

При описании переменных типа "Запись" допускается использование оператора *DUP*.

***Пример 2.128.***

```
School      CLASS      100 DUP(< >)
```

Это описание вызовет 100-кратное дублирование шаблона CLASS из примера 2.126 в памяти, что соответствует созданию массива School элементов типа "Запись".

Атрибуты переменных типа "Запись" определяются обычным образом. Например, для примеров 2.127, 2.128 имеем:

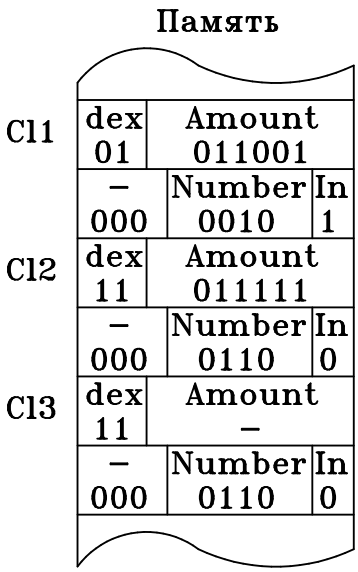


Рис. 2.22. Распределение и инициализация памяти под переменные типа " Запись"

*SEG* Cl1 = *SEG* Cl2 = *SEG* Cl3 = Data;
*OFFSET* Cl1 = 0,*OFFSET* Cl2 = 2,*OFFSET* Cl3 = 4;
*TYPE* Cl1 = *TYPE* Cl2 = *TYPE* Cl3 = *TYPE* School = 2;
*LENGTH* School = 100; *SIZE* School = 200.

Имена полей записи могут использоваться в качестве операндов операционных команд либо самостоятельно, либо совместно с директивами ассемблера *MASK* или *WIDTH*.

Директива *MASK* возвращает маску, необходимую для выделения указанного поля с помощью операции конъюнкции.

Директива *WIDTH* возвращает общую длину указанного шаблона или длину отдельного поля переменной типа "Запись" в битах.

При самостоятельном использовании имени поля в качестве операнда его значением является число, равное количеству сдвигов, необходимых для правого выравнивания указанного поля.

***Пример 2.129.***

Для шаблона записи и переменных из примеров 2.126 и 2.127 значения операндов с использованием имен полей и шаблона будут равны:

Number = 9, *MASK* Number = 1E00h, *WIDTH* Number = 4;
Index = 6, *MASK* Index = 1C0h, *WIDTH* Index = 3;

Amount = 0, *MASK* Amount = 3Fh, *WIDTH* Amount = 6;
*WIDTH* CLASS = 13.

При обработке переменных типа "Запись" ее поля, как правило, выделяются и выравниваются вправо, а затем используются обычным образом.

***Пример 2.130.***

*Подготовить к обработке поле* Index *переменной* Cl1 *из примеров* 2.126, 2.127.

```
MOV   AX, Cl1          ; Чтение переменной
AND   AX, MASK Index   ; Выделение поля
MOV   Cl, Index        ; Выравнивание поля
SHR   AX, Cl           ; вправо
```

Представление данных в виде записей обеспечивает наиболее эффективное использование памяти, что снижает требования к ней в сложных системах обработки информации.

***Пример 2.131.***

*Сведения о сотрудниках фирмы включает в себя возраст, пол, семейное положение и количество детей. Вычислить количество детей у женщин моложе* 40 *лет, не состоящих в браке. Обеспечить максимальную эффективность использования памяти.*

Из требования эффективного использования памяти вытекает необходимость применения записей. Для представления заданной информации потребуется 4 поля: возраст, пол, семейное положение и количество детей. Для определения размера полей записи будем считать, что возраст сотрудника не может превышать 127 лет, его пол и семейное положение могут принимать два значения, а количество детей составляет не более 15.

С учетом этого размер поля возраста Age должен быть 7 битов, поля пола Sex – 1 бит, поля семейного положения Marr – 1 бит и поля количества детей Amount – 4 бита.

Тогда текст программы для решения поставленной задачи будет иметь вид:

```
        ; Описание нестандартных типов данных
        ; Информация о сотруднике
    WORKER   RECORD Age:7, Sex:1, Marr:1, Amount:4
        ; Описание переменных
Data            SEGMENT
      Firm      WORKER    200 DUP (< >)          ; Описание коллектива
      Sum       DW        ?                      ; Результат
Data            ENDS
                . . . .   . . . . . . .
      Start:    . . . .   . . . . . . .
        ; Программа заполнения массива Firm
          . . . . . . . .
        ; Вычисление количества детей
            MOV     DX, 0                ; Очистка накопителя
            MOV     SI, 0                ; и индекса
            MOV     CH, LENGTH Firm      ; Загрузка счетчика циклов
  Next:     MOV     AX, Firm[SI]         ; Чтение
            MOV     BX, AX               ; и копирование записи
            TEST    AX, MASK Sex         ; Проверка пола
            JNZ     Cont                 ; Переход, если не женщина
            TEST    AX, MASK Marr        ; Проверка семейного положения
            JNZ     Cont                 ; Переход, если состоит в браке
            AND     AX, MASK Age         ; Анализ возраста
            MOV     CL, Age
            SHR     AX, CL
            CMP     AX, 40
            JGE     Cont                 ; Переход, если не моложе 40 лет
            AND     BX, MASK Amount      ; Подсчет количества детей
            ADD     DX, BX
  Cont:     ADD     SI, TYPE Firm        ; Модификация индекса
            DEC     CH                   ; и счетчика циклов
```

```
JNZ     Next          ; Переход, если не все элементы
MOV     Sum, DX       ; Запись результата
```

В рассмотренном примере предполагалось, что поля пола Sex и семейного положения Marr имеют следующее кодирование:

| | | | |
|---|---|---|---|
| Sex: | 0 – женский пол, | 1 | – мужской пол; |
| Marr: | 0 – не состоит в браке, | 1 | – состоит в браке. |

## 2.17. Макрокоманды

Макрокоманды являются мощным средством для упрощения программирования на ассемблере. Макрокоманда представляет собой мощную команду, создаваемую программистом для реализации некоторых функций. В ее состав входят обычные команды МП, определяющие выполняемые действия. В качестве макрокоманд, как правило, оформляются повторяющиеся программные фрагменты.

### 2.17.1. Макроопределения и макровызовы

Для создания макрокоманды необходимо дать макроописание выполняемых ею действий. Для этого служат директивы макроопределений *MACRO* и *ENDM*.

Формат макроопределения имеет вид:

```
Имя_макрокоманды MACRO {Формальный параметр,. . .}
                 . . . . . . . . .      ; Тело макрокоманды
                 . . . . . . . . .      ; (описание выполняемых
                 . . . . . . . . .      ; действий)
                 ENDM
```

Имя макрокоманды должно быть уникальным, т. к. оно будет использоваться в программе для ее вызова.

Формальные параметры представляют собой внутренние по отношению к данному макроопределению имена, которые служат для передачи различных значений в макрокоманду при ее вызове.

Тело макроопределения представляет собой любой текст на ассемблере, описывающий необходимые действия.

Макроопределения всегда приводятся в самом начале сегмента программного кода (перед описанием подпрограмм). После этого макрокоманда может быть вызвана простым упоминанием ее имени в любой точке исполняемой части программы. Формат макровызова имеет вид:

Имя_макрокоманды {Фактический_параметр,. . .}

Фактические параметры могут быть числовыми значениями, именами переменных, мнемониками команд и т. д. и т.п. Последовательность фактических параметров в макрокоманде должна строго соответствовать последовательности формальных параметров в ее макроопределении. В процессе трансляции макрокоманды фактические значения из ее макровызова прежде всего подставляются вместо соответствующих формальных параметров в ее макроопределении, а затем формируется машинный код. Эта подстановка осуществляется в текстовой форме и, следовательно, после нее должны получаться синтаксически корректные программные строки.

***Пример 2.132.***

*Создать макрокоманду для вычисления* S = A + B, *где* A, B – *некоторые слова в памяти.*

Макроопределение этой команды будет иметь вид:

```
AddW_Mc  MACRO  S, A, B
         MOV    AX, A          ; Пересылка операнда
         ADD    AX, B          ; Сложение
         MOV    S, AX          ; Пересылка результата
         ENDM
```

Эта макрокоманда используется путем ее макровызовов следующего вида:

```
AddW_Mc    S1, A1, B1       ; Вычисление S1 = A1 + B1
AddW_Mc    S2, A2, B2       ; Вычисление S2 = A2 + B2
AddW_Mc    Z, 10, Alfa      ; Вычисление Z = Alfa + 10
AddW_Mc    AX, DX, 5        ; Вычисление (AX) = (DX) + 5
```

Если фактических значений в макровызове больше, чем формальных параметров в макроопределении, то лишние фактические параметры игнорируются. Если же фактических параметров меньше, чем формальных, то неопределенные формальные параметры принимаются равными нулю, но при этом выдается предупреждающее сообщение об отсутствии данных.

Для контроля за заданием фактических параметров и определения способа обработки их отсутствующих значений могут использоваться директивы условной трансляции *IFB*, *IFNB*. Эти директивы записываются в следующем формате:

*IFB* <Аргумент>;
*IFNB* <Аргумент>.

Они проверяют значение указанного аргумента, рассматриваемого как строка символов, и вызывают трансляцию нижеследующего программного фрагмента, если аргумент является пробельной строкой (*IFB*) или отличен от нее (*IFNB*). В качестве аргумента задается один из формальных параметров

макроопределения. Угловые скобки < > обязательны. Зона влияния директив *IFB* и *IFNB* ограничивается директивой *ENDIF*.

***Пример 2.133.***

*Создать макрокоманду для считывания полного имени человека.*

Полное имя человека включает в себя собственно имя First_name, отчество Middle_name и фамилию Last_name. Следовательно, макроопределение для считывания полного имени в общем случае должно иметь вид:

```
GetName  MACRO First_name, Middle_name, Last_name
                                    ; Эти команды считывают
                                    ; собственно имя
         IFNB     <Middle_name>
         . . . . . . . . . . .      ; Эти команды считывают
         . . . . . . . . . . .      ; отчество (при его наличии)
         ENDIF
         . . . . . . . . . . .      ; Эти команды считывают
         . . . . . . . . . . .      ; фамилию
         ENDM
```

В рассмотренном примере учтено, что отчество не является обязательной частью имени. Поэтому команды для его считывания помещены между директивами условной трансляции и при отсутствии отчества пропускаются.

При трансляции программы ассемблер заменяет имя макрокоманды машинными кодами той последовательности команд, которые указаны в ее макроопределении. Фрагмент машинного кода, полученного путем этой замены, называется макрорасширением.

В общем случае макроопределение может содержать некоторые метки. В случае многократных макровызовов такой макрокоманды метки также будут определены в памяти многократно, что недопустимо.

Для устранения этого противоречия в ассемблере используется директива *LOCAL*. Эта директива записывается в формате:

*LOCAL*  Имя_метки{,Имя_метки . . .}

и обеспечивает локализацию указанных меток исключительно в одном макрорасширении. Директива *LOCAL* обязательно должна следовать сразу же за строкой с директивой *MACRO*. Между ними недопустимы даже пробельные строки. Использование директивы *LOCAL* приводит к присвоению меткам в каждом макрорасширении уникального имени вида ??N, где N – четырехразрядное 16-ричное число.

***Пример 2.134.***

*Создать макроопределение для реализации временной задержки.*

Программная реализация временной задержки сводится к загрузке в регистр некоторого числа и его декрементированию до нулевого значения. Это требует использования метки, и макроопределение будет иметь вид:

```
Delay   MACRO   Time
        LOCAL   M               ; Локализация метки
        MOV     CX, Time        ; Загрузка интервала
M:      LOOP    M               ; Временная задержка
        ENDM
```

Макровызовы для реализации задержки осуществляются следующим образом:

```
Delay   100     ; Задержка на 100 дискрет
Delay   0       ; Задержка на 65536 дискрет
```

### 2.17.2. Сравнение макрокоманд и подпрограмм

По своему функциональному назначению макрокоманды аналогичны подпрограммам. И те, и другие улучшают сопровождаемость и модификацию программы, а также сокращают длину ее исходного текста, однократно описывая повторяющиеся фрагменты.

Однако между макрокомандами и подпрограммами существует и принципиальная разница. Она заключается в том, что при использовании макрокоманды соответствующий ей машинный код подставляется в программу на этапе трансляции, дублируясь в памяти столько раз, сколько раз осуществляется ее макровызов. Машинный код подпрограммы подставляется в программу во время ее исполнения, располагаясь в памяти однократно. Следовательно, основным достоинством подпрограмм является сокращение затрат памяти, а недостатком – непроизводительные затраты времени на их вызов и возврат. Основным достоинством макрокоманд является высокое быстродействие из-за отсутствия непроизводительных затрат времени, а недостатком – увеличенный расход памяти.

Отсюда очевидно, что повторяющиеся фрагменты малого объема целесообразно оформлять в виде макрокоманд, а большого объема – в виде подпрограмм.

Дополнительным достоинством макрокоманд по сравнению с подпрограммами является высокая гибкость и простота передачи параметров. В качестве входных параметров в макрокоманды можно передать не только данные, но и выполняемые над ними действия, что невозможно для подпрограмм.

***Пример 2.135.***

*Создать макрокоманду для модификации адресов при обработке массивов.*

В общем случае в зависимости от вида обработки и типа элементов массивов модификация адресов выполняется путем увеличения или уменьшения содержимого определенного регистра на некоторое число.

С учетом этого макроопределение для решения поставленной задачи должно иметь вид:

```
Modif     MACRO Operation, Reg, Const
          Operation Reg, Const
          ENDM
```

Вся информация, необходимая для модификации адресов, передается в виде параметров. В этом случае макровызовы для модификации адресов записываются следующим образом:

```
Modif   ADD, BX, 2          ; (BX) = (BX) + 2
Modif   SUB, SI, 4          ; (SI) = (SI) – 4
Modif   ADD, DI, TYPE Arr   ; (DI) = (DI) + TYPE Arr
```

Макроопределения могут быть сведены в библиотеку, из которой они простейшим образом могут использоваться в дальнейших разработках.

### 2.17.3. Макрокоманды повторения

Макрокоманды повторения обеспечивают дублирование тела макроопределения по текущему адресу трансляции. К ним относятся макрокоманды *REPT*, *IRP* и *IRPC*.

Формат макроопределения *REPT* имеет вид:

```
REPT      Числовое_выражение
. . . . . .          ; Тело макроопределения
. . . . . .          ; (описание выполняемых
. . . . . .          ; действий)
ENDM
```

Количество повторений тела макроопределения в этом случае определяется значением числового выражения.

***Пример 2.136.***

```
REPT    3               ; Эквивалентно
SHR     AX, 1           ; SHR AX, 1; SHR AX, 1; SHR AX, 1
ENDM
```

Формат макроописания *IRP* имеет вид:

```
IRP       Фиктивный_параметр, <Список значений>
. . . . . . . . . .     ; Тело макроопределения
. . . . . . . . . .     ; (описание выполняемых
ENDM                    ; действий)
```

Количество повторений тела макроопределения в этом случае определяется количеством элементов списка. При каждом повторении вместо фиктивно-

го параметра в тело макроопределения подставляется очередной элемент списка значений.

***Пример 2.137.***

```
IRP     reg, <AX,BX,CX,DX>   ; Эквивалентно
PUSH    reg                  ; PUSH AX, PUSH BX,
ENDM                         ; PUSH CX, PUSH DX
```

Формат макроописания *IRPC* имеет вид:

```
IRPC    Фиктивный_параметр, Строка_символов
. . . . . . . . . . . .      ; Тело макроопределения
. . . . . . . . . . . .      ; (описание выполняемых
. . . . . . . . . . . .      ; действий)
ENDM
```

Количество повторений тела макроопределения в этом случае определяется количеством символов в строке. При каждом повторении вместо фиктивного параметра в тело макроопределения подставляется очередной символ строки.

***Пример 2.138.***

```
IRPC    Char, ABCD           ; Эквивалентно
SHL     Char&X, 1            ; SHL AX, 1; SHL BX, 1; SHL CX, 1;
ENDM                         ; SHL DX, 1
```

В этом примере использован оператор конкатенации – &. Он обеспечивает сцепление двух строк в одну и записывается в формате:

Строка1 & Строка2

В рассмотренном примере оператор & сцепляет значение фиктивного параметра Char с неизменяемой строкой X, образуя имена регистров.

## 2.17.4. Библиотеки макроопределений

Библиотека макроопределений представляет собой обычный текстовый файл, содержащий макроопределения, и может быть создана любым текстовым редактором.

В библиотеку включаются макроопределения для их использования в различных программах. При разработке макроопределений целесообразно выполнять следующие рекомендации:

1) тщательно документируйте макроопределения, включая в них как можно больше комментариев;

2) сохраняйте все используемые регистры в начале макроопределения и восстанавливайте их в его конце;

3) включайте все комментарии в макроопределения с помощью оператора макрокомментария **;;**, который должен предшествовать им.

Использование оператора **;;** исключает комментарии в расширениях макроопределений, оставляя их лишь в макроописаниях. Это уменьшает расход памяти для хранения программы и ускоряет ее трансляцию.

Для использования некоторого библиотечного макроопределения в прикладной программе оно должно быть включено в исходный текст этой программы. Это осуществляется путем подключения всей библиотеки с помощью директивы *INCLUDE*, записываемой в формате:

*INCLUDE* Имя_библиотеки

Эта директива считывает указанный файл и помещает его в транслируемую программу по текущему адресу трансляции. Как правило, директива *INCLUDE* размещается в начале исходного текста программы.

При трансляции программы ассемблер дважды просматривает ее текст. На первом проходе осуществляется расширение всех макроопределений и определяются все идентификаторы. На втором проходе генерируется машинный код программы и формируется ее листинг.

Для исключения считывания библиотеки макроопределений на втором проходе (с целью сокращения времени трансляции программы) директива *INCLUDE* помещается в скобки условной трансляции *IF1*, *ENDIF* и записывается в виде:

```
IF1
    INCLUDE     Имя_библиотеки
ENDIF.
```

В этом случае библиотека будет считываться только на первом проходе, причем из нее будут взяты лишь тела макроопределений, а текст в листинг программы не попадет, т. к. он формируется на втором проходе.

При подключении библиотеки директивой *INCLUDE* вся библиотека считывается полностью. Однако она может содержать множество макроопределений, не используемых в данной программе. В результате память заполняется ненужной информацией, что может привести к ее переполнению. Для освобождения памяти необходимо удалить неиспользуемые макроопределения. Это обеспечивается с помощью директивы *PURGE*, помещенной непосредственно после *INCLUDE*, в которой указываются имена удаляемых макроопределений.

***Пример 2.139.***

```
IF1
        INCLUDE  MACROS.lib      ; Считывание библиотеки
        PURGE    Mac5, Mac8      ; с удалением неиспользуемых
ENDIF                            ; макроопределений.
```

# 3. ТЕХНОЛОГИЯ ПОДГОТОВКИ ПРОГРАММ

После разработки любой программы она должна быть подготовлена к исполнению. В настоящее время процесс подготовки полностью опирается на использование персональных компьютеров в качестве инструмента. При этом подготовка программы включает в себя следующие этапы:

1) создание исходного файла, содержащего текст программы;
2) трансляция программы;
3) компоновка программы;
4) отладка программы.

Для реализации этих этапов персональный компьютер должен иметь соответствующие программные средства. Полная схема подготовки программы приведена на рис. 3.1. Рассмотрим выполнение ее отдельных этапов.

## 3.1. Создание исходного файла программы

Для создания исходного файла может использоваться любой текстовый редактор. В результате на дисковом накопителе компьютера создается файл исходного текста программы, которому, как правило, присваивается имя *.asm.

## 3.2. Трансляция программы

Трансляция программы осуществляется с помощью ассемблера MASM. В результате трансляции могут быть получены три файла: объектный файл *.obj, файл листинга *.lst и файл перекрестных ссылок *.crf. При этом объектный файл создается всегда, а файлы листинга и перекрестных ссылок – по желанию программиста.

Объектный файл содержит машинные коды исходной программы в перемещаемом формате. В этом файле адреса всех операндов в памяти вычислены исходя из предположения, что каждый сегмент программы начинается со смещения, равного нулю. Однако при объединении частных программных сегментов в общий сегмент все присоединенные сегменты будут иметь ненулевые начальные смещения. В этом случае адреса всех находящихся в них операндов должны быть изменены. Процесс изменения адресов в программе соответствует ее перемещению. Перемещение программы осуществляется на этапе ее компоновки с помощью редактора связей LINK.

Для программиста основную информацию о программе содержит листинг. В его правой части приводится исходный текст программы, а в левой – результат его трансляции (адреса и коды команд в 16-ричной форме). Перемещаемые адреса в листинге помечаются буквой R. В заключительной части листинга приводится таблица символьных имен, использованных в программе. При этом для сегментов указывается их объем в байтах, а для переменных и меток – все их атрибуты. Завершается листинг сообщением о количестве грубых

ошибок и предупреждений. В точках возникновения ошибок в листинге имеются соответствующие поясняющие сообщения.

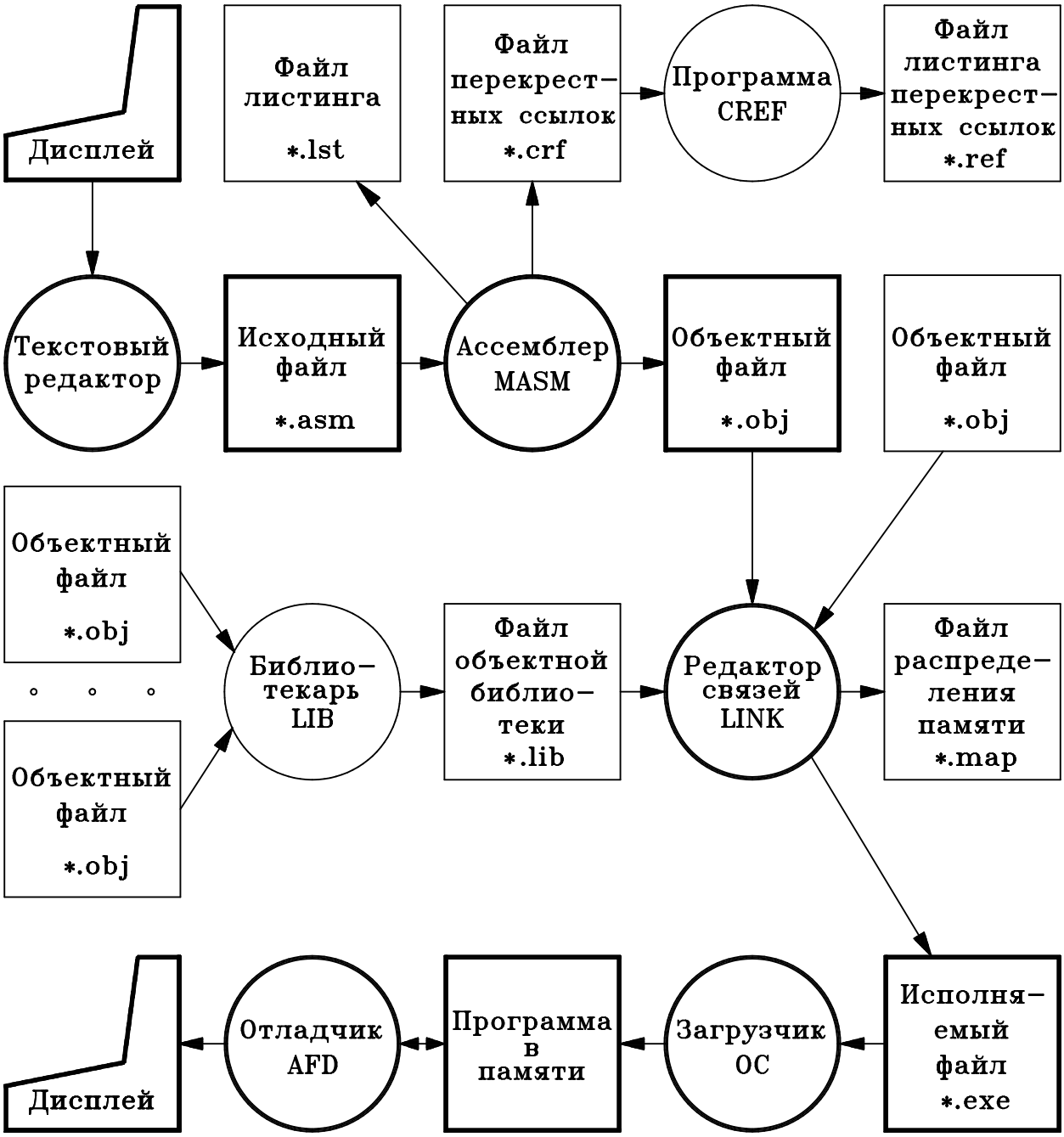


Рис. 3.1. Схема подготовки ассемблерной программы

Файл перекрестных ссылок содержит информацию о номерах строк, в которых определен и используется каждый идентификатор программы. Однако эта информация представлена во внутренней машинной форме, недоступной для чтения. Для ее преобразования в текстовую форму файл *.crf необходимо обработать программой CREF. В результате этого создается файл *.ref, содержащий всю информацию о перекрестных ссылках в текстовой форме. При этом

номер строки, в которой идентификатор определен, помечается символом #, а все остальные числа являются номерами строк, в которых имеются ссылки на него. Листинг перекрестных ссылок позволяет контролировать положение каждого идентификатора в программе, что упрощает ее отладку.

Формат командной строки вызова ассемблера для трансляции программы имеет вид:

MASM {Имя_исходного_файла},{Имя_объектного_файла},
{Имя_файла_листинга},{Имя_файла_перекрестных_ссылок},{/Ключи}{;}

На практике ассемблер, как правило, вызывается следующим образом:

1) MASM – в этом случае имена всех входных и выходных файлов запрашиваются в диалоговом режиме;

2) MASM Имя_исходного_файла,,,, – в этом случае входной диалог отсутствует, и транслируется указанный файл *.asm с формированием всех выходных файлов *.obj, *.lst и *.crf;

3) MASM Имя_исходного файла; – в этом случае входной диалог также отсутствует, и транслируется указанный файл *.asm с формированием только объектного файла *.obj.

Ключи командной строки позволяют уточнить режим трансляции программы. Важнейшими из них являются:

D – поместить в листинг диагностические сообщения после первого прохода текста программы; это дает более глубокую диагностику исходного текста, т. к. многие ошибки первого прохода исправляются на втором и в листинг без этого ключа не попадают;

ML – установить различие между строчными и прописными буквами в идентификаторах программы; при отсутствии этого ключа строчные буквы автоматически преобразуются в заглавные.

***Пример 3.1.***

*Транслировать исходный файл* SORT.asm

| | | |
|---|---|---|
| MASM | SORT,,,, | – трансляция с созданием файлов SORT.obj, SORT.lst и SORT.crf. |
| MASM | SORT; | – трансляция с созданием только файла SORT.obj. |
| MASM | SORT/D; | – трансляция с созданием только файла SORT.obj с глубокой диагностикой исходного текста на ошибки. |

## 3.3. Компоновка программы

Объектный файл *.obj, полученный в результате трансляции, не может исполняться, т. к. содержит перемещаемые адреса операндов. Для получения исполняемого файла все эти адреса должны быть определены.

Перемещаемые адреса операндов определяются на этапе компоновки программы, выполняемой с помощью редактора связей LINK. В результате компоновки могут быть получены два файла: исполняемый файл *.exe и файл распределения памяти *.map. При этом исполняемый файл создается всегда, а файл распределения памяти – по желанию программиста.

Редактор связей может объединить в общий исполняемый файл множество объектных файлов, указываемых как по отдельности, так и в составе объектных библиотек, создаваемых библиотекарем LIB.

Исполняемый файл содержит машинный код программы, в котором определены все адреса, кроме начальных адресов логических сегментов. Эти адреса остаются перемещаемыми и определяются при загрузке программы в память.

Файл распределения памяти содержит полные сведения о логических сегментах программы. Для каждого сегмента указывается его имя, размер в байтах и класс. Для характеристики класса используются стандартные имена: Code – сегмент программного кода, Data – основной сегмент данных, Extra – дополнительный сегмент данных, Stack – сегмент стека.

Формат командной строки вызова редактора связей на компоновку программы имеет вид:

LINK {Имя_объектного_файла}, {Имя_исполняемого_файла}, {Имя_файла_
_распределения_памяти}, {Имя_файла_объектной_библиотеки} {/Ключи}{;}

Если при компоновке связываются несколько объектных модулей, то они указываются в первой позиции командной строки через знак +.

На практике редактор связей, как правило, вызывается следующим образом:

1) LINK – в этом случае имена всех входных и выходных файлов запрашиваются в диалоговом режиме;

2) LINK Имя_объектного_ файла,,,, – в этом случае входной диалог отсутствует и связывается указанный файл *.obj с формированием всех выходных файлов *.exe, *.map;

3) LINK Имя_объектного_файла1 + Имя_объектного_файла2; – в этом случае входной диалог также отсутствует и компонуются два входных файла *1.obj и *2.obj с формированием только исполняемого файла *.exe.

Ключи командной строки позволяют уточнить режим компоновки программы и используются достаточно редко.

## 3.4. Отладка программы

Для исполнения и отладки программы она должна находиться в памяти системы. Загрузка программы из дискового файла *.exe в память осуществляется загрузчиком операционной системы.

В зависимости от текущего состояния памяти загрузчик определяет начальные адреса сегментов программы и загружает их в сегментные регистры МП. В результате программа полностью привязывается к текущей области памяти. Вся информация, необходимая для привязки, хранится в заголовке программы, расположенном в начале исполняемого файла.

Отладка загруженной программы осуществляется с помощью какого-либо отладчика, позволяющего наглядно наблюдать состояние системы в различных режимах исполнения программы и вносить изменения в нее. Наиболее удобным является отладчик AFD, имеющий экранную форму представления информации и широкий набор функции [5].

# 4. МОДУЛЬНОЕ ПРОГРАММИРОВАНИЕ

При разработке сложных программ общая задача должна разбиваться на подзадачи, реализуемыми независимыми программными модулями. В качестве модулей, как правило, используются подпрограммы, вызываемые с макроуровня программы.

На этапе подготовки программы весь ее текст может быть помещен в один файл. В этом случае исходный файл сложной программы становится очень большим, что существенно усложняет его редактирование и трансляцию.

Для исключения этого недостатка необходимо каким-либо образом представить общую программу в виде совокупности отдельных файловых модулей небольшого размера с последующим их объединением в общий исполняемый файл программы. Эта идея лежит в основе модульного программирования.

## 4.1. Методы и проблемы модульного программирования

Модульное программирование может быть реализовано различными путями. Простейшим из них является использование библиотек макроопределений.

### 4.1.1. Реализация модульности за счет библиотек макроопределений

В этом случае некоторые фрагменты исходного текста программы оформляются в виде макроопределений и выносятся в отдельные файлы библиотек. В результате исходный файл программы представляется в виде совокупности нескольких файлов меньшего размера, что упрощает их редактирование.

Библиотеки макроопределений включаются в исходный текст программы с помощью директивы *INCLUDE*.

Подобным образом можно выделить в отдельный файл с последующим его подключением к исходному файлу не только макроопределения, но и любой другой фрагмент программы.

Недостатком такого подхода является то, что выделенные файлы подключаются к исходному файлу и транслируются вместе с ним при каждой его трансляции. В результате общее время трансляции программы в этом случае не уменьшается и для сложных программ может иметь недопустимые значения.

### 4.1.2. Реализация модульности за счет связывания объектных модулей

Дальнейшим развитием модульного программирования является представление всей программы в виде совокупности независимых файловых модулей с возможностью их раздельной трансляции. Объединение раздельно транс-

лированных модулей в общий исполняемый файл программы возможно с помощью редактора связей на этапе компоновки программы.

Однако при таком подходе возникают специфические проблемы объединения модулей. Они обусловлены следующими причинами:

1) все файловые модули могут иметь идентичную структуру, включая в себя логические сегменты данных, стека и программного кода, которые при их объединении должны образовать общие логические сегменты программы;

2) в любой программе между модулями существуют связи по данным и управлению и, следовательно, из одного модуля возможны обращения к поименованным операндам (переменным и меткам) из других модулей, а другие модули могут обращаться к аналогичным элементам в данном модуле.

При раздельной трансляции исходных файловых модулей ассемблер будет обнаруживать неопределенные имена, что вызовет, без принятия дополнительных мер, фиксацию множества ошибок.

Для решения этих проблем в ассемблер были введены директивы описания типа объединения логических сегментов связываемых файловых модулей, а также описания межмодульных обращений. При трансляции отдельных файловых модулей эта информация включается в их объектные файлы и используется редактором связей для определения всех адресов, которые были неизвестны ассемблеру.

Таким образом, подготовка многомодульной программы к исполнению включает в себя следующие этапы:

1) создание исходных файлов независимых программных модулей;

2) раздельная трансляция всех исходных файловых модулей;

3) объединение полученных объектных файлов отдельных модулей в общий исполняемый файл программы.

Исходные файлы модулей создаются обычным образом. Исходный текст всех объединяемых модулей должен завершаться директивой *END*. Но только в одном из них в директиве *END* может указываться стартовый адрес программы. Этот модуль считается главным и должен содержать макроуровень программы, а остальные модули являются подчиненными и должны присоединяться к главному модулю.

Трансляция модулей также осуществляется обычным образом. Однако объединяемые объектные модули могут быть получены не только ассемблированием с помощью MASM, но и компиляторами языков высокого уровня, например, Си или Паскаль. Это позволяет писать математически сложные программные модули на языках высокого уровня, а модули логического управления на ассемблере, что упрощает процесс разработки программ.

Объединение объектных модулей в общий исполняемый файл также осуществляется обычным образом с помощью редактора связей LINK.

***Пример 4.1.***

*Создать исполняемый файл программы, состоящей из трех исходных файловых модулей* X.asm, Y.asm *и* Z.asm, *которые могут транслироваться раздельно.*

Процесс подготовки такой программы будет иметь вид:

MASM X; – ассемблирование X.asm с формированием X.obj.

MASM Y; – ассемблирование Y.asm с формированием Y.obj.

MASM Z; – ассемблирование Z.asm с формированием Z.obj.

LINK X + Y + Z, PROG – объединение файлов X.obj, Y.obj и Z.obj с формированием исполняемого файла программы PROG.exe.

Для реализации модульного программирования необходимо рассмотреть директивы для объединения логических сегментов, а также для описания межмодульных обращений в связываемых файловых модулях.

**Объединение логических сегментов**

Предположим, что разрабатываемая программа состоит из двух файловых модулей MOD1.asm и MOD2.asm, каждый из которых содержит сегменты программного кода, данных и стека. Пусть эти сегменты соответственно имеют имена Code1, Data1, Stack1 и Code2, Data2, Stack2 и определены конструкцией:

Имя_сегмента *SEGMENT*

. . . . . .

Имя_сегмента *ENDS*

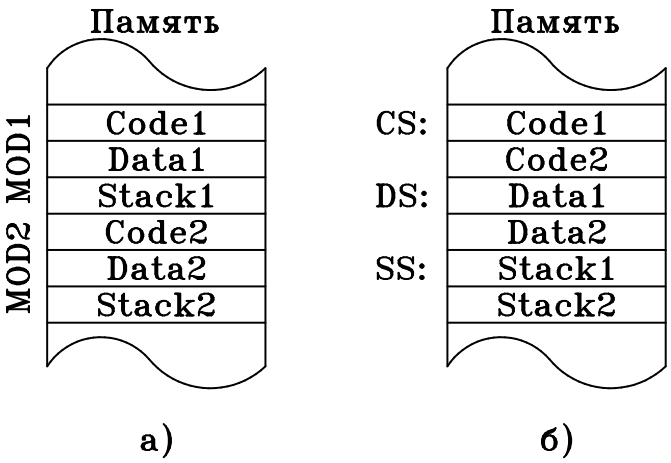


Рис. 4.1. Обобщенная схема объединения логических сегментов:
а) без описания типа объединения;
б) с описанием типа объединения

При объединении таких модулей редактором связей (LINK MOD1 + MOD2) размещение программы в памяти будет иметь вид, приведенный на рис. 4.1,а. При этом соответствующие логические сегменты обоих модулей не могут адресоваться при неизменном содержимом сегментных регистров. Поэтому при каждой передаче управления между модулями необходимо изменять регистр CS, а при обращении к данным из различных модулей – модифицировать регистр DS. Самым неприятным является то, что програм-

ма должна иметь один стек, и, следовательно, один из сегментов стека оказывается ненужным, а размер другого может оказаться недостаточным для работы полной программы.

Для устранения рассмотренных недостатков размещение программы в памяти должно иметь вид, приведенный на рис. 4.1,б. В этом случае адресация объединенных логических сегментов может осуществляться при неизменном содержимом сегментных регистров.

Возможны различные типы объединения логических сегментов. Описание типа объединения приводится в директиве *SEGMENT*. Полный формат конструкции описания логического сегмента имеет вид:

Имя_сегмента *SEGMENT* {Тип_выравнивания} {Тип_объединения} {'Имя_класса'}

. . . . . .

. . . . . .

Имя_сегмента *ENDS*

Параметр "Тип_выравнивания" определяет адресную границу, начиная с которой логический сегмент будет размещаться в памяти. Этот формальный параметр может принимать следующие основные фактические значения: *PARA*, *WORD* и *BYTE*.

Атрибут *PARA* указывает, что объединяемые логические сегменты должны начинаться с границы параграфа, т. е. с адреса, кратного 16. Поскольку размер параграфа равен 16 байтам, то в этом случае между сегментами может оказаться от 0 до 15 неиспользуемых байтов, что соответствует потерям памяти до 15 байтов для каждой границы между сегментами. Атрибут *PARA* может использоваться для любых логических сегментов и принят по умолчанию.

Атрибут *WORD* обеспечивает начало сегментов с четного адреса. Поскольку обращение к словам по четным адресам осуществляется быстрее, чем к словам по нечетным адресам, то этот атрибут наиболее целесообразно использовать для сегмента стека. Потери памяти в этом случае могут составлять до 1 байта для каждой границы между объединяемыми сегментами.

Атрибут *BYTE* указывает, что объединяемые логические сегменты могут начинаться с любого адреса. Этот атрибут соответствует наиболее плотному размещению логических сегментов в памяти, т. к. между ними не будет никаких промежутков. Атрибут *BYTE* рекомендуется использовать только для сегментов программного кода и не рекомендуется для сегментов данных и стека, т. к. это может привести к уменьшению быстродействия системы.

Параметр "Тип_объединения" определяет способ объединения одноименных логических сегментов связываемых файловых модулей. Сегменты с различными именами не объединяются, а при наличии одноименных сегментов без описания типа объединения редактор связей зафиксирует ошибку. Это обусловлено тем, что сегмент без описания типа объединения рассматривается как независимая программная часть и не может объединяться ни с чем. Формаль-

ный параметр "Тип_объединения" может принимать следующие основные фактические значения: *PUBLIC*, *STACK*, *COMMON* и *AT*.

Атрибут *PUBLIC* обеспечивает объединение одноименных логических сегментов путем их конкатенации (сцепления). Порядок сцепления определяется последовательностью имен объектных файлов в команде вызова LINK. При этом смещения переменных или меток в первом из связываемых модулей остаются неизменными, а во всех последующих модулях корректируются с учетом суммарного размера ранее объединенных сегментов.

***Пример 4.2.***

*Объединить в общую программу модули* MOD1 *и* MOD2.

Пусть в исходных модулях описаны сегменты данных вида:

```
; В модуле MOD1.asm
PrData      SEGMENT  WORD  PUBLIC
      Dig   DB       10h, 11h, 12h, 13h
PrData      ENDS


; В модуле MOD2.asm
PrData      SEGMENT  WORD  PUBLIC
      BV1   DB       21h
      BV2   DB       ?
      BV3   DB       23h
PrData      ENDS
```

и редактор связей вызывается командной строкой LINK MOD1+MOD2,PROG; .

Тогда схема распределения памяти, соответствующая объединению этих модулей будет иметь вид, приведенный на рис. 4.2,а. Редактор связей скорректировал адреса переменных из логического сегмента PrData модуля MOD2 на значение, равное объему одноименного сегмента модуля MOD1. При этом общий размер объединенного сегмента PrData равен сумме размеров объединяемых сегментов.

Атрибут *PUBLIC* используется для объединения сегментов программного кода и сегментов данных.

Атрибут *STACK* обеспечивает объединение одноименных сегментов стека различных файловых модулей в один непрерывный сегмент суммарного размера. При этом смещения, соответствующие верхушкам пустых стеков обоих модулей, корректируются так, чтобы указывать на верхушку объединенного пустого стека.

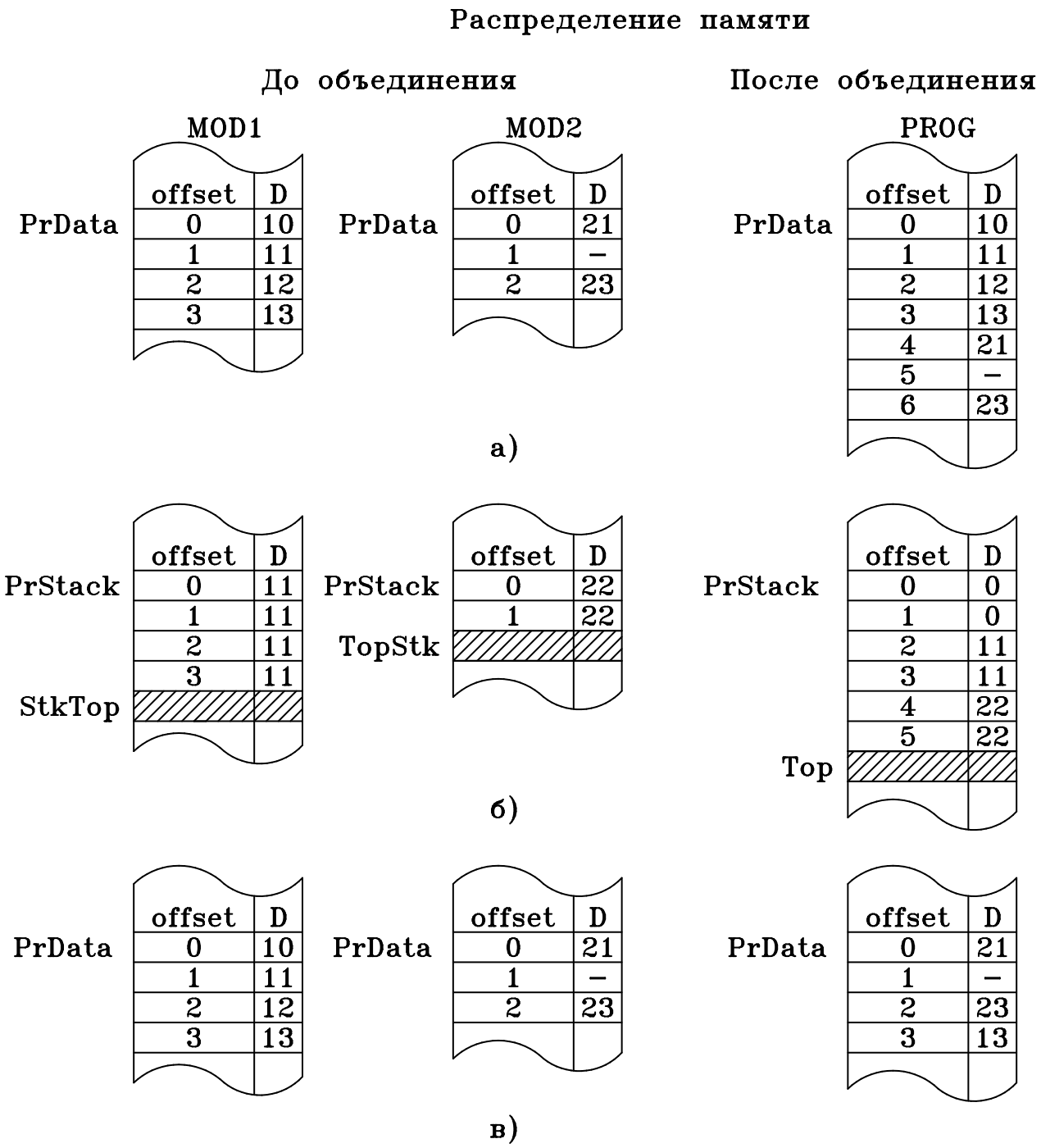


Рис. 4.2. Объединение логических сегментов:
а) по типу PUBLIC; б) по типу STACK; в) по типу COMMON

***Пример 4.3.***
*Объединить в общую программу модули* MOD1 *и* MOD2.
Пусть в исходных модулях описаны сегменты стека вида:

```
        ; В модуле MOD1.asm
PrStack         SEGMENT  WORD  STACK
                DW          2 DUP (1111h)
```

```
        StkTop  LABEL       WORD
PrStack         ENDS

      ; В модуле MOD2.asm
PrStack         SEGMENT  WORD  STACK
                DW          2222h
        TopStk  LABEL       WORD
PrStack         ENDS
```

и редактор связей вызывается командой LINK MOD1 + MOD2,PROG; .

При объединении этих сегментов образуется общий сегмент стека из 3 слов, и схема распределения памяти будет иметь вид, приведенный на рис. 4.2,б. Объединяемые сегменты стека обоих модулей располагаются так, чтобы они имели те же смещения относительно общей верхушки стека, что и в исходных модулях. В результате они накладываются друг на друга, и инициализируемые данные, объявленные последними, замещают ранее объявленные данные.

Смещения в командах LEA SP,StkTop и LEA SP,TopStk , загружающих верхушки пустых стеков в исходных модулях, автоматически корректируются так, что в SP загружается верхушка Top объединенного пустого стека.

Атрибут *COMMON* обеспечивает объединение одноименных логических сегментов путем их наложения друг на друга так, чтобы они имели один и тот же начальный адрес. Размер общего сегмента равен размеру максимального из объединяемых сегментов. Если данные инициализированы более чем в одном сегменте, то данные, объявленные последними, замещают все предыдущие.

***Пример 4.4.***

*Объединить в общую программу модули* MOD1 *и* MOD2.

Пусть в исходных модулях описаны сегменты данных вида:

```
      ; В модуле MOD1.asm
PrData          SEGMENT WORD COMMON
        Dig     DB          10h,11h,12h,13h
PrData          ENDS


      ; В модуле MOD2.asm
PrData          SEGMENT WORD COMMON
        BV1     DB          21h
        BV2     DB          ?
        BV3     DB          23h
PrData          ENDS
```

и редактор связей вызывается командной строкой LINK MOD1+MOD2,PROG; .

Схема распределения памяти, соответствующая объединению этих модулей, будет иметь вид, приведенный на рис. 4.2,в.

Атрибут *COMMON* используется для объединения сегментов данных при организации оверлейных программ. При этом переменные и метки из одноименных сегментов всех файловых модулей будут иметь одинаковые адреса.

Во всех вышерассмотренных случаях описания сегментов они являются перемещаемыми, и их начальный адрес определяется загрузчиком операционной системы во время загрузки исполняемого файла программы в память. Однако, часто требуется разместить сегмент по фиксированному адресу. Это необходимо, например, для обращения к фиксированным областям памяти (таблице векторов прерываний, буферу экрана, ПЗУ или ОЗУ в МПС).

Для задания абсолютного адреса сегмента служит тип объединения *AT*, записываемый в формате:

*AT* Числовое_выражение

Значение числового выражения определяет начальный адрес логического сегмента, что обеспечивает его фиксацию в адресном пространстве. Числовое выражение может быть представлено любой корректной константой без ссылок вперед.

Как правило, тип объединения *AT* используется для сегментов данных или стека. Такой сегмент содержит описание переменных, имена которых используются в сегменте программного кода для обращения к соответствующим ячейкам памяти.

Особенностью директивы *AT* является то, что редактор связей LINK не помещает в соответствующий ей сегмент инициализированные данные или программный код. Поэтому идентификаторы сегмента *AT* используются только для обращения к ячейкам памяти, а их инициализация должна осуществляться каким-либо другим, например, программным путем. Это особенно характерно для МПС, не имеющих операционной системы и требующих программной инициализации всех необходимых ячеек памяти данных.

Для задания абсолютного смещения offset внутри сегмента служит директива *ORG*, записываемая в формате:

*ORG* Числовое_выражение

Значение числового выражения определяет абсолютное смещение относительно начала сегмента, в котором находится эта директива.

***Пример 4.5.***

*Обеспечить обращение к фиксированному сегменту по адресу* B800h *и к фиксированным ячейкам памяти в нем:*

```
Data        SEGMENT   AT  0B800h   ; Описание фиксированного сегмента
     V1     DW        5555h        ; Эти переменные не будут
```

```
        V2      DW      3333h       ; инициализированы и примут
                                    ; нулевые значения
                ORG     10h         ; Задание фиксированного смещения
                                    ; offset = 16
        V3      DB      ?           ; Описание переменных
        V4      DW      ?
Data            ENDS
Code            SEGMENT
                ASSUME  CS:Code, DS:Data
        Start:
                MOV     AX, Data    ; Инициализация DS
                NOV     DS, AX
                MOV     AX, V1      ; Эти регистры будут загружены
                MOV     DX, V2      ; нулевым значением
                MOV     V1, 5555h   ; Программная инициализация
                MOV     V2, 3333h   ; переменных
                MOV     V3, 22h
                MOV     V4, 4444h
Code            ENDS
                END     Start
```

Параметр "Имя_класса" определяет порядок следования логических сегментов в памяти. Сегменты одного класса размещаются в памяти один за другим. После их размещения аналогичным образом размещаются сегменты других классов. Все сегменты имеют класс. Сегменты, для которых класс не указан, считаются принадлежащими к классу с пустым именем. Количество сегментов одного класса неограничено, но их суммарный объем не должен превышать 64 килобайта. Для характеристики класса сегмента может использоваться любой идентификатор.

***Пример 4.6.***

*Обеспечить размещение сегментов программного кода и констант, а также данных и стека в смежных областях памяти.*

Для этого перечисленные сегменты должны описываться следующим образом:

```
Code            SEGMENT BYTE PUBLIC 'ROM'
                . . . . . . . . . . . . . . . .
Code            ENDS
Data            SEGMENT WORD PUBLIC 'RAM'
                . . . . . . . . . . . . . . . .
Data            ENDS
Stack           SEGMENT WORD STACK 'RAM'
                . . . . . . . . . . . . . . . .
```

```
Stack       ENDS
Const       SEGMENT BYTE PUBLIC 'ROM'
            . . . . . . . . . . . . . . .
Const       ENDS
```

При этом сегменты Code и Const будут размещены в области памяти с именем ROM, а сегменты Data и Stack – в области с именем RAM.

Другим способом управления последовательностью сегментов является использование ключа /A в командной строке вызова ассемблера MASM. В этом случае MASM размещает сегменты в объектном файле в алфавитном порядке.

**Описание межмодульных обращений**

Для описания межмодульных обращений используются директивы глобального объявления *PUBLIC* и *EXTRN*.

Директива *PUBLIC* служит для описания переменных или меток, определенных в данном файловом модуле, но к которым возможно обращение из других модулей. Ее формат имеет вид:

*PUBLIC* Имя{, Имя . . .}

Тип переменных или меток в директиве *PUBLIC* не указывается, т. к. они определены в этом же модуле и известны ассемблеру.

Директива *EXTRN* описывает внешние по отношению к данному файловому модулю переменные и метки, к которым имеются обращения из него. Ее формат имеет вид:

*EXTRN* Имя : Тип{, Имя : Тип . . .}

В качестве типа для переменных может указываться *BYTE*, *WORD* или *DWORD*, а для меток – *NEAR* или *FAR*.

Директивы *PUBLIC* и *EXTRN* рекомендуется размещать в начале файловых модулей для исключения проблем, связанных с обращением вперед.

***Пример 4.7.***

*Написать программу, состоящую из двух файловых модулей, в каждом из которых должно обеспечиваться вычисление* S = A + B *и* S = A – B, *где* A, B – *некоторые слова в памяти.*

Текст программы для решения этой задачи с учетом всех проблем модульного программирования будет иметь вид:

```
; файловый модуль MOD1.asm (главный)
            PUBLIC      A1,B1,S1,AddW,SubW      ; Описание внутренних
                                                ; операндов
            EXTRN       A2:WORD,B2:WORD,S2:WORD ; Описание
                                                ; внешних операндов
```

```
Data            SEGMENT   WORD  PUBLIC
        A1      DW        ?
        B1      DW        ?
        S1      DW        ?
Data            ENDS
Code            SEGMENT   BYTE  PUBLIC
                ASSUME    CS : Code, DS : Data
        AddW    PROC      NEAR                  ; Описание подпрограммы
                ADD       AX, DX                ; сложения
                RET
        AddW    ENDP
        SubW    PROC      NEAR                  ; Описание подпрограммы
                SUB       AX, DX                ; вычитания
                RET
        SubW    ENDP
        Start:  . . . .   . . . . . . .
                MOV       AX, A2                ; Вычисление
                MOV       DX, B2                ; S2 = A2 + B2
                CALL      AddW
                MOV       S2, AX
                . . . .   . . . . . . .
                MOV       AX, A1                ; Вычисление
                MOV       DX, B1                ; S1 = A1 – B1
                CALL      SubW
                MOV       S1, AX
Code            ENDS
                END       Start                 ; Конец главного модуля

    ; файловый модуль MOD2 . asm (подчиненный)
                PUBLIC    A2,B2,S2                ;Описание внутренних
                                                  ;операндов
                EXTRN     A1 : WORD, B1 : WORD,S1 : WORD   ; Описание
                EXTRN     AddW : NEAR, SubW : NEAR         ; внешних
                                                           ; операндов
Data            SEGMENT   WORD PUBLIC
        A2      DW        ?
        B2      DW        ?
        S2      DW        ?
Data            ENDS
Code            SEGMENT   BYTE  PUBLIC
                ASSUME    CS : Code, DS : Data
                . . .     . . . . . . .
```

```
                MOV     AX, A1          ; Вычисление
                MOV     DX, B1          ; S1 = A1 + B1
                CALL    AddW
                MOV     S1, AX
                . . .   . . . . . . .
                MOV     AX, A2          ; Вычисление
                MOV     DX, B2          ; S2 = A2 – B2
                CALL    SubW
                MOV     S2, AX
Code            ENDS
                END                     ; Конец подчиненного
                                        ; модуля
```

При раздельной трансляции этих модулей ассемблер не может полностью сформировать их машинный код, т. к. неизвестны адреса внешних обращений. Однако, зная тип внешних обращений, он зарезервирует необходимое количество байтов для описания этих операндов и внесет соответствующие пометки в объектные файлы. При связывании файловых модулей редактор связей просматривает эти пометки и, используя доступную ему информацию, определяет все необходимые адреса, подставляя их в зарезервированные области. Таким образом, создается общий исполняемый файл программы, в котором все адреса полностью определены.

Необходимо обратить внимание на то, что подчиненный модуль не имеет стартовой метки программы. Поэтому при его подключении к главному модулю в общий файл программы включаются все имеющиеся в нем команды, в том числе и команды описания подпрограмм. В результате в общем, программном файле команды, составляющие тело подпрограммы, могут оказаться между исполняемыми фрагментами главного и подчиненного модулей, что недопустимо. Отсюда следует, что в подчиненные модули целесообразно включать либо только описания подпрограмм, либо только исполняемые фрагменты программы.

## 4.2. Библиотеки объектных модулей

Библиотека представляет собой совокупность объектных модулей, объединенных в одном файле. Как правило, в библиотеку включаются модули, реализующие некоторые функции общего назначения, с целью их использования в последующих разработках. Эти модули оформляются в виде подпрограмм, и в результате их раздельной трансляции формируются соответствующие файлы *.obj.

### 4.2.1. Создание библиотек объектных модулей и операции над ними

Для создания библиотек из объектных модулей служит программа-библиотекарь LIB. Формат командной строки для ее вызова имеет вид:

LIB {Имя_старой_библиотеки} {Команда} {Имя_файла_листинга}
{,Имя_новой_библиотеки} {;}

Первый параметр командной строки указывает имя файла библиотеки, над которой будут выполняться заданные операции. По умолчанию используются файлы с расширением *.lib.

Второй параметр определяет операцию, выполняемую над библиотекой.

Третий параметр указывает имя файла листинга перекрестных ссылок библиотеки, который содержит два списка:

1) список общих идентификаторов, входящих в несколько модулей библиотеки с указанием их имен;

2) список модулей библиотеки с указанием содержащихся в них общих идентификаторов.

Четвертый параметр командной строки вызова LIB определяет имя файла выходной библиотеки. Библиотека с указанным именем будет создана путем копирования старой библиотеки, и все операции будут выполняться над ней. При отсутствии этого параметра все операции выполняются над старой библиотекой, но ее неизмененная копия сохраняется в файле *.bak.

Операции над библиотекой задаются с помощью команд библиотекаря LIB.

**Команды библиотекаря**

Команды LIB служат для создания и сопровождения библиотек, а также выполнения различных операций над ними:

1) добавление, удаление, замена, копирование и перемещение модулей;

2) объединение, тестирование и просмотр каталога библиотек.

Добавление модуля в библиотеку осуществляется командой

+ Имя_объектного файла

Модуль, находящийся в объектном файле, включается в библиотеку под именем, указанном в директиве *NAME*. Если эта команда задана с именем несуществующей библиотеки, то создается новая библиотека.

Удаление модуля из библиотеки осуществляется командой

– Имя_модуля

Замена модуля в библиотеке выполняется командой

–+ Имя_модуля

Модуль с указанным именем замещается модулем из объектного файла с тем же именем. При этом LIB сначала удаляет модуль из библиотеки, а затем ищет файл.

Копирование модуля из библиотеки осуществляется командой

*Имя_модуля

Модуль с указанным именем копируется из библиотеки в объектный файл с тем же именем и остается в библиотеке.

Перемещение модуля из библиотеки реализуется командой

–*Имя_модуля

Эта команда выполняется аналогично команде копирования с той лишь разницей, что после копирования модуль удаляется из библиотеки.

Объединение библиотек осуществляется командой

+ Имя_библиотеки

Содержимое указанной библиотеки добавляется в конец старой библиотеки. В этом случае нельзя опускать расширение имени добавляемой библиотеки, так как иначе указанное имя будет интерпретироваться как имя объектного файла.

Между знаками выполняемой операции и указываемыми именами не должно быть пробелов.

Тестирование библиотеки выполняется в случае задания в командной строке вызова LIB только имени библиотеки с последующим символом **;** , т. е. командой

LIB Имя_библиотеки**;**

Оно заключается в проверке корректности всех входов в библиотеку и находящихся в ней модулей. LIB автоматически тестирует все модули при добавлении их в библиотеку.

Для просмотра содержимого библиотеки необходимо получить ее каталог, который может быть помещен либо в файл, либо на экран дисплея.

Файл с каталогом библиотеки создается командой вида:

LIB Имя_библиотеки, Имя_библиотеки**.**dir

и представляет собой файл с указанным именем и расширением dir.

Для отображения каталога библиотеки на экране может использоваться команда

LIB Имя_библиотеки**,** con

Наряду с именами объектных модулей каталог содержит перечень тех имен каждого модуля, которые указаны в операторе *PUBLIC*. Это напоминает, какие имена должны использоваться в прикладной программе при ссылках на процедуры, переменные и константы, определенные в вызываемых ею модулях библиотеки.

На практике библиотекарь LIB, как правило, вызывается следующим образом:

1) LIB – в этом случае все входные параметры запрашиваются и вводятся в диалоговом режиме;

2) LIB OBJECT + MUL32**,,** – в этом случае модуль из файла MUL32.obj будет добавлен или создаст библиотеку OBJECT.lib;

3) LIB OBJECT,OBJECT.dir – в этом случае будет создан файл каталога библиотеки OBJECT.dir;

4) LIB OBJECT**,**con – отображение каталога библиотеки OBJECT.lib на экране.

### 4.2.2. Использование библиотек объектных модулей

Для использования подпрограммы из объектной библиотеки необходимо объявить ее в разрабатываемой программе внешней (с помощью директивы *EXTRN* и вызвать командой CALL.

Затем на этапе компоновки программы сообщить редактору связей LINK имя используемой объектной библиотеки. Редактор связей извлечет вызываемую процедуру из библиотеки и включит ее в состав программы так же, как если бы она была отдельным модулем.

Если в прикладной программе требуются объектные модули из нескольких различных библиотек, то все библиотеки указываются редактору связей (например, в ответ на приглашение Libraries:) через знак сложения +.

***Пример 4.8.***

*Создать библиотеку, содержащую подпрограммы для вычисления суммы* S = A + B *и разности* D = A – B *слов, и использовать их в прикладной программе.*

Для демонстрации модульного подхода к программированию представим каждую из подпрограмм в виде отдельного модуля, которые будут включены в библиотеку, а затем использованы из нее в прикладной программе. С учетом этого все файловые модули программы будут иметь вид:

```
        ; Модуль суммирования (файл ADDW.asm)
                NAME      AddW          ; Описание имени
                PUBLIC    AddW          ; Описание обращения
Code            SEGMENT   PARA  PUBLIC  ; Описание сегмента кода
                ASSUME    CS:Code
        AddW    PPOC      NEAR          ; Описание подпрограммы
```

```
                ADD     AX, DX          ; Выполнение сложения
                RET
        AddW    ENDP
Code            ENDS
                END                     ; Конец подчиненного модуля

        ; Модуль вычитания (файл SUBW.asm)
                NAME    SubW            ; Описание имени
                PUBLIC  SubW            ; Описание обращения
Code            SEGMENT PARA  PUBLIC    ; Описание сегмента кода
                ASSUME  CS:Code
        SubW    PROC    NEAR            ; Описание подпрограммы
                SUB     AX, DX          ; Выполнение вычитания
                RET
        SubW    ENDP
Code            ENDS
                END                     ; Конец подчиненного модуля

        ; Модуль основной программы (файл PROG.asm)
                NAME    PROG_AS         ; Описание имени
                EXTRN   AddW:NEAR,SubW:NEAR  ; Описание обращений
Data            SEGMENT                 ; Описание данных
        A       DW      5h
        B       DW      3h
        S       DW      ?
        D       DW      ?
Data            ENDS
Code            SEGMENT PARA PUBLIC     ; Описание сегмента кода
                ASSUME  CS:Code,DS:Data
        Start:  MOV     AX, Data        ; Инициализация
                MOV     DS, AX          ; сегментного регистра
                MOV     AX, A           ; Передача входных
                MOV     DX, B           ; параметров
                CALL    AddW            ; Вычисление S = A + B
                MOV     S, AX           ; Передача результата
                . . . .  . . . . . . .
                . . . .  . . . . . . .
                MOV     AX, A           ; Передача входных
                MOV     DX, B           ; параметров
                CALL    SubW            ; Вычисление D = A – B
                MOV     D, AX           ; Передача результата
Code            ENDS
                END     Start           ; Конец главного модуля.
```

Библиотека объектных модулей создается следующим образом:

1) трансляция файловых модулей ADDW.asm и SUBW.asm с помощью команд:

```
MASM    ADDW;    – Трансляция файла ADDW.asm
MASM    SUBW;    – Трансляция файла SUBW.asm,
```

в результате которой создаются объектные файлы ADDW.obj и SUBW.obj;

2) создание библиотеки с помощью команд:

```
LIB     ARITHM + ADDW,,  – Создание библиотеки ARITHM.lib с
                           включением в нее модуля ADDW.obj
LIB     ARITHM + SUBW,,  – Включение модуля SUBW.obj в
                           библиотеку ARITHM.lib.
```

В результате создается библиотека ARITHM.lib, содержащая оба модуля (ADDW и SUBW).

Подготовка основной программы к исполнению в этом случае осуществляется следующим образом:

1) трансляция файлового модуля PROG.asm с созданием объектного файла PROG.obj с помощью команды

```
MASM    PROG;
```

2) компоновка программы редактором связей с подключением библиотеки с помощью команды

```
LINK PROG, , , ARITHM.
```

При этом создается исполняемый файл прикладной программы PROG.exe, путем исполнения которого в отладчике легко убедиться в достоверности полученного результата.

## ЗАКЛЮЧЕНИЕ

В настоящем пособии рассмотрена большая часть аспектов программирования на языке ассемблера. Однако в связи с ограниченным объемом некоторая часть вопросов осталась за пределами рассмотрения. Это касается использования прерываний DOS и BIOS при программировании на ассемблере, директив управления листингом и т. д. и т.п. Все эти вопросы подробно рассмотрены в источниках, приведенных в списке литературы [6 – 9].

# СПИСОК РЕКОМЕНДУЕМЫХ ИСТОЧНИКОВ


1. Лямин Л.В. Макроассемблер MASM. – М.: Радио и связь, 1994.
2. Использование Turbo Assembler при разработке программ. – Киев: Диалектика, 1994.
3. Дао Л. Программирование микропроцессора 8088: Пер. с англ. – М.: Мир, 1988.
4. Лю Ю-Чжен, Гибсон Г. Микропроцессоры семейства 8086/8088. Архитектура, программирование и проектирование микрокомпьютерных систем: Пер. с англ. – М.: Радио и связь, 1987.
5 Шубин Н.А. Отладчик программ AFD. – Рыбинск: РАТИ, 1991.
6. Скэнлон Л. Персональные ЭВМ IBM PC и XT. Программирование на языке ассемблера: Пер. с англ. – М.:Радио и связь, 1989.
7. Абель П. Язык ассемблера для IBM PC и программирования: Пер. с англ. – М.: Высшая школа, 1992.
8. Джордейн Р. Справочник программиста персональных компьютеров типа IBM PC XT и AT: Пер. с англ. – М.:Финансы и статистика.
9. Нортон П. Персональный компьютер фирмы IBM и операционная система MS-DOS: Пер. с англ. – М.: Радио и связь, 1991.